

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

Факультет прикладної математики

Кафедра програмного забезпечення комп'ютерних систем

«До захисту допущено»

Завідувач кафедри

_____ Євгенія СУЛЕМА

«___» _____ 2023 р.

Дипломний проєкт

на здобуття ступеня бакалавра

**за освітньо-професійною програмою «Інженерія програмного
забезпечення мультимедійних та інформаційно-пошукових систем»**

спеціальності 121 Інженерія програмного забезпечення

на тему: «Вебзастосунок для бронювання переговорних кімнат офісу»

Виконала:

студентка IV курсу, групи КП-92

Музичук Марина Анатоліївна

Керівник:

Доцент кафедри ПЗКС, к.т.н., доцент,

Заболотня Тетяна Миколаївна

Консультант з нормоконтролю:

Доцент кафедри ПЗКС, к.т.н., доцент,

Онай Микола Володимирович

Рецензент:

Доцент кафедри ММСА ІІСА, к.т.н., доцент,

Дідковська Марина Віталіївна



Засвідчую, що у цьому дипломному
проєкті немає запозичень з праць інших
авторів без відповідних посилань.

Студентка _____

Київ – 2023 року

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет прикладної математики

Кафедра програмного забезпечення комп'ютерних систем

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 «Інженерія програмного забезпечення»

Освітньо-професійна програма «Інженерія програмного забезпечення
мультимедійних та інформаційно-пошукових систем»

ЗАТВЕРДЖУЮ

Завідувач кафедри

_____ Євгенія СУЛЕМА

«___» _____ 2022 р.

ЗАВДАННЯ

на дипломний проєкт студенту

Музичук Марині Анатоліївні

1. Тема проєкту «Вебзастосунок для бронювання переговорних кімнат офісу», керівник проєкту Заболотня Тетяна Миколаївна, доцент кафедри ПЗКС, к.т.н., доцент, затверджені наказом по університету від «31» травня 2023 р. №2107-С.
2. Термін подання студентом проєкту «16» червня 2023 р.
3. Вихідні дані до проєкту: див. Технічне завдання.
4. Зміст пояснювальної записки:
 - огляд існуючих програмних рішень;
 - обґрунтування вибору засобів розроблення;
 - структурно-алгоритмічна організація системи;
 - особливості реалізації вебзастосунку.
5. Перелік обов'язкового графічного матеріалу:
 - структура бази даних (креслення);
 - алгоритм перевірки переговорної кімнати на зайнятість (креслення);
 - архітектура вебзастосунку (плакат);
 - дерево проблем потенційних користувачів системи (плакат).

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв
Нормоконтроль	Онаї М.В., доцент		

7. Дата видачі завдання «31» жовтня 2022 р.

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1.	Вивчення літератури за тематикою проєкту	13.11.2022	
2.	Розроблення та узгодження технічного завдання	25.11.2022	
3.	Розроблення структури вебзастосунку	15.12.2022	
4.	Підготовка матеріалів першого розділу дипломного проєкту	30.12.2022	
5.	Розроблення дизайну сторінок та графічних елементів	03.02.2023	
6.	Підготовка матеріалів другого розділу дипломного проєкту	19.02.2023	
7.	Програмна реалізація вебзастосунку	10.03.2023	
8.	Тестування вебзастосунку	17.03.2023	
9.	Підготовка матеріалів третього розділу дипломного проєкту	30.03.2023	
10.	Підготовка матеріалів четвертого розділу дипломного проєкту	12.04.2023	
11.	Підготовка графічної частини дипломного проєкту	21.04.2023	
12.	Оформлення документації дипломного проєкту	26.05.2023	

Студент

Марина МУЗИЧУК

Керівник проєкту

Тетяна ЗАБОЛЮТНЯ

АНОТАЦІЯ

Даний дипломний проєкт присвячений створенню вебзастосунку для бронювання переговорних кімнат офісу.

Вебзастосунок реалізовано у вигляді вебсайту, призначеного для користування працівниками компаній, які мають офісні приміщення та прагнуть автоматизувати процес створення та відстеження бронювань переговорних кімнат. Авторизувавшись у даному сервісі, користувачі отримають доступ не лише до створення, редагування та скасування власних зустрічей, але й матимуть змогу переглядати інформацію про бронювання інших організаторів та надсилати їм електронні листи. Крім цього, користувач з правами адміністратора матиме доступ до спеціальної панелі, де можна управляти різними сутностями, такими як: години роботи офісу, поверх, кімната, обладнання, ціль зустрічі, користувач та команда.

Що стосується створення бронювання, він має обмеження по часу: мінімальна його тривалість – 15 хвилин, а максимальна – та, яка досягає початку наступної зустрічі у цьому приміщенні або верхньої межі роботи офісу.

Метою даного дипломного проєкту є забезпечення ефективного використання переговорних кімнат, уникнення конфліктів зустрічей при бронюванні приміщення, шляхом розроблення відповідного програмного забезпечення.

У даному дипломному проєкті розроблено: архітектуру серверної та клієнтської частин системи, інтерфейс користувача у вигляді вебзастосунку, модуль для створення бронювань, панель адміністратора, спроектовано базу даних, а також алгоритм перевірки переговорної кімнати на зайнятість у певний період часу.

ABSTRACT

This diploma project deals with the development of an eb application for booking office meeting rooms.

The web application is implemented as a website intended for use by employees of companies that have office premises and seek to automate the process of creating and tracking meeting room reservations. Authenticating to this service provide user an ability not to create, edit and cancel their own meetings, but also allows able to view booking information from other organizers and send them emails. In addition, users with administrative rights will have access to a special panel where they can manage various entities such as: office hours, floor, room, asset, meeting purpose, user, and business unit.

As for creating a booking, it has a time limit: its minimum duration is 15 minutes, and the maximum duration is the one that reaches the start of the next meeting in this room or the upper limit of office hours.

The goal of this diploma project is to create a software product that would help office visitors to use office meeting rooms more efficiently and manage their reservations.

The purpose of this diploma project is to ensure the effective use of conference rooms, to avoid meeting conflicts when booking a room, by developing the appropriate software.

While working on this project, the following artefacts have been developed: both, client and server architecture; user web interface, booking module, admin panel, database and an algorithm for checking the meeting room for occupancy in a certain period of time.

ДП.045440-01-90. Вебзастосунок для бронювання переговорних кімнат офісу. Відомість проєкту

Позначення	Найменування	Кіл-ть	Примітка
	Документація проєкту		
ДП.045440-02-91	Вебзастосунок для	5	
	бронювання переговорних		
	кімнат офісу. Технічне		
	завдання		
ДП.045440-03-81	Вебзастосунок для	81	
	бронювання переговорних		
	кімнат офісу. Пояснювальна		
	записка		
ДП.045440-04-51	Вебзастосунок для	4	
	бронювання переговорних		
	кімнат офісу. Програма та		
	методика тестування		
ДП.045440-05-34	Вебзастосунок для	18	
	бронювання переговорних		
	кімнат офісу. Керівництво		
	користувача		
ДП.045440-06-99	Вебзастосунок для	1	
	бронювання переговорних		
	кімнат офісу. Структура		
	бази даних. ERD-діаграма		

[illegible]

Факультет прикладної математики
Кафедра програмного забезпечення комп'ютерних систем

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Євгенія СУЛЕМА

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНОК ДЛЯ БРОНЮВАННЯ ПЕРЕГОВОРНИХ

КІМНАТ ОФІСУ

Технічне завдання

ДП.045440-02-91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЗАБОЛОТНЯ

Нормоконтроль:

_____ Микола ОНАЙ

Виконавець:

_____ Марина МУЗИЧУК

ЗМІСТ

1. Найменування та галузь застосування.....	3
2. Підстава для розроблення.....	3
3. Призначення розробки.....	3
4. Вимоги до програмного продукту.....	3
5. Вимоги до проєктної документації	4
6. Етапи проєктування	5
7. Порядок тестування розробки	5

1. НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Вебзастосунок для бронювання переговорних кімнат офісу.

Галузь застосування: інформаційні технології.

2. ПІДСТАВА ДЛЯ РОЗРОБЛЕННЯ

Підставою для розроблення є завдання на дипломне проектування, затверджене кафедрою програмного забезпечення комп'ютерних систем Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського» (КПІ ім. Ігоря Сікорського).

3. ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для використання працівниками компаній, які мають офісні приміщення та прагнуть автоматизувати процес створення та відстеження бронювань переговорних кімнат. Користувачі матимуть доступ не лише до створення, редагування та скасування власних зустрічей, але й матимуть змогу переглядати інформацію про бронювання інших організаторів та надсилати їм електронні листи.

4. ВИМОГИ ДО ПРОГРАМНОГО ПРОДУКТУ

Вебзастосунок повинен забезпечувати такі основні функції:

- 1) можливість динамічного оновлення вмісту вебсторінок;
- 2) можливість створення, редагування, скасування власних одноразових та регулярних бронювань;
- 3) можливість надсилання електронного листа організатору іншої зустрічі;

- 4) можливість отримання електронних листів про запрошення на зустріч, її редагування та скасування;
- 5) доступ до панелі адміністратора користувачам з відповідними правами.

Додаткові вимоги:

- 1) можливість імпорту та експорту CSV-файлу з інформацією про користувачів системи;
- 2) зберігання паролів користувачів у захешованому вигляді;
- 3) відображення бронювань у таблиці завантаженості кімнат відповідним кольором команди, що запрошена на зустріч ;
- 4) дизайн сторінок з використанням в якості базових білого та сірого кольорів.

5. ВИМОГИ ДО ПРОЄКТНОЇ ДОКУМЕНТАЦІЇ

У процесі виконання проекту повинна бути розроблена наступна документація:

- 1) пояснювальна записка;
- 2) програма та методика тестування;
- 3) керівництво користувача;
- 4) креслення:
 - «Перевірка переговорних кімнат офісу на зайнятість. Схема алгоритму»;
 - «Структура бази даних. ERD-діаграма».

6. ЕТАПИ ПРОЄКТУВАННЯ

Вивчення літератури за тематикою роботи	16.11.2022
Розроблення та узгодження технічного завдання	27.11.2022
Розроблення структури вебзастосунку	15.12.2022
Розроблення дизайну сторінок та графічних елементів.....	03.02.2023
Програмна реалізація вебзастосунку	15.02.2023
Тестування вебзастосунку	07.04.2023
Підготовка матеріалів текстової частини проєкту	28.04.2023
Підготовка матеріалів графічної частини проєкту	12.05.2023
Оформлення технічної документації проєкту	25.05.2023

7. ПОРЯДОК ТЕСТУВАННЯ РОЗРОБКИ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

Факультет прикладної математики
Кафедра програмного забезпечення комп'ютерних систем

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Євгенія СУЛЕМА

“___” _____ 2023 р.

ВЕБЗАСТОСУНОК ДЛЯ БРОНЮВАННЯ ПЕРЕГОВОРНИХ
КІМНАТ ОФІСУ

Пояснювальна записка

ДП.045440-03-81

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЗАБОЛОТНЯ

Нормоконтроль:

_____ Микола ОНАЙ

Виконавець:

_____ Марина МУЗИЧУК

2023

ЗМІСТ

СПИСОК ТЕРМІНІВ, СКОРОЧЕНЬ ТА ПОЗНАЧЕНЬ	3
ВСТУП	6
1. ОГЛЯД ІСНУЮЧИХ ПРОГРАМНИХ РІШЕНЬ	7
1.1. Огляд проблеми, яка вирішується ПЗ	7
1.2. Аналіз існуючих рішень	8
1.3. Результати проведеного аналізу	17
1.4. Висновки до розділу 1	20
2. ОБГРУНТУВАННЯ ВИБОРУ ЗАСОБІВ РОЗРОБЛЕННЯ	21
2.1. Вибір мови програмування для реалізації серверної частини.....	21
2.2. Вибір технологій для реалізації клієнтської частини.....	28
2.3. Вибір СКБД.....	33
2.4. Висновки до розділу 2	38
3. СТРУКТУРНО-АЛГОРИТМІЧНА ОРГАНІЗАЦІЯ СИСТЕМИ	39
3.1. Опис вимог до розроблюваної системи	39
3.2. Опис архітектури системи.....	55
3.3. Опис структур даних системи.....	58
3.4. Опис реалізованих алгоритмів функціонування системи.....	60
3.5. Висновки до розділу 3	62
4. ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ВЕБЗАСТОСУНКУ	63
4.1. Особливості реалізації	63
4.2. Опис реалізованого користувацького інтерфейсу	65
4.3. Особливості проведеного тестування	73
4.4. Рекомендації щодо подальшого вдосконалення	75
4.5. Висновки до розділу 4	76
ВИСНОВКИ.....	77
СПИСОК ВИКОРИСТАНИХ ЛІТЕРАТУРНИХ ДЖЕРЕЛ.....	78
ДОДАТКИ.....	81

СПИСОК ТЕРМІНІВ, СКОРОЧЕНЬ ТА ПОЗНАЧЕНЬ

ПЗ – програмне забезпечення;

ІТ – Information Technology (інформаційні технології);

Букінг – Booking (бронювання);

Коворкінг – Co-working (колективний спосіб влаштування офісу у відкритому просторі)

СКБД – система керування базами даних (програма для доступу до бази даних);

БД – база даних (сукупність даних, організованих відповідно до концепції, яка описує характеристики цих даних і взаємозв'язки між елементами);

HTTP – HyperText Transfer Protocol (протокол передачі даних, що використовується в комп'ютерних мережах);

JVM – Java Virtual Machine (віртуальна машина Java, програма, призначена для виконання інших програм);

ООП – об'єктно-орієнтоване програмування;

Garbage Collector – збирання сміття, одна з форм автоматичного керування пам'яттю комп'ютера під час виконання програм;

AJAX – Asynchronous JavaScript And XML (технологія, яка дозволяє відправляти та отримувати дані з серверу без перезавантаження вебсторінки);

DOM – Document Object Model (стандартний спосіб представлення вебсторінок за допомогою набору об'єктів, що визначає набір інтерфейсів, незалежних від платформи і мови, який дозволяє програмам і сценаріям динамічно змінювати структуру, вміст і стиль документів);

Virtual DOM – це техніка та набір бібліотек і алгоритмів, які дозволяють покращити продуктивність на клієнтській стороні, уникаючи прямої роботи з DOM шляхом взаємодії з JavaScript-об'єктом, що імітує DOM-дерево;

JSX – JavaScript Extension (розширення JavaScript, що дозволяє декларативно створювати компоненти користувацького інтерфейсу);

XML – eXtensible Markup Language (розширювана мова розмітки);

CSV – Comma-Separated Values (формат даних з роздільниками, в якому поля та стовпці розділені символом коми);

JSON – JavaScript Object Notation (текстовий формат обміну даними між комп'ютерами);

HTML – HyperText Markup Language (мова тегів, засобами якої здійснюється розмічання вебсторінок для мережі Інтернет);

AOT Compilation – Ahead-of-Time (компілятор перед виконанням, вид транслятора, який використовує метод компіляції перед виконанням);

Фреймворк – Framework (програмне середовище, яке спрощує та прискорює створення програмного забезпечення);

DML – Data Manipulation Language (сімейство комп'ютерних мов, що використовуються в комп'ютерних програмах або користувачами баз даних для отримання, вставки, видалення або зміни даних в базах даних);

ACID – Atomicity, Consistency, Isolation, Durability (набір властивостей, що гарантують надійну роботу транзакцій бази даних: атомарність, узгодженість, ізолюваність, довговічність);

BSON – Binary JavaScript Object Notation (двійкова форма представлення простих структур даних і асоціативних масивів);

Скейкхолдери – Stakeholders (зацікавлені в проєкті особи чи сторони, що мають якесь до нього відношення);

QA-інженер – Quality Assurance engineer (фахівець із забезпечення якості, діяльність якого спрямована на поліпшення процесу розробки ПЗ, запобігання дефектів і виявлення помилок в роботі продукту);

MVC – Model-View-Controller (архітектурний шаблон, який використовується під час проєктування та розроблення ПЗ);

ORM – Object-Relational Mapping (технологія програмування, яка зв'язує бази даних з концепціями об'єктно-орієнтованих мов програмування, створюючи «віртуальну об'єктну базу даних»).

ВСТУП

На кінець 2022 та початок 2023 року спостерігається різке повернення працівників до офісів, що йде на зміну дистанційному формату роботи. Це спричинено декількома факторами, основний з яких – регулярні відключення електроенергії у населених пунктах України. Щоб забезпечити своїх підлеглих можливістю повноцінно працювати 8 годин на добу, власники більшості компаній встановлюють генератори в офісних приміщеннях або орендують коворкінги з належними умовами для праці.

З метою синхронізації роботи членів команд та оптимізації процесів менеджери проєктів проводять різноманітні зустрічі. Серед регулярних нарад можна виділити щоденну 15-хвилинну, яка націлена на оновлення статусів задач по мірі їх виконання. Приблизно раз у два тижні команда збирається, щоб обговорити нові вимоги до програмного продукту, над яким вони працюють, та розподілити задачі між собою. Також проводяться зустрічі для виявлення залежностей між командами або членами команд та знаходження шляхів їх вирішення. Зважаючи на вище описане, можна зробити висновок, що спільне обговорення планів та труднощів – це невід’ємна частина роботи у команді та ключ до успішного ведення проєкту.

На сьогодні не існує єдиного рішення, яке б задовольняло усі вимоги компаній, що мають офісні приміщення та прагнуть автоматизувати процес відстеження бронювань переговорних кімнат. Більшість з них не дозволяють фільтрувати усі доступні приміщення за певними критеріями та повідомляти запрошених на зустріч людей про внесені до неї зміни або скасування. Тому актуальність програмного продукту, що розробляється у рамках даного дипломного проєкту, є високою. Програмне забезпечення, отримане у результаті розроблення, дозволить відвідувачам офісу обирати та бронювати на визначений період часу переговорну кімнату, яка задовольняє їхні критерії, а менеджерам офісу – управляти даними про конференц-зали та користувачів системи.

1. ОГЛЯД ІСНУЮЧИХ ПРОГРАМНИХ РІШЕНЬ

У першому розділі пояснювальної записки подано матеріали про проблеми, наявні у відвідувачів та менеджерів офісів різних за розміром компаній, та проведення аналізу існуючих програмних рішень. Нижче сформовано перелік завдань, що повинні бути виконані у рамках даного розділу:

- 1) аналіз ринку та визначення проблем, з якими стикаються відвідувачі офісу в розрізі бронювання переговорних кімнат;
- 2) дослідження найпопулярніших програмних забезпечень, якими користуються відвідувачі офісу для бронювання приміщення, виконання порівняльного аналізу, визначення переваг та недоліків кожного з них;
- 3) побудова порівняльної таблиці результатів аналізу сервісів;
- 4) формулювання переліку потрібної функціональності розроблюваного ПЗ.

1.1. Огляд проблеми, яка вирішується ПЗ

Для проведення нарад та особистих зустрічей працівники використовують переговорні кімнати, щоб не заважати іншим відвідувачам офісу та не розповсюджувати конфіденційну інформацію проєктів серед усіх. Основні проблеми, з якими стикаються відвідувачі офісу – конфлікти зустрічей та відсутність необхідного обладнання кімнат. Конфлікти зустрічей спричинені перетином часу проведення нарад, що, у свою чергу, виникає через недостатню комунікацію між учасниками – домовленістю, хто та у який період часу займе конкретне приміщення. Інша поширена серед відвідувачів офісу незручність – відсутність обладнання. Якщо переговорна кімната не має необхідного устаткування (наприклад, проектора, пристроїв для виведення звуку або інтерактивної дошки), то організатор зустрічі буде змушений підбирати інший зал або вносити зміни

до початкового плану конференції. Тому напередодні планування наради координатору потрібно завчасно знайти приміщення, яке відповідало б усім його вимогам і при цьому було вільним на певний проміжок часу.

Наведені проблеми можуть принести багато незручностей для учасників ділових зустрічей, вплинути на продуктивність та ефективність їхньої роботи. Тому для запобігання подібним ризикам відвідувачі офісу повинні дотримуватись чіткого графіку проведення переговорів, бути завчасно повідомленими щодо будь-яких змін у бронюваннях і мати можливість обирати приміщення, яке задовольнятиме їхні потреби.

Метою даного дипломного проєкту є забезпечення ефективного використання переговорних кімнат, уникнення конфліктів зустрічей при бронюванні приміщення, шляхом розроблення відповідного програмного забезпечення.

Для досягнення поставленої мети сформовано перелік задач, які потрібно виконати:

- 1) аналіз ринку ПЗ для бронювання переговорних кімнат офісу;
- 2) порівняння конкурентів, виявлення їхніх сильних та слабких сторін;
- 3) визначення функціональних можливостей системи, яких потребують потенційні користувачі;
- 4) підготовка дизайну застосунку;
- 5) обґрунтування вибору стеку технологій для розроблення ПЗ;
- 6) програмна реалізація вебзастосунку для бронювання переговорних кімнат офісу;
- 7) тестування системи.

1.2. Аналіз існуючих рішень

Аналіз існуючих рішень допомагає краще зрозуміти предметну галузь, дослідити переваги та недоліки конкурентів, а також розробити стратегію розвитку власного програмного продукту.

Першим етапом у процесі аналізу є підготовка до нього, а саме пошук застосунків, які задовольняли б основні потреби потенційних користувачів. Як виявилось, таких систем налічується невелика кількість. Це спричинено тим, що офіси, різні за кількістю та розмірами переговорних кімнат і чисельністю відвідувачів, мають різні вимоги до системи для бронювання приміщень. Серед наявних рішень виділено найпопулярніші: Evoko Liso [1], Microsoft Excel [2] і Google Calendal [3].

Для структурованого аналізу програм, які потенційно могли б вирішити проблеми відвідувачів офісу, виділено основні критерії для порівняння їхніх функціональних можливостей. Тобто Evoko Liso, Microsoft Excel та Google Calendar порівнюватимуться за тим, чи задовольняють вони наступні потреби:

- 1) перегляд завантаженості кімнат на тиждень або місяць;
- 2) невеликі затрати часу на створення бронювання;
- 3) захищеність даних від випадкового або навмисного видалення іншими користувачами;
- 4) можливість переглянути організаторів інших зустрічей;
- 5) фільтрація кімнат за певними параметрами;
- 6) наявність панелі адміністратора;
- 7) можливість управляти кількістю кімнат, їхнім наповненням;
- 8) нотифікація учасників зустрічі про її створення, внесення змін або скасування.

1.2.1. *Evoko Liso*

Evoko Liso – електронна система для створення та управління запланованими зустрічами. Вона виглядає як панель, що розміщується поряд зі входом до переговорної кімнати на стіні або скляних дверях [1].

За замовчуванням на екрані Evoko Liso відображається статус приміщення, а саме його поточна завантаженість. Дисплей підсвічується червоним та веде зворотній рахунок до кінця поточної зустрічі, якщо

переговорна кімната зайнята. У протилежному випадку підсвітка панелі зелена, а відлік часу ведеться до початку наступного бронювання.



Рис. 1.1. Фрагмент головного екрану EvoKo Liso

Процес створення бронювання є досить простим та не вимагає багато зусиль. Для цього користувачу потрібно натиснути на клавішу «Book», обрати початок та кінець зустрічі, підтвердити свій вибір та ввести пароль, який ідентифікує відвідувача офісу. Загалом процес займає приблизно 1,5 хвилини. Для того, щоб збільшити час зустрічі, потрібно знайти свій запис, поряд з ним натиснути на клавішу «Extend», обрати бажаний час, підтвердити вибір та ще раз ввести свій пароль. Внесення таких змін до створеного бронювання займає у середньому 1,5 хвилини. Також користувач має можливість скасувати зустріч, здійснивши маніпуляції, подібні до її створення та редагування.

Серед наявних недоліків передусім виділяється те, що для великих офісів, на рахунок у яких декілька десятків переговорних кімнат, встановлення EvoKo Liso коштуватиме немало. Ціна однієї такої панелі станом на початок 2023 майже 70 тисяч гривень [1]. Якщо в офісі розміщено

20 конференц-залів, то для їх оснащення власнику приміщення доведеться виділити близько півтора мільйона гривень.

Також виявлено недопрацювання системи, яке полягає в тому, що відвідувачі офісу можуть скасовувати зустрічі інших людей. Не варто надавати користувачам такої можливості, адже це може призвести до непорозумінь та сутичок всередині колективу. З цього сценарію використання виділено обов'язкову вимогу до системи: користувачі системи повинні мати можливість видаляти лише ті зустрічі, адміністраторами яких вони є.

Серед переваг Evoko Liso можна зазначити:

- 1) надання розуміння про поточний статус кімнати за допомогою світлового індикатору (зелений, якщо приміщення вільне, та червоний у протилежному випадку);
- 2) легке керування завдяки простому та інтуїтивно зрозумілому інтерфейсу;
- 3) невеликі затрати часу на створення, редагування та скасування зустрічі (близько 1,5 хвилини на кожну з операцій).

З недоліків електронної системи виділено:

- 1) велику вартість у порівнянні з іншими застосунками для бронювання переговорних кімнат (70 тисяч гривень за 1 панель);
- 2) залежність від хмарних технологій, які використовує Evoko Liso;
- 3) необхідність підключення до електромережі, що може вплинути на місце розташування дисплею та її роботу загалом;
- 4) надання можливості усім користувачам видаляти зустрічі, створені іншими людьми.

Отже, електронна панель Evoko Liso покриває більшу частину поставлених перед нею вимог. Серед переваг варто виділити швидкість створення бронювання та внесення до нього змін, що досягається завдяки зручному та простому користувацькому інтерфейсу. Якщо говорити про

недоліки Evoko Liso, то найбільшими мінусами є висока вартість та необхідність постійного підключення до електромережі.

1.2.2. *Microsoft Excel*

Досить часто у компаніях середнього розміру зустрічається використання інструменту Microsoft Excel для управління бронюваннями переговорних кімнат або робочих місць офісу.

Microsoft Excel – це програма, яка надає можливість користувачам працювати з електронними таблицями, обчислювати, аналізувати та виконувати візуалізацію даних різноманітними способами. Даний інструмент досить широко використовується у бухгалтерії, адже за допомогою нього спеціалісти мають змогу створювати потрібні їм графіки та діаграми, оформлювати документи та розрахунки [2].

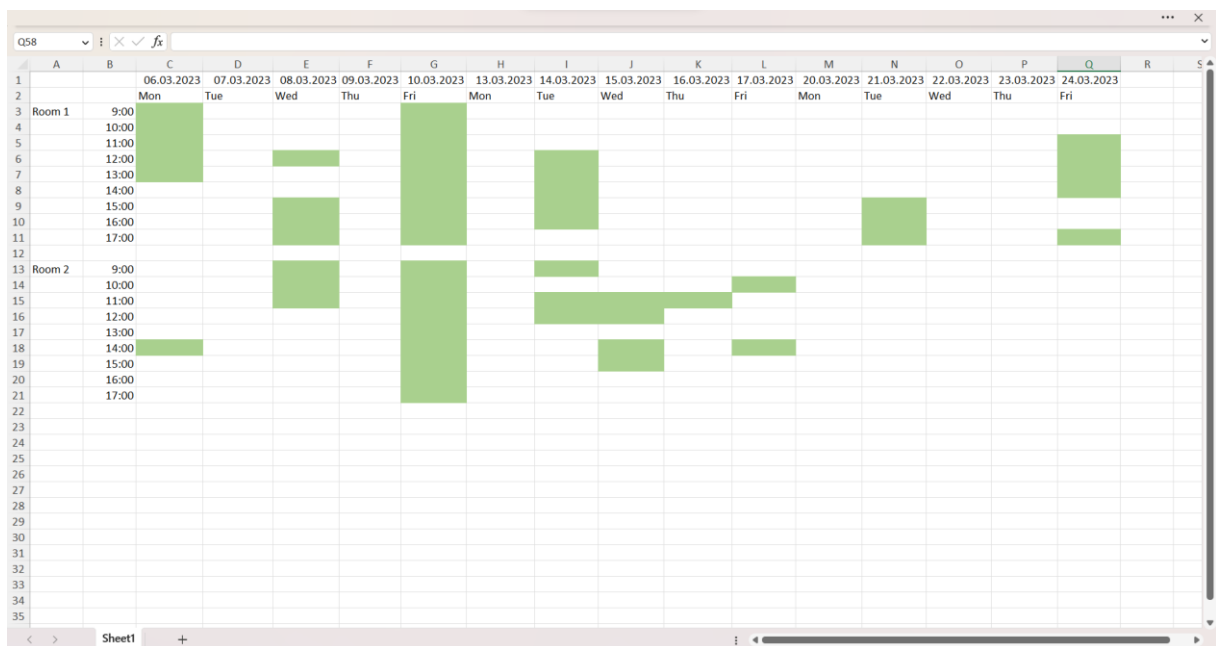


Рис. 1.2. Приклад таблиці графіку зайнятості кімнат при використанні програми Microsoft Excel

Що стосується застосування Microsoft Excel у створенні бронювання, то процес, який складається з 3 кроків, є примітивним та зрозумілим для усіх його учасників. Передусім, менеджер офісу або інша відповідальна за це

людина повинна створити таблицю. По горизонталі розміщуються дати, на які відвідувачі офісу потенційно можуть забронювати переговорну кімнату для проведення зустрічей, а по вертикалі – список доступних залів та їхні графіки роботи з інтервалом, наприклад, в 1 годину. Далі людина, якій потрібно запланувати нараду та забронювати для цього приміщення, повинна відкрити створений раніше менеджером офісу таблицю та виділити кольоровою заливкою комірки, які є перетином потрібної дати та часового проміжку обраної переговорної кімнати. На цьому процес створення бронювання завершується.

Під час проходження покрокової інструкції створення бронювання кімнати помічено декілька значних недоліків:

- 1) потреба у постійній підтримці таблиці в оновленому статусі – менеджер офісу буде змушений регулярно заповнювати комірки датами на декілька днів вперед і видаляти минулі бронювання, що може призвести до випадкового вилучення потрібних записів;
- 2) відсутність захищеності даних – усі користувачі, яким надано права на редагування документу, матимуть змогу видалити чужі записи, у результаті чого можуть виникати непорозуміння між колегами;
- 3) відсутність розуміння, хто є організатором та учасниками певної зустрічі;
- 4) незручність для використання офісними приміщеннями з великою кількістю переговорних кімнат – у такому випадку таблиця займе екстремально багато місця у висоту;
- 5) відсутність інформації про переговорну кімнату (її місткість, встановлене обладнання і т.д.);
- 6) процес створення бронювання не можна вважати автоматизованим.

Але варто зазначити і плюси використання програми Microsoft Excel для управління бронюваннями.

Серед переваг даного рішення виділено:

- 1) миттєве оновлення даних – внесення змін до таблиці буде відображатись усім користувачам, які відкрили її на своєму пристрої, у режимі реального часу;
- 2) простота у використанні;
- 3) гнучкість до змін – можливість розширення таблиці шляхом додавання годин роботи офісу, кількості кімнат та іншої потрібної інформації.

Отже, Microsoft Excel як інструмент для бронювання переговорних кімнат офісу не задовольняє більшу частину поставлених перед ним вимог або робить це частково, зі значною кількістю недоліків. Проте у той же час програма відрізняється своїми перевагами, такими як: гнучкість до змін, простота у використанні та оновлення даних у режимі реального часу. Тому при створенні нової системи потрібно врахувати ці пункти, які є досить важливими для її користувачів.

1.2.3. Google Calendar

Іншим популярним інструментом для управління бронюваннями переговорних кімнат є Google Calendar, який ще часто називають електронним календарем. Загалом це безкоштовний застосунок для планування подій та призначення зустрічей, розроблений компанією Google. Для того, щоб мати доступ до даного сервісу, користувачам достатньо увійти до свого акаунту Google або створити його [3].

Розглянемо як електронний календар можна використовувати для бронювання переговорних кімнат офісу. Насамперед важливо зауважити, що ця функціональна можливість доступна лише для користувачів пакету Google Workspace, який не є безкоштовним. Станом на початок 2023 року вартість за рік на одного користувача – 62,4 євро, проте при цьому для корпоративних організацій діють спеціальні знижки [4].

Маючи корпоративний акаунт, користувач може з легкістю створити бронювання в одній із запропонованих кімнат, інформацію про які заздалегідь додали менеджери або інші відповідальні за це працівники. Сервіс не надає можливості фільтрувати дані про приміщення за критеріями, проте частково вирішує проблему з вибором потрібної кімнати. Коли користувач починає вводити у рядок пошуку назву конференц-зали, йому пропонуються релевантні варіанти. Проте у цьому випадку відвідувачам офісу необхідно тримати у голові назви усіх кімнат, щоб швидше знаходити потрібну їм.

Якщо ж у компанії немає корпоративного акаунту, її працівники теж можуть використовувати Google Calendar для управління бронюваннями, але менш ефективно. Такий спосіб полягає у створенні окремого календаря для кожного з конференц-залів. При цьому потрібно додати назву кімнати, вказати максимальну місткість, наповнення або іншу інформацію за необхідності.

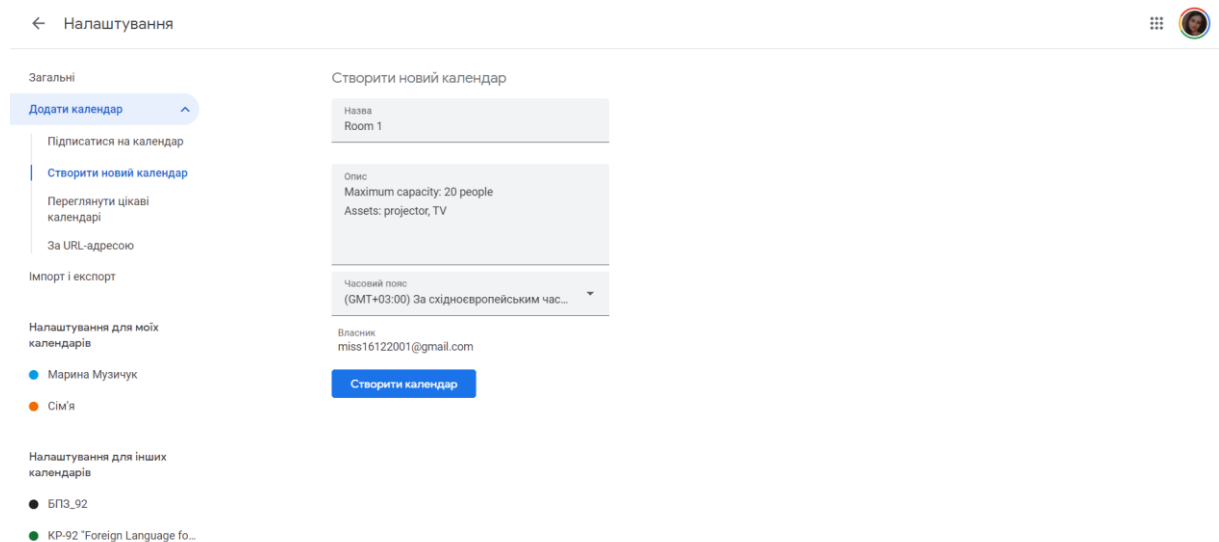


Рис. 1.3. Фрагмент створення календаря як переговорної кімнати у Google Calendar

Для того, щоб переглядати бронювання кімнати та створювати власні, користувачеві потрібно обрати відповідний календар із доступних йому у

лівій панелі сервісу. Процес створення букінгу аналогічний до планування онлайн-зустрічі у Google Calendar.



Рис. 1.4. Фрагмент наповненого календаря переговорної кімнати у Google Calendar

Також користувач може вносити зміни до існуючих зустрічей (змінювати час початку або кінця, редагувати назву та опис події) або видаляти їх. Сервіс відображає інформацію у режимі реального часу.

Серед переваг Google Calendar як інструменту для управління бронюваннями виділено наступні:

- 1) простота у використанні: завдяки інтуїтивно зрозумілому користувацькому інтерфейсу та зручним інструментам для управління власним бронюванням (його створення, налаштування, редагування та скасування): користувачам не доведеться витратити багато часу на те, щоб навчитись виконувати вказані дії;
- 2) мінімальні витрати часу на процес створення, внесення змін та скасування букінгу кімнати;
- 3) спільний доступ до календаря: після коректного налаштування простору усі додані до нього учасники матимуть доступ до

календаря, що зробить бронювання кімнат для групи людей більш ефективним.

Незважаючи на наведені переваги сервісу Google Calendar, наявні і його недоліки:

- 1) висока щорічна вартість обслуговування, яка прямо пропорційна кількості працівників компанії, що використовує пакет Google Workspace;
- 2) обмеженість сервісу у налаштуваннях, які користувачі можуть змінити, що може призвести до пошуку інших інструментів для управління бронюваннями;
- 3) усі учасники, додані до календаря, мають права на внесення змін без згоди адміністратора, що може мати неприємні наслідки;
- 4) відсутність фільтрації кімнат за критеріями, яка є досить критичним недоліком для офісів з великою кількістю приміщень з різними розташуванням, місткістю та наповненням.

Отже, зважаючи на наведені вище переваги та недоліки сервісу хочеться зазначити, що Google Calendar може бути корисним інструментом бронювання для офісів з невеликою кількістю переговорних кімнат. У такому разі відсутність фільтрації кімнат не принесе відвідувачам відчутних незручностей, а вартість обслуговування корпоративного пакету не коштуватиме занадто багато. Проте компаніям з великою чисельністю працівників та переговорних кімнат в офісі варто підібрати систему, яка б задовольняла найголовніші їхні потреби.

1.3. Результати проведеного аналізу

Після проведення аналізу існуючих сервісів, які досить часто користуються попитом серед компаній, що мають офісні приміщення та прагнуть автоматизувати процес відстеження бронювань на ту чи іншу переговорну кімнату, сформовано декілька висновків.

У першу чергу хочеться зазначити, що кожен з них (Evolo Liso, Microsoft Excel та Google Calendar) виконує основну функцію: дозволяє відвідувачам офісу бронювати той чи інший конференц-зал, проте рівень ефективності та зручності виконання цієї дії в усіх них різний.

Також варто звернути увагу на той факт, що жодне із розглянутих програмних забезпечень не впоралось на 100% із поставленими задачами, усі вони мають значні недоліки. Наприклад, Microsoft Excel та безкоштовна версія Google Calendar не передбачають захисту даних від випадкового або навмисного редагування та видалення іншими користувачами простору. Також учасники бронювань не отримують сповіщень про створення, скасування або внесення змін до зустрічей, на яких вони мають бути присутніми. Що стосується Evoko Liso та платного пакету Google Workspace для Google Calendar, то вони краще впорались з вимогою захисту інформації. Наприклад, коли користувач намагається виконати будь-яку дію з бронюваннями, Evoko Liso щоразу запитує підтвердження особи. Google Calendar вирішує дану проблему тим, що дозволяє користувачам редагувати та скасовувати лише їхні власні записи.

Що стосується переваг проаналізованих сервісів, то Evoko Liso відрізняється від своїх конкурентів швидкістю та простотою створення і редагування бронювань. Дійсно, цей процес займає у середньому 1,5 хвилини та не вимагає від користувачів великих зусиль. Серед сильних сторін Microsoft Excel та Google Calendar варто відмітити те, що обидва інструменти підтримують миттєве оновлення даних, завдяки чому усі учасники спільного простору можуть переглядати зміни графіку зайнятості кімнат для переговорів у режимі реального часу.

Перед початком проведення аналізу існуючих програмних рішень сформовано перелік функціональних вимог, які повинна забезпечувати система для бронювання переговорних кімнат офісу. Нижче наведено таблицю порівняння сервісів за критеріями оцінки.

Таблиця 1.1

Порівняння існуючих програмних рішень

Сервіс Функціональна вимога	Evoko Liso	Microsoft Excel	Google Calendar
Перегляд завантаженості кімнат на тиждень і місяць	–	+/-	+
Невеликі затрати часу на створення бронювання	+	+	+
Захищеність даних від випадкового або навмисного видалення іншими користувачами	+/-	–	+/-
Перегляд організаторів інших зустрічей	+	–	–
Фільтрація кімнат за певними параметрами	–	–	–
Наявність панелі адміністратора	+	–	+
Управління кількістю та параметрами кімнат	–	+/-	+/-
Нотифікація учасників зустрічі про її створення, внесення змін або скасування	+/-	–	+

Жодне з наявних рішень не задовольняє усі функціональні вимоги, які могли б вирішити основні проблеми потенційних користувачів системи для бронювання переговорних кімнат офісу. Тому на даному етапі виявлено та сформовано перелік можливостей, які повинні бути наявними у ПЗ:

- 1) перегляд завантаженості усіх переговорних кімнат офісу на день, тиждень та місяць;
- 2) можливість обрати потрібне приміщення, використовуючи фільтрацію за певними параметрами, такими як поверх розташування, максимальна місткість та обладнання кімнати;
- 3) створення одноразового та повторюваного бронювання;
- 4) внесення змін до створеного бронювання;
- 5) скасування бронювання;
- 6) нотифікація учасників зустрічі про створення бронювання.

1.4. Висновки до розділу 1

У ході підготовки матеріалів до першого розділу дипломного проєкту виконано наступні поставлені задачі:

- 1) проаналізовано ринок ПЗ для бронювання переговорних кімнат офісу;
- 2) встановлено основні проблеми, з якими стикаються користувачі подібних систем;
- 3) виявлено популярні серед компаній ПЗ для бронювання переговорних кімнат;
- 4) сформовано перелік критеріїв порівняння систем у вигляді функціональних вимог;
- 5) виконано порівняльний аналіз існуючих програмних рішень, виділено переваги та недоліки кожної з них;
- 6) побудовано порівняльну таблицю результатів аналізу сервісів;
- 7) сформовано перелік функціональних можливостей, які повинні бути наявними у застосунку, створюваному у рамках дипломного проєкту.

2. ОБГРУНТУВАННЯ ВИБОРУ ЗАСОБІВ РОЗРОБЛЕННЯ

На даному етапі потрібно сформулювати основні вимоги до засобів реалізації програмного продукту, провести структурований аналіз обраних засобів, навести список особливостей кожного з інструментів та скласти таблиці для їх порівняння за сформованими критеріями.

Система для бронювання переговорних кімнат офісу поділена на дві основні частини: клієнтську та серверну. Таке розділення забезпечить високу ефективність роботи застосунку, оскільки даний підхід значно зменшить завантаженість клієнтської частини. Крім того, це підвищує безпеку системи, оскільки сервер може виконувати авторизацію та перевірку прав доступу користувачів до конфіденційної інформації, що, у свою чергу, зменшує ризик її витоку.

Серверна частина повинна управляти даними з БД, обробляти запити та надавати на них відповіді клієнтській частині. Що стосується клієнтської частини, вона забезпечує застосунок користувацьким інтерфейсом і комунікацією з сервером за допомогою протоколу HTTP. СКБД відповідає за управління даними у БД, тобто за допомогою неї можна зберігати, відновлювати та робити пошук серед даних.

2.1. Вибір мови програмування для реалізації серверної частини

Мова програмування, за допомогою якої реалізовано серверну частину застосунку для бронювання переговорних кімнат офісу, повинна задовольняти наступні висунуті до неї вимоги:

- 1) підтримка асинхронного програмування;
- 2) наявність модульності;
- 3) великий вибір модулів та бібліотек;
- 4) зручність управління залежностями;
- 5) легкість розширювання;
- 6) наявність фреймворку для побудови вебсерверу;

- 7) можливість використовувати один і той самий код для розроблення серверної та клієнтської частини;
- 8) наявність великої кількості документації та ресурсів для вивчення.

2.1.1. Python

Python – високорівнева інтерпретована об'єктно-орієнтована мова програмування. Вона вважається зручною для швидкого розроблення ПЗ завдяки якісній вбудованій структурі даних та динамічній типізації [5]. Значною перевагою Python над іншими мовами програмування є його простота у використанні завдяки зрозумілому та легкому для читання синтаксису. Через те, що Python підтримує модулі та пакети, розробники можуть повторно використовувати код і забезпечувати модульність своїх програм. А це, у свою чергу, допомагає локалізувати виникнення помилок на рівні модулів.

Відсутність етапу компіляції Python-коду перед його запуском може мати як позитивний, так і негативний характер. З однієї сторони програмісти одразу бачитимуть помилки, допущені у коді програми, що скоротить час на їх виявлення. Але з іншого боку через те, що інтерпретатор виконує Python-код рядок за рядком, натрапивши на помилку, він відразу ж зупинить запуск програми, що вплине на продуктивність.

Python має велику кількість вбудованих функцій, що теж є одночасно і перевагою, і недоліком. Плюсом є те, що використання цих функцій значно спрощує процес написання коду, оскільки розробникам не потрібно власноруч створювати алгоритми. Проте за рахунок цього Python може займати більше пам'яті, ніж інші мови програмування.

Серед переваг мови програмування Python виділено:

- 1) підтримку об'єктно-орієнтованої парадигми програмування, що дозволяє організовано та структуровано створювати складні програми;
- 2) наявність модульності;

- 3) інтерпретованість – відсутність етапу компіляції перед запуском, що пришвидшує процес розроблення;
- 4) динамічну типізацію даних;
- 5) багату екосистему (фреймворки, інструменти та бібліотеки), що значно полегшують процес створення програм;
- 6) кросплатформеність – підтримка Python на різних операційних системах, завдяки чому розроблені застосунки можуть коректно працювати на різних платформах;
- 7) зрозумілий та легкий у читанні синтаксис.

Проте наявні і недоліки Python, наприклад:

- 1) негативний вплив на продуктивність через інтепретованість мови;
- 2) відсутність стандарту для розроблення вебзастосунків;
- 3) складне уникнення помилок під час виконання – розробник не дізнається про помилки (навіть синтаксичні), доки програма не запуститься;
- 4) інтенсивне використання пам'яті – застосунок, написаний Python, може використовувати приблизно у 10 разів більше оперативної пам'яті, ніж програми, створені за допомогою інших мов програмування [6];
- 5) відсутність підтримки багатопоточної архітектури – натомість Python використовує багатопроцесорність, тобто виконує кожен «потік» в окремому процесі.

2.1.2. Java

Java – об'єктно-орієнтована кросплатформна мова програмування, яка є відомою завдяки своїй надійності та простоті у використанні. Особливістю Java є те, що вона не залежить від платформ. Це означає, що скомпільована програма, написана мовою програмування Java, запуститься на будь-якій платформі з встановленою віртуальною машиною JVM (Java Virtual Machine).

Мова програмування Java має досить гнучку систему безпеки, через що вважається надійною і безпечною. Це спричинено тим, що запуск та виконання програми контролюється віртуальною машиною JVM. Тому усі спроби несанкціонованого доступу до даних або виконання операцій, що відрізняються від дозволених прав, будуть заблоковані [7].

Java підтримує наслідування, поліморфізм та статичну типізацію, дозволяє використовувати об'єкти, класи та інші концепції ООП. Завдяки цьому Java дозволяє створювати одночасно структуровані великі гнучкі застосунки, які можуть розширюватись у подальшому.

Серед переваг Java можна зазначити:

- 1) виконання парадигм ООП, завдяки чому Java-розробники можуть створювати масштабовані великі проєкти;
- 2) автоматичне керування пам'яттю – наявність Garbage Collector;
- 3) паралельне виконання програм;
- 4) кросплатформеність – виконання коду можливе на будь-якій платформі або операційній системі;
- 5) надійність – оскільки Java є строго типізованою мовою, на момент компіляції усі змінні мають конкретний тип, що, у свою чергу, спрощує виявлення недоліків і помилок коду [8];
- 6) наявність великої кількості фреймворків та бібліотек.

Крім переваг мови Java, наявні і її недоліки, серед яких:

- 1) велике споживання пам'яті, що спричинене виконанням програм поверх віртуальної машини JVT;
- 2) досить складний та багатослівний синтаксис;
- 3) низька продуктивність через високий рівень компіляції, виконанням коду за допомогою віртуальної машини та наявності збирача сміття, що займає більше 20% процесорного часу [9];
- 4) повільний запуск програм у порівнянні з застосунками, написаними іншими мовами програмування;
- 5) великий розмір виконуваних файлів.

2.1.3. *JavaScript*

JavaScript – об’єктно-орієнтована динамічна мова програмування. Найчастіше вона асоціюється з розробленням клієнтської частини, оскільки дозволяє створювати інтерактивні користувацькі інтерфейси застосунків та динамічні вебсторінки. До використання JavaScript вебсторінки були статичні і для того, щоб оновити вигляд сторінки, користувачеві потрібно було виконати запит на отримання нової версії з серверу та перезавантажити браузер. Сучасні вебсторінки, використовуючи підхід AJAX, отримують актуальну інформацію з серверу без мануального оновлення браузера. Станом на початок 2023 року, більше 97% вебсайтів створені за допомогою JavaScript [10].

Проте дана мова програмування нерідко зустрічається і як засіб реалізації серверної частини застосунків. Середовище виконання Node.js розширило можливості JavaScript і перетворило її з технології для створення лише клієнтської частини застосунків на мову програмування загального вжитку. Завдяки платформі Node.js код JavaScript може використовуватись для створення як клієнтської, так і серверної частин програми, що значно полегшує та пришвидшує процес розроблення системи.

Серед переваг JavaScript виділено наступні:

- 1) можливість її використання для розроблення повного стеку;
- 2) наявність фреймворку для побудови вебсерверу;
- 3) підтримка асинхронного програмування, завдяки чому сервер матиме змогу обробляти декілька запитів одночасно, не очікуючи завершення попереднього;
- 4) висока швидкість виконання – оскільки компіляція та виконання JavaScript-коду відбувається на сервері, запуск програми займатиме значно менше часу, ніж якби його потрібно було завантажувати та інтерпретувати у браузері [11];
- 5) підтримка модульної організації, яка полегшує розуміння, зберігання та повторне використання написаного коду;

- 6) обробка винятків;
- 7) наявність збирача сміття.

Незважаючи на переваги JavaScript, наявні і недоліки даної мови програмування, наприклад:

- 1) збільшення розміру та часу розгортання програми через використання фреймворків та бібліотек;
- 2) динамічна типізація – хоча іноді це пришвидшує процес написання коду, оскільки розробникам не потрібно обов’язково вказувати тип оголошених змінних, це може призвести і до негативних наслідків під час виконання програми;
- 3) можливе виникнення труднощів при налаштуванні платформи Node.js для виконання на сервері програми, написаної мовою JavaScript.

2.1.4. Результати проведеного аналізу

На початку розділу висунуто вимоги до технологій для створення серверної частини вебзастосунку для бронювання переговорних кімнат. Нижче наведено порівняльну таблицю засобів реалізації (Python, Java, JavaScript) за раніше сформованими критеріями.

Таблиця 2.1

Порівняння мов програмування для реалізації серверної частини застосунку

Мова програмування	Python	Java	JavaScript
Критерій порівняння			
Підтримка асинхронного програмування	+	+	+
Наявність модульності	+	+	+

Великий вибір модулів та бібліотек	+	+/-	+
Зручність управління залежностями	—	—	+
Легкість розширювання	+/-	+/-	+
Наявність фреймворку для побудови вебсерверу	+	+	+
Можливість використовувати один і той самий код для розроблення серверної та клієнтської частини	—	—	+
Наявність великої кількості документації та ресурсів для вивчення	+	+/-	+

Результати порівняння мов програмування показують, що JavaScript задовольнила усі поставлені вимоги до засобу реалізації серверної частини програми. Вона має не лише фреймворк для побудови вебсерверу, але й великий вибір інших модулів та бібліотек, що дозволяють розробникам використовувати вбудовані можливості мови і економити час на створення подібних функцій. Що стосується наявності документації, то на просторах Інтернету можна знайти велику кількість ресурсів для вивчення JavaScript, що є досить важливою перевагою.

Якщо говорити про Python, дана мова програмування відрізняється від своїх аналогів зрозумілим та легким у читанні синтаксисом, який нагадує текст, написаний англійською. Python має багату екосистему, яка налічує велику кількість фреймворків, інструментів та бібліотек, що полегшують процес створення програм. Плюсом Python є наявність фреймворку Django для побудови вебсерверу. Але незважаючи на це, значним недоліком мови програмування є те, що вона зазвичай не використовується у побудові користувацького інтерфейсу. Тому виникає неможливість використовувати один і той самий код для реалізації серверної та клієнтської частин.

Що стосується Java, ця мова програмування підтримує асинхронну обробку коду, що дозволяє одночасно виконувати декілька запитів, і значно

пришвидшує роботу програми. Також перевагою Java є модульна організація, яка допомагає зберігати код у зручному вигляді. Проте наявні і недоліки даної мови програмування: порівняно висока складність розробки та незручність управління залежностями. До мінусів Java можна віднести і невелику кількість вбудованих бібліотек та фреймворків.

Отже, у результаті проведеного аналізу можна стверджувати, що JavaScript задовольняє усі поставлені перед нею вимоги. Суттєвою перевагою JavaScript над іншими її аналогами є те, що вона може використовуватись як технологія для розроблення повного стеку програми. Таким чином, частина коду, написаного цією мовою, може бути використана як у серверній, так і клієнтській частинах застосунку, що значно полегшує та прискорює процес створення програми.

2.2. Вибір технологій для реалізації клієнтської частини

Найпопулярнішими технологіями для розроблення клієнтської частини застосунків є React.js, Angular та Vue.js. Саме їх обрано для подальшого аналізу, у результаті якого планується виділити одну для створення користувацького інтерфейсу користувача системи.

Технологія для реалізації клієнтської частини повинна задовольняти наступні вимоги:

- 1) підтримка компонентного підходу до архітектури;
- 2) висока швидкість оновлення стану сторінки;
- 3) лаконічність коду;
- 4) великий вибір бібліотек;
- 5) висока продуктивність.

2.2.1. React.js

React.js — гнучка бібліотека JavaScript з відкритим кодом, за допомогою якої розробляються динамічні інтерактивні користувацькі інтерфейси застосунків.

Оскільки React використовує компонентний підхід архітектури, процес створення застосунку нагадує будівництво, де кожен компонент виступає у ролі окремої цеглинки, яка є важливою частиною кінцевого результату. Плюсом даного підходу є те, що створені компоненти є ізольованими та можуть неодноразово використовуватись у різних частинах системи. Більш того, деякі компоненти можуть містити у собі один або декілька інших. Таким чином, інтерфейс застосунку нагадує деревовидну структуру компонентів.

З метою ефективного оновлення стану сторінки React.js використовує технологію Virtual DOM, яка зберігає у собі інформацію про стан інтерфейсу користувача. Коли відбуваються зміни у стані компонентів Virtual DOM, React.js порівнює попередню та оновлену версії, виявляє, які саме зміни потрібно внести до браузерного DOM, та відображає новий вигляд інтерфейсу [12]. На проведення таких маніпуляцій витрачається значно менше ресурсів, ніж на завантаження усієї сторінки загалом. Таким чином, використання технології Virtual DOM має позитивний вплив на швидкість та продуктивність роботи застосунку.

React.js – відкрита бібліотека, тому її можна поєднувати з іншими інструментами. Кількість бібліотек, що можуть комбінуватись з React.js, є дуже великою. Тому спеціалістам пропонується широкий вибір необхідних їм інструментів для розроблення тієї чи іншої функціональності.

Іншою особливістю React.js є те, що ця бібліотека надає можливість розробникам писати структурований та лаконічний код завдяки використанню JSX. Це допоможе їм у майбутньому з легкістю забезпечувати підтримку проєкту та розширювати його за потреби. JSX – це розширення синтаксису JavaScript, подібне до XML та HTML. Підхід використання JSX значно спрощує розуміння коду та його читання.

Отже, перевагами React.js є компонентний підхід до архітектури, лаконічність та простота коду, швидкість запуску програми та оновлення

стану сторінки, можливість взаємодії з великою кількістю бібліотек та інших інструментів.

2.2.2. *Angular*

Angular – фреймворк з відкритим кодом, що використовується для створення односторінкових або багатосторінкових застосунків, які визначаються швидкістю та продуктивністю роботи.

Як і бібліотека React.js, Angular підтримує компонентний підхід до архітектури, дозволяючи повторно використовувати написаний код у різних місцях системи. Таким чином, розробники витрачають менше часу та зусиль на створення системи, оскільки вони маніпулюють не окремими рядками коду, а цілими компонентами або його частинами, що можуть легко масштабуватись у майбутньому завдяки їхній автономності. Крім економії часу, такий підхід допомагає інженерам ПЗ підтримувати порядок у коді, розбиваючи його на компоненти, що відповідають за різні функціональності.

Говорячи про рівень продуктивності застосунків, що використовують Angular у якості технології для розроблення клієнтської частини, важливо зазначити, що він є досить високим. Оскільки браузер не розуміє компонентів та HTML-шаблонів, з яких складається застосунок Angular, він вимагає виконання компіляції. Швидка візуалізація у браузері досягається завдяки AOT Compilation – компіляції коду Angular у JavaScript на етапі збірки програми ще до запуску застосунку у браузері [13]. Таким чином, процес завантаження і запуску проходить значно швидше, що матиме позитивний вплив на продуктивність роботи програми.

Іншою важливою особливістю Angular є те, що даний фреймворк може взаємодіяти з необмеженою кількістю інших інструментів. Проте при підключенні інших бібліотек важливо перевіряти їх на сумісність, оскільки через неправильне їхнє налаштування можуть виникнути проблеми.

Отже, серед особливостей фреймворку Angular можна виділити високу продуктивність, використання декларативної парадигми програмування, підтримку компонентного підходу до архітектури та необмежену кількість бібліотек для взаємодії.

2.2.3. *Vue.js*

Vue.js – фреймворк для створення інтерактивного користувацького інтерфейсу складних вебзастосунків. Дана технологія є молодшою у порівнянні з React.js та Angular, проте, незважаючи на це, є не менш популярною серед розробників ПЗ, завдяки чому ком'юніті Vue.js постійно зростає.

Vue.js відрізняється від своїх аналогів тим, що для реалізації тієї самої задачі він потребує мінімальну кількість коду. Завдяки цьому його робота є простою та швидкою, а код є легким та читання, що має позитивний вплив на продуктивність і час запуску програми. Крім лаконічності коду, на продуктивність застосунку впливає і використання Virtual DOM. Як і в React.js, Virtual DOM дає можливість Vue.js оновлювати лише ті частини сторінок, де дійсно відбулись зміни.

Vue.js підтримує компонентний підхід до архітектури, але на відміну від React.js, він не є таким гнучким. Тому його використання може обмежувати розробників у зручності та швидкості створення програми.

Що стосується екосистеми Vue.js, то даний фреймворк прогресивно розвивається, завдяки чому збільшується кількість власних бібліотек та полегшується процес взаємодії з великою кількістю різноманітних сторонніх інструментів.

Отже, Vue.js має наступні особливості: підтримка компонентного підходу, лаконічність та простота коду, висока продуктивність, багата екосистема.

2.2.4. Результати проведеного аналізу

Побудуємо порівняльну таблицю технологій для реалізації клієнтської частини застосунку за критеріями, сформованими на початку проведення аналізу.

Таблиця 2.2

Порівняння технологій для реалізації клієнтської частини застосунку

Технологія	React.js	Angular	Vue.js
Критерій порівняння			
Підтримка компонентного підходу до архітектури	+	+	+/-
Висока швидкість оновлення стану сторінки	+	+	+
Лаконічність коду	+	+/-	+
Великий вибір бібліотек	+	+/-	+
Висока продуктивність	+	+	+

Отже, з порівняльної таблиці можна побачити, що React.js бездоганно впорався з усіма поставленими вимогами. Дана бібліотека підтримує гнучкий компонентний підхід до архітектури, який, у свою чергу, допомагає простіше будувати складні програми. Це досягається завдяки розбиттю великої частини на декілька складових і маніпулюванню цими елементами у різних місцях системи. Крім цього, React.js має високу швидкість оновлення стану сторінки і рівень продуктивності, оскільки використовує Virtual DOM. Що стосується коду, то він є лаконічним і легким для читання.

Якщо говорити про Angular, то даний фреймворк має 2 недоліки, які вплинули на вибір технології не в його користь: лаконічність коду та залежність від інших бібліотек. Щоб досягти один і той самий результат, Angular потребує більше коду, ніж React.js та Vue.js.

Що стосується Vue.js, він гарно впорався практично з усіма вимогами, висунутими до технології для розроблення клієнтської частини застосунку. Проте наявний і його недолік, а саме низький рівень гнучкості компонентного підходу до архітектури у порівнянні з React.js.

Отже, інструментом для розроблення користувацького інтерфейсу вебзастосунку для бронювання переговорних кімнат офісу обрано бібліотеку React.js.

2.3. Вибір СКБД

СКБД – програма, що відповідає за керування даними бази даних. Вона надає можливість додавати, зберігати, вносити зміни, читати, видаляти та виконувати пошук серед інформації в БД з контролем доступу до неї. Дані маніпуляції проводяться за допомогою мови запитів DML [14].

Оскільки найважливішим критерієм для вибору СКБД є доступність та безкоштовність їх використання, для порівняльного аналізу обрано лише відкриті системи, серед яких: PostgreSQL, MongoDB та MySQL. До СКБД, яка використовується для зберігання даних вебзастосунку для бронювання переговорних кімнат офісу, виділено наступні вимоги:

- 1) підтримка великого обсягу даних і запитів;
- 2) захищеність інформації, що зберігається у СКБД, від несанкціонованого доступу та видалення;
- 3) підтримка реплікації;
- 4) можливість відновити дані;
- 5) підтримка індексів;
- 6) відповідність вимогам ACID.

2.3.1. PostgreSQL

PostgreSQL [15] – безкоштовна об’єктно-реляційна система керування базами даних, яка є досить поширеною серед розробників ПЗ.

Особливостями PostgreSQL, що виділяють її серед інших СКБД є:

- 1) безкоштовність, що дає можливість використовувати усі її функціональні можливості без обмежень;
- 2) підтримка різних форматів даних, наприклад, XML, CSV, JSON;
- 3) безпечний і стабільний стан бази даних завдяки виконанню вимог ACID [16];
- 4) високий рівень надійності: PostgreSQL забезпечує відновлення даних після збоїв;
- 5) масштабованість, завдяки чому дана СКБД може працювати з великими обсягами даних і обробляти запити;
- 6) розширюваність: є можливість додавати власні оператори, індекси і перетворення типів [17];
- 7) підтримка декількох видів реплікації (копіювання і розподілу даних між серверами PostgreSQL для створення резервних копій даних, збільшуючи надійність і безпеку даних): асинхронна потокова, логічна та реплікація за допомогою плагінів [15].

Незважаючи на те, що PostgreSQL є потужною і надійною СКБД, вона має і свої недоліки, серед яких:

- 1) великий розмір файлів бази даних;
- 2) можлива висока складність при налаштуванні СКБД для подальшого користування нею;
- 3) відсутність вбудованого кластерування, що може мати негативні наслідки при роботі з великими базами даних.

2.3.2. MongoDB

MongoDB [18] – одна з найпопулярніших документо-орієнтованих (нереляційних) СКБД з відкритим кодом. Вона є гнучкою та простою у використанні, а синтаксис запитів MongoDB нагадує мову програмування JavaScript.

Основними перевагами СКБД MongoDB є:

- 1) підтримка індексів;
- 2) управління файлами, що мають подібний до JSON формат;
- 3) зберігання даних у вигляді документів, колекцій, у бінарному форматі BSON [17];
- 4) доступність та надійність даних високого рівня за рахунок підтримки кількох видів реплікації;
- 5) наявність журналізації усіх операцій і можливість виконати відновлення даних;
- 6) висока швидкість роботи з великою кількістю даних;
- 7) простий і інтуїтивно зрозумілий інтерфейс, що значно полегшує маніпулювання даними.

Якщо говорити про наявні недоліки MongoDB, то серед них можна виділити:

- 1) не повну відповідність принципам ACID (MongoDB не підтримує узгодженість та ізолюваність, проте задовольняє вимоги до атомарності та довговічності);
- 2) високу вартість налаштування кластерів, якщо виникне потреба у масштабуванні бази даних;
- 3) через відсутність підтримки операції JOIN складні запити до бази даних є більш важкими та займають більше часу на виконання.

2.3.3. MySQL

MySQL [19] – реляційна СКБД з відкритим кодом, що працює на усіх платформах. Дана система користується попитом серед проєктів різного розміру та складності, адже її легко масштабувати.

MySQL має значні переваги, що виділяють її серед інших систем керування базами даних, наприклад:

- 1) стабільність та надійність, що досягається завдяки відповідності усім стандартам ACID;

- 2) підтримка реплікації, за рахунок якої можливе відновлення даних і стійкість до відмов;
- 3) інтуїтивно зрозумілий інтерфейс, що допомагає користувачам з легкістю керувати даними;
- 4) швидкий пошук, сортування та оброблення запитів завдяки підтримці індексів;
- 5) захищеність даних.

Крім переваг, наявні і недоліки СКБД MySQL, серед яких виділено наступні:

- 1) відсутність вбудованої підтримки XML та підтримка JSON недостатнього рівня;
- 2) вразливість до атак у випадку, якщо розробник не дотримується певних вимог безпеки;
- 3) проблеми з масштабованістю, у зв'язку з чим виникають обмеження можливостей при роботі з великими обсягами даних;
- 4) повільний запис даних при внесенні великої кількості інформації до бази даних;
- 5) рідке оновлення MySQL.

2.3.4. Результати проведеного аналізу

Насамперед важливо зазначити, що кожна з проаналізованих СКБД (PostgreSQL, MongoDB та MySQL) впоралась з більшістю поставлених вимог, проте в однієї з них це вийшло досконало.

PostgreSQL задовольнила абсолютно всі вимоги. Що стосується наведених її недоліків: складність налаштування та великий розмір бази даних, то вони ніяк не вплинуть на зручність її використання, адже дана СКБД уже завантажена на персональному комп'ютері, що використовується при реалізації вебзастосунку.

MongoDB також показала високий результат, окрім одного з найважливіших критеріїв порівняння: відповідності вимогам ACID. Дані

стандарти є досить важливими при керуванні даними, тому наявність прогалин у них неприпустима при створенні вебзастосунку, що зберігає конфіденційну інформацію.

Якщо говорити про СКБД MySQL, то вона має дві часткові невідповідності поставленим перед нею вимогами: підтримку великого обсягу даних і запитів, а також захищеність інформації. На жаль, дана СКБД є вразливою до атак, що несе за собою ризики незахищеності даних, які зберігаються у ній. Що стосується масштабованості, MySQL значно обмежує функціональні можливості при роботі з великим обсягом даних і запитів.

Для наочного відображення результатів аналізу наведено порівняльну таблицю за сформованими критеріями.

Таблиця 2.3

Порівняння СКБД

СКБД Критерій порівняння	PostgreSQL	MongoDB	MySQL
Підтримка великого обсягу даних і запитів	+	+	+/-
Захищеність інформації, що зберігається у СКБД	+	+	+/-
Підтримка реплікації	+	+	+
Можливість відновити дані	+	+	+
Підтримка індексів	+	+	+
Відповідність вимогам ACID	+	+/-	+

Отже, у результаті проведеного аналізу та порівняння найпопулярніших СКБД, для керування базами даних вебзастосунку для бронювання переговорних кімнат обрано PostgreSQL.

2.4. Висновки до розділу 2

Під час роботи над другим розділом пояснювальної записки виконано ряд завдань:

- 1) сформовано перелік основних вимог до засобів реалізації програмного продукту: мови програмування для розроблення серверної частини, технологій для реалізації клієнтської частини застосунку та СКБД;
- 2) виділено найпопулярніші безкоштовні засоби реалізації;
- 3) проведено аналіз обраних інструментів, наведено переваги та недоліки кожного з них;
- 4) побудовано порівняльні таблиці за сформованими раніше критеріями;
- 5) сформульовано висновки порівняння та обрано по одному інструменту для реалізації серверної, клієнтської частин застосунку та СКБД.

3. СТРУКТУРНО-АЛГОРИТМІЧНА ОРГАНІЗАЦІЯ СИСТЕМИ

Даний розділ пояснювальної записки включає у себе результати виконання переліку наступних завдань:

- 1) аналіз проблематики та цілей вебзастосунку;
- 2) формулювання вимог до ПЗ;
- 3) побудова архітектури системи;
- 4) проєктування структур даних;
- 5) розроблення алгоритмів роботи вебзастосунку.

3.1. Опис вимог до розроблюваної системи

Вимога – представлення потреби у формі, зрозумілій для усіх учасників, що залучені до її реалізації. Це означає, що вимоги до ПЗ повинні задовольняти критерії SMART, тобто бути:

- 1) S (specific) – чітко визначеними;
- 2) M (measurable) – вимірюваними;
- 3) A (assignable) – мати виконавців;
- 4) R (realistic) – реальними для реалізації;
- 5) T (time-related) – обмеженими у часі [20].

Виявлення та аналіз вимог є одним з найперших та найважливіших етапом при створенні програмного продукту. Справа в тому, що внесення змін до вимог уже під час розроблення системи займе набагато більше часу та інших ресурсів, ніж якби ці ризики були відомі до початку роботи над продуктом. Для того, щоб проєкт був завершений вчасно та відповідав усім очікуванням зацікавлених сторін, вимоги повинні бути ретельно проаналізованими та чітко сформованими. Тому не варто нехтувати важливістю даного процесу.

Якщо говорити про типи вимог, то їх поділяють на 4 рівні:

- 1) бізнес-вимоги – ті, що описують користь, яку принесе бізнесу система, що реалізовується;

- 2) вимоги зацікавлених сторін – ті, що описують потреби усіх стейкхолдерів у рамках ПЗ, яке створюється;
- 3) вимоги до рішення:
 - функціональні – ті, які описують, що саме повинна вміти робити система для того, щоб задовольнити вимоги зацікавлених сторін;
 - нефункціональні – ті, що описують, як і при яких умовах система повинна виконувати певні дії, а також якості ПЗ;
- 4) перехідні вимоги – ті, що описують, яка має бути система для подальшого її масштабування.

3.1.1. *Аналіз проблематики та цілей вебзастосунку*

Важливо, щоб реалізоване ПЗ приносило користь своїм потенційним користувачам, тому при описі вимог до вебзастосунку для бронювання переговорних кімнат потрібно приділяти увагу проблемам, з якими стикаються відвідувачі офісу. Відповідно до результатів проведеного аналізу предметної галузі та огляду проблем потенційних користувачів системи сформовано наступний перелік проблем, які заважають відвідувачам офісу ефективно бронювати переговорні кімнати:

- 1) наявність конфліктів зустрічей в одній переговорній кімнаті;
- 2) відсутність у кімнаті обладнання, необхідного для проведення зустрічі;
- 3) відсутність сповіщення користувачів про створення, редагування або скасування зустрічі, на яку вони запрошені.

Дерево проблем з декомпозицією перелічених раніше пунктів наведене на рис. 3.1.

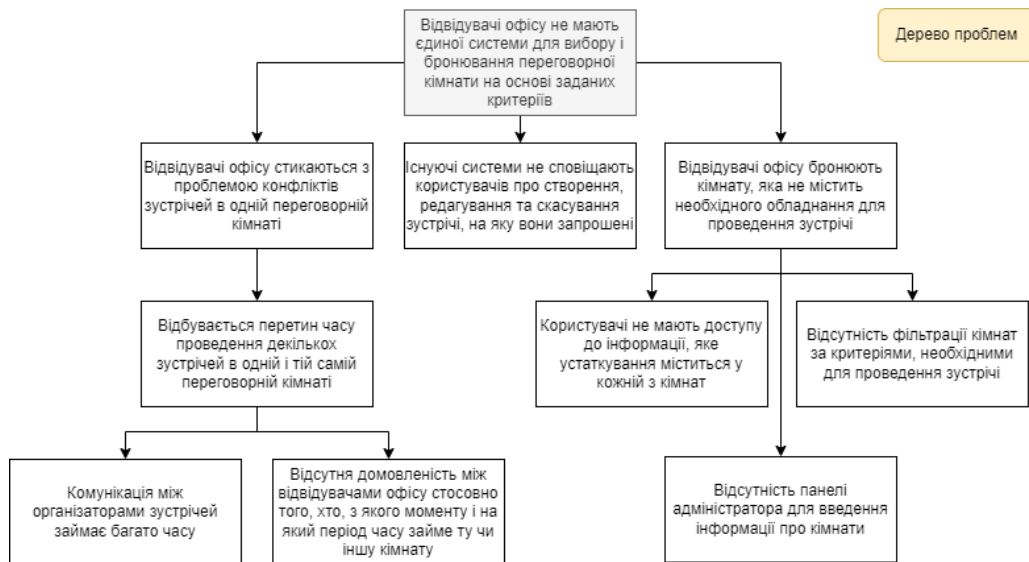


Рис. 3.1. Дерево проблем відвідувачів офісу

Головною метою проєкту є створення програмного продукту, що забезпечує можливість бронювати переговорні кімнати офісу. Реалізація ПЗ включає розроблення п'яти складових системи: модулю, що дозволяє переглядати завантаженість кімнат на день, тиждень або місяць; створення бронювання, що опирається на алгоритм перевірки кімнати на зайнятість у певний період часу; нотифікації користувачів про створення, внесення змін або скасування зустрічі, на яку вони запрошені; панелі адміністратора для введення даних про кімнати, а також модулю для фільтрації переговорних кімнат за заданими параметрами. Усі ці цілі, враховуючи дерево проблем, об'єднано в ієрархічну структуру, що називається деревом цілей (рис. 3.2).



Рис. 3.2. Дерево цілей розроблення вебзастосунку

Далі наведено дерево результатів розроблення вебзастосунку для бронювання переговорних кімнат офісу, що базується на дереві цілей, які мають бути досягнуті під час виконання проєкту.



Рис. 3.3. Дерево результатів розроблення вебзастосунку

3.1.2. Формулювання вимог до програмного забезпечення

Існує декілька способів представлення вимог: сценарії використання, користувацькі історії, критерії приймання, прототипи, діаграми різної складності та деталізації і т.д. При формулюванні вимог важливо визначити, у якому з наведених форматів зручніше буде донести потрібну інформацію так, аби її могли зрозуміти усі зацікавлені сторони: клієнти, що орієнтуються на покращення бізнес-процесів, розробники ПЗ, яким потрібно реалізувати описане, QA-інженери, що повинні перевірити відповідність створеного програмного продукту вимогам до нього.

Для опису вимог до вебзастосунку для бронювання переговорних кімнат офісу обрано формат сценаріїв використання (use case). Він найкраще відображає взаємодію між користувачем і системою, оскільки показує поведінку застосунку на ту чи іншу дію. Use case представляє собою покроковий опис запитів користувачів і відповідей системи на них для досягання певної цілі користувача.

Сценарій використання включає у себе наступну інформацію:

- 1) ідентифікатор і назву;

- 2) акторів;
- 3) початкові умови;
- 4) очікуваний результат;
- 5) основний потік;
- 6) альтернативні потоки.

Нижче в описаному форматі наведено сценарії використання вебзастосунку для бронювання переговорних кімнат офісу.

Таблиця 3.1

UC-1: Зареєструватись у системі

UC-1	Зареєструватись у системі
Актори	Незареєстровані відвідувачі вебсайту
Початкові умови	Користувач відкрив вебсайт
Очікуваний результат	Користувач має логін і пароль, за допомогою яких може здійснити вхід у систему
Основний потік	1. Система відображає форму для входу у систему. 2. Користувач натискає на клавішу «Sign up». 3. Система відображає форму для реєстрації, яка містить поля для введення логіну (електронної адреси), паролю і підтвердження паролю, клавішу «Sign up». 4. Користувач заповнює поля. 5. Користувач натискає на клавішу «Sign up». 6. Система перевіряє заповнені поля. 6a. Одне або декілька полів пусті. 6b. Поля паролю і підтвердження паролю не співпадають. 6c. Формат логіну некоректний. 6d. Серед завантажених адміністратором користувачів системи немає введеного логіну. 7. Система направляє користувача на головну сторінку.

Альтернативні потоки	<u>6a. Одне або декілька полів пусті.</u> 6a-1. Система виділяє червоним пропущені поля і відображає помилку «Password is required». 6a-2. Користувач заповнює пропущені поля. 6a-3. Повернення до кроку 5 основного потоку.
	<u>6b. Поля паролю і підтвердження паролю не співпадають.</u> 6b-1. Система виділяє червоним поле підтвердження паролю і відображає помилку «Passwords do not match». 6b-2. Користувач повторно заповнює поля паролю і підтвердження паролю. 6b-3. Повернення до кроку 5 основного потоку.
	<u>6c. Формат логіну некоректний.</u> 6c-1. Система виділяє червоним поле логіну і відображає помилку «Incorrect format: @ or . is missed». 6c-2. Користувач повторно заповнює поле логіну. 6c-3. Повернення до кроку 5 основного потоку.
	<u>6d. Серед завантажених адміністратором користувачів системи немає введенного логіну.</u> 6d-1. Система виділяє червоним поле логіну і відображає помилку «There is no user with such email in the system». 6d-2. Користувач повторно заповнює поле логіну. 6d-3. Повернення до кроку 5 основного потоку.

Таблиця 3.2

UC-2: Авторизуватись у системі

UC-2	Авторизуватись у системі
Актори	Зареєстровані відвідувачі вебсайту
Початкові умови	Користувач попередньо успішно зареєструвався, має логін і пароль

Очікуваний результат	Користувач має доступ до основної функціональності системи
Основний потік	1. Система відображає форму для входу у систему з полями для введення логіну і паролю, клавiшу «Sign in». 2. Користувач заповнює поля. 3. Користувач натискає на клавiшу «Sign in». 4. Система перевіряє заповнені поля. 4a. Одне або декілька полів пусті. 4b. Формат логіну некоректний. 4c. Введений пароль не відповідає захешованому паролю. 4d. Серед завантажених адміністратором користувачів системи немає введенного логіну. 5. Система направляє користувача на головну сторінку.
Альтернативні потоки	<u>4a. Одне або декілька полів пусті.</u> 4a-1. Система виділяє червоним пропущені поля і відображає помилку «Password is required». 4a-2. Користувач заповнює пропущені поля. 4a-3. Повернення до кроку 3 основного потоку.
	<u>4b. Формат логіну некоректний.</u> 4b-1. Система виділяє червоним поле логіну і відображає помилку «Incorrect format: @ or . is missed». 4b-2. Користувач повторно заповнює поле логіну. 4b-3. Повернення до кроку 3 основного потоку.
	<u>4c. Введений пароль не відповідає захешованому паролю.</u> 4c-1. Система виділяє червоним поле паролю і відображає помилку «Invalid password». 4c-2. Користувач повторно заповнює поле паролю. 4c-3. Повернення до кроку 3 основного потоку.

	<u>4d. Серед завантажених адміністратором користувачів системи немає введеного логіну.</u> 4d-1. Система виділяє червоним поле логіну і відображає помилку «Invalid credentials». 4d-2. Користувач повторно заповнює поле логіну. 4d-3. Повернення до кроку 3 основного потоку.
--	---

Таблиця 3.3

UC-3: Переглянути завантаженість переговорних кімнат станом на день, тиждень та місяць

UC-3	Переглянути завантаженість переговорних кімнат станом на день, тиждень та місяць
Актори	Авторизований користувач системи
Початкові умови	Користувач здійснив вхід, знаходиться на головній сторінці вебзастосунку
Очікуваний результат	Користувач має можливість переглянути вільні та заброньовані години усіх переговорних кімнат
Основний потік	1. Система відображає таблицю з усіма переговорними кімнатами офісу з обраним видом «Day» станом на поточну дату. 2. Користувач натискає на іншу дату в календарі. 3. Система відображає таблицю завантаженості станом на обрану користувачем дату. 4. Користувач обирає «Week» вигляд таблиці. 5. Система відображає таблицю завантаженості кімнат на період тижня з понеділка до неділі, у якому міститься поточна дата. 6. Користувач виділяє дати іншого тижня на календарі. 7. Система відображає таблицю станом на обраний користувачем тиждень. 8. Користувач обирає «Month» вигляд таблиці.

Основний потік	9. Система відображає таблицю завантаженості кімнат на місяць, у якому міститься поточна датає. 10. Користувач виділяє дати іншого місяця на календарі. 11. Система відображає таблицю станом на обраний користувачем місяць.
----------------	--

Таблиця 3.4

UC-4: Створити бронювання переговорної кімнати

UC-4	Створити бронювання переговорної кімнати
Актори	Авторизований користувач системи
Початкові умови	Користувач здійснив вхід, знаходиться на головній сторінці вебзастосунку
Очікуваний результат	Бронювання успішно створено, обраний період часу позначається зайнятим у таблиці завантаженості кімнат
Основний потік	1. Система відображає таблицю з усіма переговорними кімнатами офісу з обраним видом «Day» станом на поточну дату. 2. Користувач обирає у календарі дату, на яку хочу забронювати переговорну кімнату. 3. Система оновлює вигляд таблиці станом на обрану користувачем дату. 4. Користувач натискає на клітинку таблиці у рядку, який відповідає за певну кімнату. 5. Система перевіряє, чи зайнята дана кімната у час, що був виділений користувачем. 5a. Кімната заброньована даним користувачем у виділений період часу. 5b. Кімната заброньована іншим користувачем у виділений період часу. 6. Система відкриває сторінку для створення бронювання обраної переговорної кімнати на обрану дату. 7. Користувач обирає час початку його бронювання.

Основний потік	8. Система автоматично заповнює кінцевий час зустрічі, що є на 15 хвилин більше, ніж початок бронювання. 9. Користувач заповнює усі необхідні поля і натискає на клавішу «Submit». 10. Система перевіряє усі заповнені поля на коректність. 11. Система зберігає дані і відображає час збереженого бронювання у таблиці завантаженості кімнат. 12. Система перенаправляє користувача на головну сторінку системи.
Альтернативні потоки	<u>5a. Кімната заброньована даним користувачем у виділений період часу.</u> 5a-1. Система відображає користувачеві модальне вікно з клавішами для редагування та видалення власного бронювання.
	<u>5b. Кімната заброньована іншим користувачем у виділений період часу.</u> 5b-1. Система відображає модальне вікно з інформацією про дане бронювання та клавішою для надсилання електронного листа його організатору.

Таблиця 3.5

UC-5: Використати фільтрацію переговорних кімнат для вибору
підходящої

UC-5	Використати фільтрацію переговорних кімнат для вибору підходящої
Актори	Авторизований користувач системи
Початкові умови	Користувач здійснив вхід, знаходиться на головній сторінці вебзастосунку
Очікуваний результат	У таблиці завантаженості кімнат відображаються лише ті кімнати, що задовольняють параметри фільтрації
Основний потік	1. Система відображає сторінку з усіма переговорними кімнатами офісу і параметрами для їхньої фільтрації.

Основний потік	2. Користувач обирає необхідні йому параметри з запропонованих. 3. Користувач натискає на клавішу «Filter». 4. Система перевіряє, чи є кімнати, які задовольняють усі обрані параметри фільтрації:. 4а. Жодна з кімнат не задовольняє обрані параметри. 5. Система відображає у таблиці завантаженості лише ті кімнати, що задовольняють обрані користувачем параметри фільтрації.
Альтернативні потоки	<u>4а. Жодна з кімнат не задовольняє обрані параметри.</u> 4а-1. Система відображає пусту таблицю завантаженості і повідомлення про те, що за заданими параметрами фільтрації не знайдено результатів. 4а-2. Користувач змінює параметри фільтрації. 4а-3. Повернення до кроку 3 основного потоку.

Таблиця 3.6

UC-6: Надіслати електронний лист організатору зустрічі

UC-6	Надіслати електронний лист організатору зустрічі
Актори	Авторизований користувач системи
Початкові умови	Користувач здійснив вхід, знаходиться на головній сторінці вебзастосунку
Очікуваний результат	Користувач направляється на поштовий сервер
Основний потік	1. Система відображає таблицю завантаженості переговорних кімнат. 2. Користувач натискає на зафарбовану клітинку таблиці. 3. Система перевіряє, чи організатором обраного бронювання є інший користувач. За. Організатором обраного бронювання є даний користувач.

Основний потік	4. Система відображає модальне вікно з інформацією про обране бронювання і клавішу «Contact». 5. Користувач натискає на клавішу «Contact». 6. Система направляє користувача на поштовий сервер зі створеним електронним листом, у якому отримувачем є організатор обраного бронювання, автором – даний користувач, темою листа – назва, дата та час бронювання.
Альтернативні потоки	<u>За. Організатором обраного бронювання є даний користувач</u> За-1. Система відображає користувачеві модальне вікно з клавішами для редагування та видалення власного бронювання

Таблиця 3.7

UC-7: Редагувати власне бронювання

UC-7	Редагувати власне бронювання
Актори	Авторизований користувач системи
Початкові умови	Користувач здійснив вхід, знаходиться на головній сторінці вебзастосунку, має створене ним бронювання
Очікуваний результат	Система зберігає внесені зміни до бронювання
Основний потік	1. Користувач натискає на зафарбовану клітинку таблиці 2. Система перевіряє, чи організатором обраного бронювання є даний користувач. 2а. Організатором обраного бронювання є інший користувач. 3. Система відображає модальне вікно з інформацією про обране бронювання і клавішами «Edit» та «Delete». 4. Користувач натискає на клавішу «Edit». 5. Система направляє користувача на сторінку редагування власного бронювання.

Основний потік	6. Користувач вносить необхідні зміни. 7. Користувач натискає на клавішу «Submit», підтверджуючи внесені зміни. 8. Система перевіряє усі заповнені поля на коректність. 9. Система зберігає дані і відображає час збереженого бронювання у таблиці завантаженості кімнат.
Альтернативні потоки	<u>2а. Організатором обраного бронювання є інший користувач</u> 2а-1. Система відображає модальне вікно з інформацією про дане бронювання.

Таблиця 3.8

UC-8: Скасувати власне бронювання

UC-8	Скасувати власне бронювання
Актори	Авторизований користувач системи
Початкові умови	Користувач здійснив вхід, знаходиться на головній сторінці вебзастосунку, має створене ним бронювання
Очікуваний результат	Бронювання скасоване
Основний потік	1. Система відображає таблицю завантаженості переговорних кімнат. 2. Користувач натискає на зафарбовану клітинку таблиці. 3. Система перевіряє, чи організатором обраного бронювання є даний користувач. За. Організатором обраного бронювання є інший користувач. 4. Система відображає модальне вікно з детальною інформацією про обране бронювання і клавішами «Edit» та «Delete». 5. Користувач натискає на клавішу «Delete».

Основний потік	6. Система відображає модальне вікно з інформацією про бронювання, попередженням про те, що бронювання буде видалено і клавішами «Yes» та «No». 7. Користувач натискає на «Yes». 8. Система зберігає зміни, видаляє бронювання і змінює період часу видаленого бронювання з зайнятого та вільний. 9. Система перенаправляє користувача на головну сторінку системи.
Альтернативні потоки	<u>За. Організатором обраного бронювання є інший користувач.</u> За-1. Система відображає модальне вікно з інформацією про дане бронювання та клавішою для надсилання електронного листа його організатору.

З огляду на описані сценарії використання вебзастосунку для бронювання переговорних кімнат офісу і з врахуванням дерева проблем, сформовано таблицю з переліком функціональних вимог, які повинна задовольняти система, що створюється у рамках дипломного проєкту.

Таблиця 3.9

Реєстр функціональних вимог до вебзастосунку

Код вимоги	Зміст вимоги	Атрибути	
		Пріоритет	Складність
F1	Можливість реєстрації у системі за допомогою електронної пошти	Високий	Середня
F2	Можливість авторизації звичайним користувачем за електронною поштою та паролем	Високий	Середня
F3	Можливість авторизації як адміністратор системи за електронною поштою та паролем	Високий	Середня

Продовження табл. 3.9

F4	Можливість перегляду завантаженості переговорних кімнат станом на день, тиждень та місяць	Високий	Висока
F5	Можливість одноразового бронювання переговорної кімнати	Високий	Висока
F6	Можливість створення регулярного бронювання	Середній	Висока
F7	Можливість фільтрації переговорних кімнат за заданими параметрами	Високий	Середня
F8	Можливість надсилання організатору зустрічі електронного листа	Низький	Середня
F9	Можливість перегляду інформації про обране бронювання	Високий	Низька
F10	Можливість редагування власного бронювання	Високий	Середня
F11	Можливість видалення власного бронювання	Високий	Низька
F12	Можливість перегляду списку власних бронювань	Середній	Висока
F13	Можливість доступу до панелі адміністратора	Високий	Середня
F14	Можливість налаштування робочих годин офісу	Високий	Низька
F15	Можливість додавання, редагування та видалення поверху у системі	Високий	Середня
F16	Можливість створення, редагування та видалення кімнати всередині поверху	Високий	Середня
F17	Можливість створення, редагування та видалення інформації про обладнання переговорних кімнат	Середній	Низька

F18	Можливість додавання, редагування та видалення цілей зустрічей	Низький	Низька
F19	Можливість додавання, редагування та видалення інформації про користувачів системи	Високий	Середня
F20	Можливість виконання експорту усіх користувачів	Середній	Висока
F21	Можливість виконання імпорту користувачів	Середній	Висока
F22	Можливість створення, редагування та видалення команди	Високий	Середня

Наступним кроком є визначення нефункціональних вимог. Усього існує декілька категорій нефункціональних вимог, серед них можна виділити вимоги до безпеки, продуктивності та реалізації. Їхня мета – описати, як саме повинна працювати система, щоб у користувачів не виникало проблем з конфіденційністю даних або швидкістю роботи програми. Нижче наведено нефункціональні вимоги до вебзастосунку для бронювання переговорних кімнат офісу.

Таблиця 3.10

Нефункціональні вимоги до безпеки

Код вимоги	Зміст вимоги
SEC-1	Можливість редагування та видалення бронювання повинні бути доступними лише для його організатора
SEC-2	Доступ до панелі управління системи повинен мати лише його адміністратор
SEC-3	Паролі користувачів повинні зберігатись у захешованому вигляді md5

Таблиця 3.11

Нефункціональні вимоги до продуктивності

Код вимоги	Зміст вимоги
PER-1	Система повинна підтримувати одночасне підключення 1000 користувачів
PER-2	Система повинна виконувати повне завантаження даних у відповідь на запит користувача не довше, ніж 5 секунд
PER-3	Система повинна підтримувати одночасне виконання 200 запитів

Таблиця 3.12

Нефункціональні вимоги до реалізації

Код вимоги	Зміст вимоги
IMP-1	Система повинна коректно працювати у браузері Google Chrome версії 112.0.5615.137 на операційних системах Windows, macOS та Ubuntu Linux
IMP-2	Система повинна бути кросплатформною і коректно працювати на ОС Windows, macOS та Ubuntu Linux
IMP-3	Час початку бронювання повинно бути меншим, ніж час його завершення
IMP-4	Мінімальна тривалість одного бронювання повинна бути 15 хвилин

3.2. Опис архітектури системи

Архітектура програмного продукту – це сукупність принципів, що описують, яким чином система повинна бути розділена на модулі і як ці компоненти мають бути пов’язані між собою.

У сучасному світі розвитку ІТ існує велика кількість типів архітектури ПЗ, які широко застосовуються при створенні систем. При виборі архітектури важливо звертати увагу на декілька факторів, наприклад:

- 1) функціональні та нефункціональні вимоги до системи;
- 2) терміни розроблення ПЗ;
- 3) масштабованість, що дозволить додавати нові функції.

Для реалізації вебзастосунку для бронювання переговорних кімнат офісу обрано клієнт-серверну архітектуру, в основі якої лежить розподілення системи на дві частини: сервер і клієнт. Сервер відповідає за постачання ресурсу, а клієнт – за запит його. Комунікація між цими частинами відбувається за допомогою протоколу запит-відповідь. Насамперед клієнт формує запит з певними даними і надсилає це повідомлення серверу. Сервер отримує запит, обробляє дані всередині нього, формує відповідь з успішного результату або помилки і надсилає клієнту. Нарешті клієнт отримує повідомлення-відповідь і обробляє його.

Серед особливостей клієнт-серверної архітектури важливо виділити наступне:

- 1) розділення функціональності між серверною і клієнтською частинами, завдяки чому жоден з компонентів не є перевантаженим непотрібними даними або складними обчисленнями;
- 2) централізований доступ до даних;
- 3) адаптивність – можливість збільшення кількості клієнтів та серверів;
- 4) захищеність даних;
- 5) простота у роботі з файлами завдяки тому, що вони зберігаються на одному сервері [21];
- 6) відсутність залежності від фреймворків;
- 7) простота у подальшому тестуванні;
- 8) зручність використання.

Архітектура вебзастосунку для бронювання переговорних кімнат офісу побудована за допомогою шаблону проєктування MVC і умовно поділяється на три модулі: Model (модель), View (представлення) та Controller (контролер). Такий підхід забезпечує автономністю кожного з трьох компонентів і дозволяє модифікувати будь-який незалежно від інших двох.

Кожен з компонентів MVC шаблону представляє собою окремий рівень системи. Model є рівнем даних і відповідає саме за оперування ними. View – рівень інтерфейсу програмного продукту, за допомогою якого користувачі взаємодіють з системою. Controller – бізнес-рівень, який відповідає за логіку застосунку.

Узагальнену архітектуру застосунку з використанням патерну проєктування Model-View-Controller наведено на рис. 3.4.

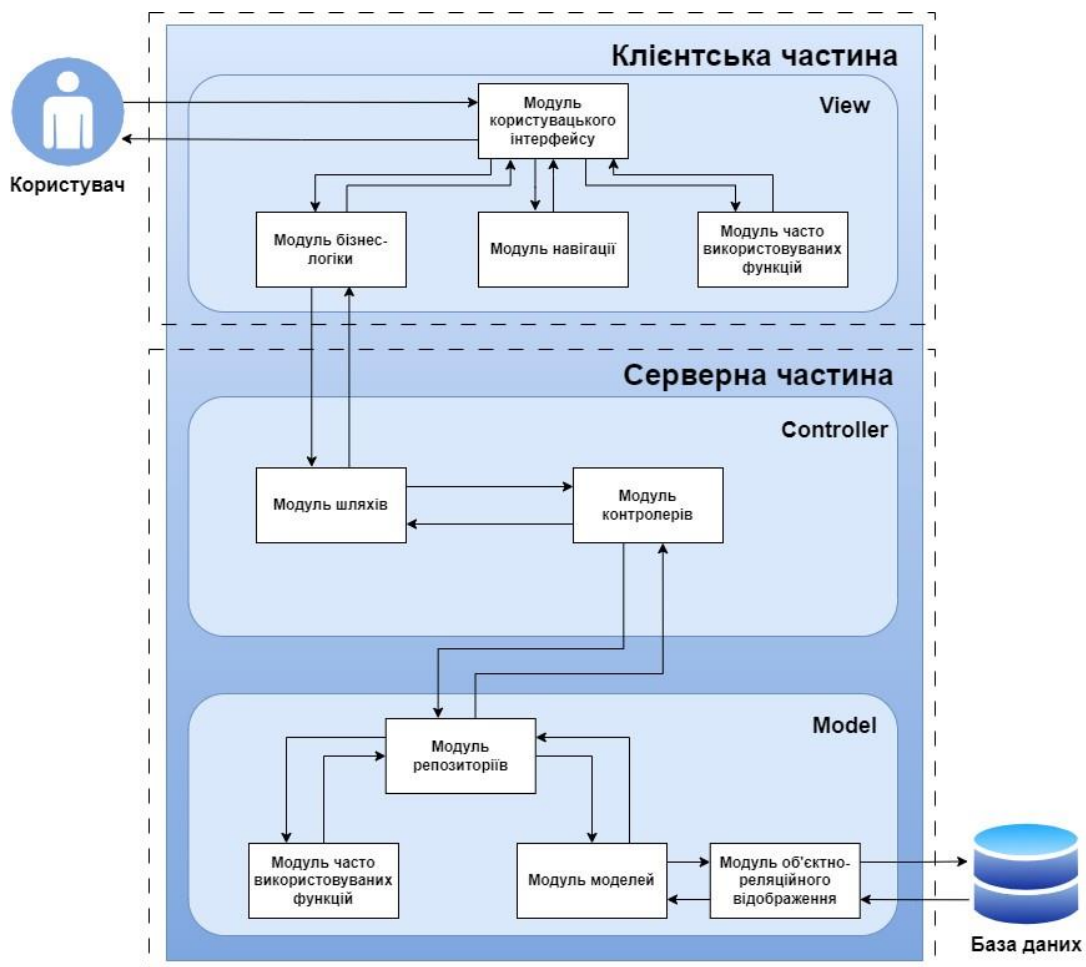


Рис. 3.4. Узагальнена архітектура вебзастосунку

3.3. Опис структур даних системи

Діаграму взаємозв'язків між вище згаданими таблицями наведено на рис. 3.5.

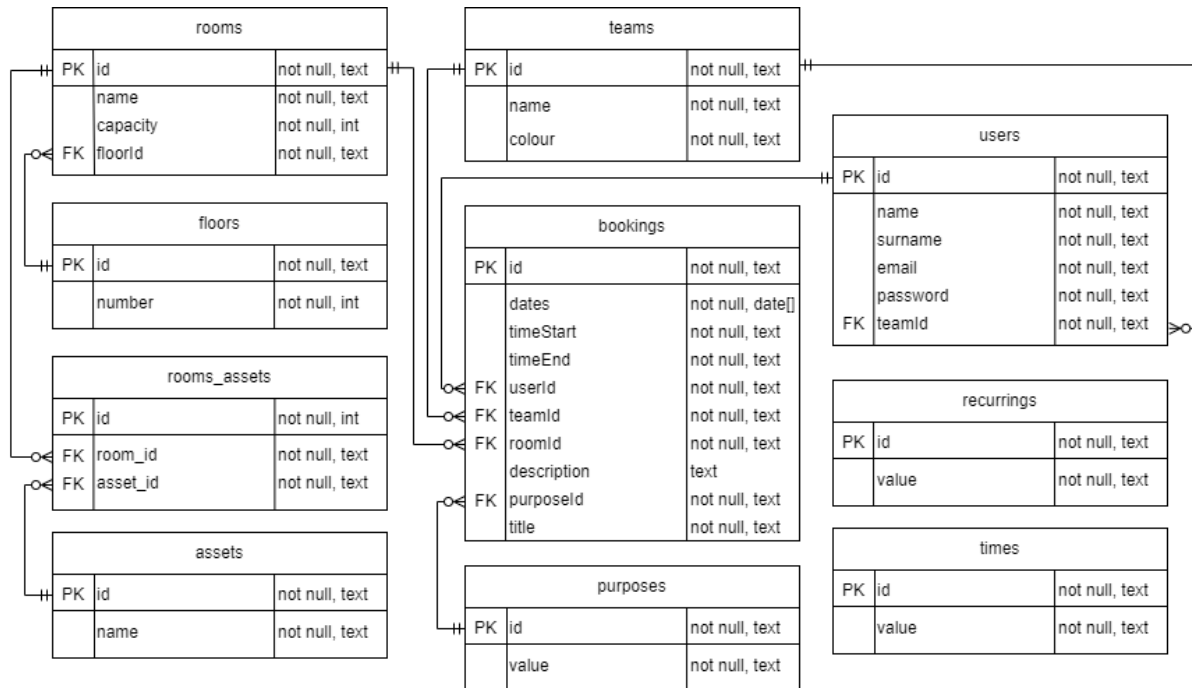


Рис. 3.5. Діаграма взаємозв'язків між сутностями

Далі наведено таблиці даних, що зберігаються у базі даних вебзастосунку для бронювання переговорних кімнат офісу.

1. Бронювання (Booking).

- Id – унікальний ідентифікатор сутності бронювання.
- Title – назва бронювання.
- Dates – масив дат, коли відбуватиметься дане бронювання.
- TimeStart – час початку бронювання.
- TimeEnd – кінцевий час бронювання.
- UserId – зовнішній ключ посилання на таблицю користувачів, ідентифікатор організатора бронювання.
- TeamId – зовнішній ключ посилання на таблицю команд, ідентифікатор команди, доданої до бронювання.

- h. RoomId – зовнішній ключ посилання на таблицю кімнат, ідентифікатор переговорної кімнати, що заброньована на обраний період часу організатором.
 - i. Description – додаткова інформація про бронювання.
 - j. PurposeId – зовнішній ключ посилання на таблицю цілей бронювання.
2. Повторюваності (Recurrings).
- a. Id – унікальний ідентифікатор повторюваності.
 - b. Value – частота повторюваності бронювання (No recurring, Daily, Every two days, Weekly, Monthly).
3. Поверхи (Floors).
- a. Id – унікальний ідентифікатор сутності поверху.
 - b. Number – номер поверху.
4. Кімнати (Rooms).
- a. Id – унікальний ідентифікатор сутності переговорної кімнати.
 - b. Name – назва переговорної кімнати.
 - c. Capacity – максимальна місткість переговорної кімнати.
 - d. FloorId – зовнішній ключ посилання на таблицю поверхів.
5. Обладнання (Assets).
- a. Id – унікальний ідентифікатор сутності обладнання.
 - b. Name – назва обладнання.
6. Цілі (Purposes).
- a. Id – унікальний ідентифікатор сутності цілі бронювання.
 - b. Value – ціль бронювання.
7. Проміжна таблиця, що зв'язує обладнання і кімнати (Rooms_assets).
- a. Id – унікальний ідентифікатор зв'язку між переговорною кімнатою та обладнанням.
 - b. RoomId – зовнішній ключ посилання на таблицю кімнат.
 - c. AssetId – зовнішній ключ посилання на таблицю обладнання.

8. Команди (Teams).

- a. Id – унікальний ідентифікатор сутності команди.
- b. Name – назва команди.
- c. Colour – колір представлення команди.

9. Часи (Times).

- a. Id – унікальний ідентифікатор часу погодинно.
- b. Value – значення часу погодинно.

10. Користувачі (Users).

- a. Id – унікальний ідентифікатор користувача системи.
- b. Name – ім'я користувача.
- c. Surname – прізвище користувача.
- d. Email – електронна пошта користувача.
- e. Password – захешований пароль користувача.
- f. TeamId – зовнішній ключ посилання на таблицю команд.

3.4. Опис реалізованих алгоритмів функціонування системи

Основним алгоритмом у вебзастосунку для бронювання переговорних кімнат офісу є перевірка завантаженості кімнати на певний період часу.

Він застосовується у декількох місцях. Насамперед даний алгоритм вступає у силу, коли користувач знаходиться на сторінці «View availability» з таблицею завантаженості усіх кімнат. Клітинки таблиці, які містять у собі заброньовані періоди часу, зафарбовуються кольорами, що відповідають за команду, для якої створено бронювання. Поля таблиці з вільними часовими періодами позначаються білим кольором. Таким чином, у відвідувача вебзастосунку при перегляді таблиці може з'явитись розуміння, яка кімната є вільною на бажаний день.

Інше місце, де застосовується даний алгоритм – сторінка для створення і редагування бронювання. При натисканні на поля «Start time» та «End time» користувач побачить список доступних меж для букінга з врахуванням інших бронювань на задану дату.

На рис. 3.6 наведено алгоритм перевірки кімнати на зайнятість у певний період часу у вигляді блок-схеми.

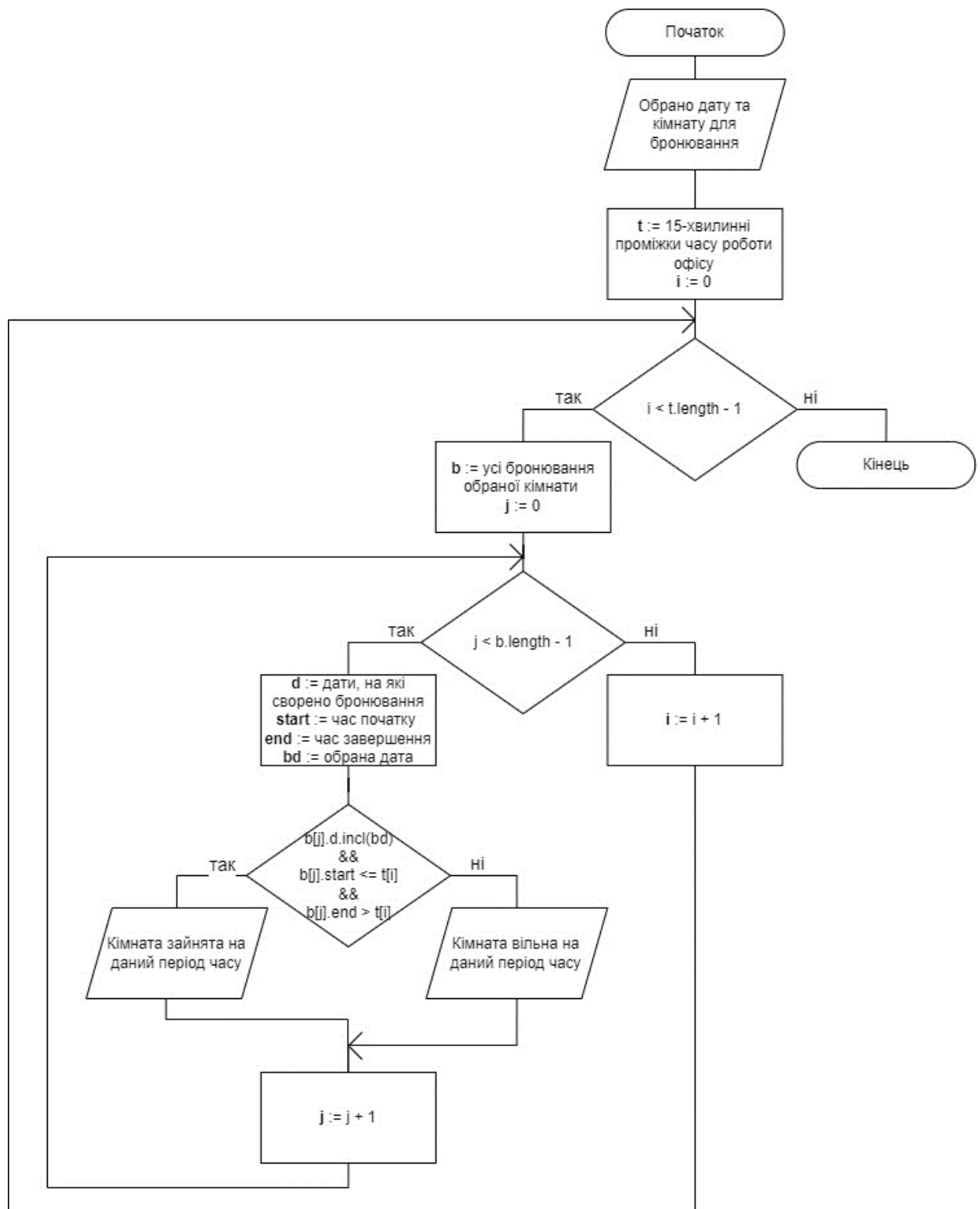


Рис. 3.6. Блок-схема алгоритму перевірки кімнати на зайнятість у певний період часу

Вхідними точками для роботи алгоритму є вибір кімнати та дня у календарі. Далі система визначає усі наявні бронювання даної кімнати на обраний день. Маючи цю інформацію, система починає проходитись 15-хвилинним кроком по періоду, обмеженому годинами роботи офісу. Якщо на ітерації знайшлося бронювання, початковий час якого менший або рівний лівій межі 15-хвилинного інтервалу та кінцевий час більший правої межі, то переговорна кімната є зайнятою на даний період часу. У протилежному випадку, кімната вважається вільною.

Таким чином, результатом алгоритму є визначення зайнятості 15-хвилинного періоду часу обраної переговорної кімнати на задану дату.

3.5. Висновки до розділу 3

У результаті роботи над третім розділом пояснювальної записки виконано перелік поставлених задач:

- 1) проаналізовано проблематику та цілі вебзастосунку, побудовано дерева проблем, цілей та результатів;
- 2) сформульовано функціональні вимоги у вигляді сценаріїв використання, а також нефункціональні вимоги до безпеки, продуктивності та реалізації;
- 3) побудовано та описано архітектуру системи;
- 4) описано спроектовану структуру даних та побудовано діаграму взаємозв'язків між сутностями;
- 5) розроблено алгоритм перевірки кімнати на зайнятість та побудовано його блок-схему.

4. ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ВЕБЗАСТОСУНКУ

Метою даного розділу є опис особливостей реалізації серверної та клієнтської частини системи, створеного користувацького інтерфейсу, надання плану проведеного тестування та напрями подальшого розвитку і покращення вебзастосунку для бронювання переговорних кімнат офісу.

4.1. Особливості реалізації

Вебзастосунок для бронювання переговорних кімнат офісу реалізовувався з використанням найкращих практик написання «чистого коду», а саме: стислих та змістовних назв змінних, відсутності повторюваності фрагментів коду, модульності системи та читабельності її коду. Такий підхід забезпечить розробників можливістю без значних зусиль підтримувати та розширювати функціональні можливості застосунку у майбутньому, а також витратити менше часу на пошук місця появи дефектів.

4.1.1. *Серверна частина вебзастосунку*

Серверна частина вебзастосунку повинна мати можливість взаємодії з СКБД. Це можливо за допомогою виконання певних команд у терміналі, але такий спосіб є складним та часозатратним. Тому у рамках даного продукту для взаємодії з СКБД використовується ORM-інструмент Sequelize, за допомогою якого виконуються запити на сервер бази даних. Його перевага над використанням терміналу є надання базових функцій, що дозволяють виконувати CRUD операції.

При створенні серверної частини використовувались практики написання «чистого коду», одним із принципів якого є модульність. Для кожної з сутностей, наявних у системі, розроблено модулі роутеру, контролеру, репозиторію та моделі. На рис. 4.1 наведено модульну залежність серверної частини вебзастосунку.

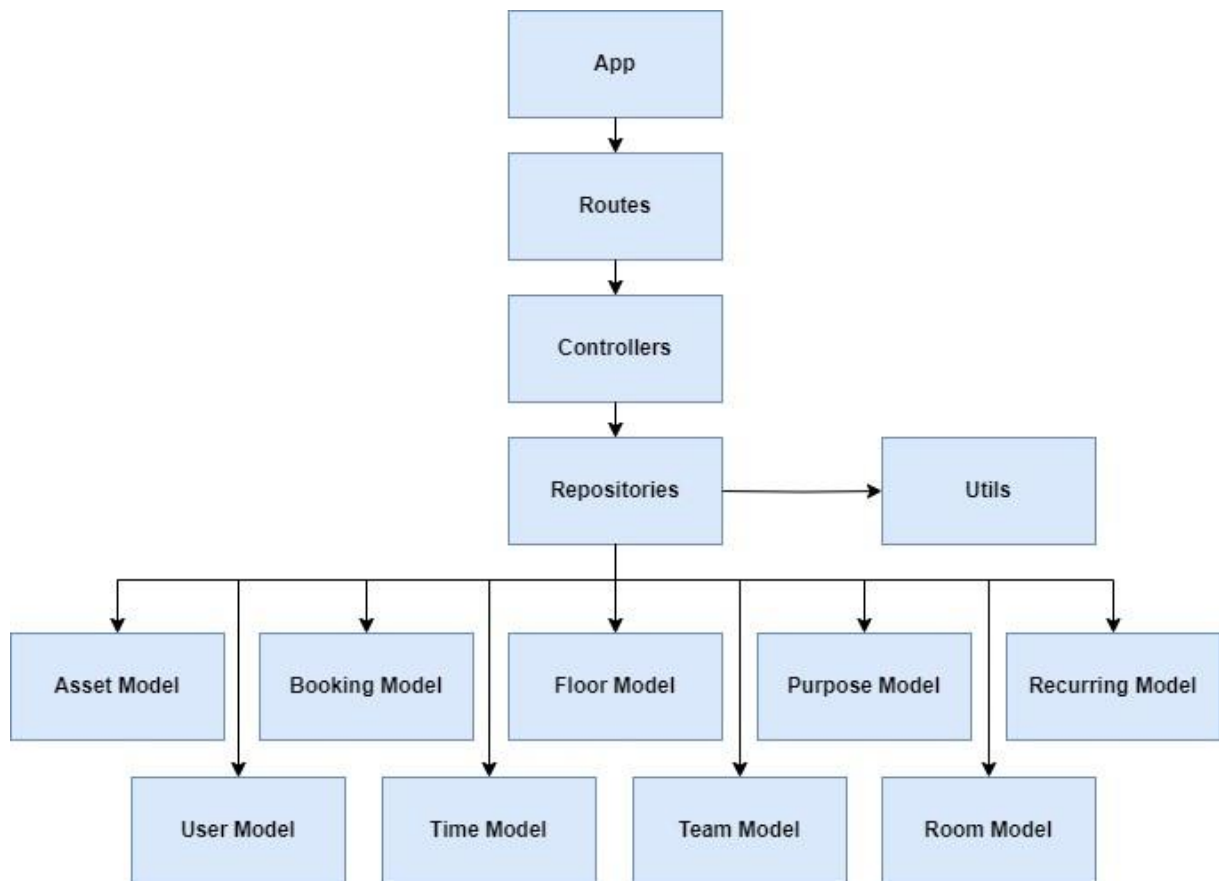


Рис. 4.1. Діаграма модульної залежності серверної частини вебзастосунку

4.1.2. Клієнтська частина вебзастосунку

Перенесення основних обчислювальних функцій на серверну частину вебзастосунку значно полегшує та пришвидшує роботу клієнтської. Проте деяка бізнес-логіка знаходиться на клієнті, наприклад, функції оброблення даних, зміни стану вебсторінки та запитів на сервер. Таке розміщення поряд з логікою відображення користувацького інтерфейсу має негативний вплив на якість коду: його читабельність та складність розроблення. Для того, щоб позбутись цієї проблеми, у даному вебзастосунку виокремлено функції роботи з даними від відображення користувацького інтерфейсу. Це досягнуто завдяки використанню Redux Toolkit – надбудови над бібліотекою для управління станом вебсайту Redux [22].

Для розширення можливостей роботи з даними у форматі дати та часу, що є основною необхідністю при реалізації вебзастосунку для бронювання переговорних кімнат офісу, використано бібліотеку Moment.js [23]. Вона

допомагає відображати, формувати, порівнювати та маніпулювати даними про час та дати.

Крім цього, при розробленні програмного продукту використовувались і інші бібліотеки. У вебзастосунку реалізована взаємодія з файловою системою (імпорт та експорт CSV-файлів) за допомогою бібліотеки File-loader [24]. Надсилання електронних листів відбувається завдяки Nodemailer [25].

4.2. Опис реалізованого користувацького інтерфейсу

Користувацький інтерфейс системи для бронювання переговорних кімнат офісу реалізовано у вигляді односторінкового вебзастосунку, що містить у собі наступні складові:

- 1) сторінку реєстрації та авторизації;
- 2) головну сторінку з таблицею завантаженості переговорних кімнат;
- 3) сторінку для створення бронювання;
- 4) сторінку для редагування бронювання;
- 5) сторінку для перегляду власних бронювань;
- 6) сторінку для налаштування робочих годин офісу;
- 7) сторінку для управління сутностями поверхів та переговорних кімнат офісу;
- 8) сторінку для управління обладнанням кімнат;
- 9) сторінку для управління цілями зустрічей;
- 10) сторінку для управління користувачами системи;
- 11) сторінку для управління командами.

При відвідуванні вебзастосунку для бронювання переговорних кімнат офісу користувач потрапляє на сторінку для авторизації. В залежності від того, чи даний користувач уже має обліковий запис у системі чи ні, він вирішує, яку процедуру йому потрібно виконати: реєстрацію чи авторизацію.

Якщо користувач має на меті зареєструватись у системі, йому потрібно натиснути на кнопку «Sign up», у результаті чого система відобразить форму для реєстрації з трьома полями для заповнення: логін, пароль та підтвердження паролю. Заповнивши усі поля та натиснувши на кнопку «Sign up», користувач направляється на головну сторінку системи за умови, що усі правила валідації виконано успішно. У разі некоректного заповнення полів форми реєстрації відвідувач системи побачить помилку і не буде допущеним до головної сторінки, поки не задовольнить вимоги.

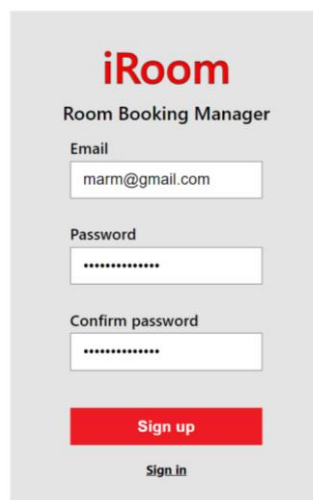
The image shows a registration form for 'iRoom Room Booking Manager'. At the top, the 'iRoom' logo is in red, followed by 'Room Booking Manager' in black. Below this are three input fields: 'Email' with the value 'marm@gmail.com', 'Password' with masked characters, and 'Confirm password' also with masked characters. At the bottom, there is a red button labeled 'Sign up' and a smaller link labeled 'Sign in'.

Рис. 4.2. Сторінка реєстрації

Якщо користувач при відвідуванні вебзастосунку уже має свій обліковий запис у системі, то проходити реєстрацію йому не потрібно. У такому разі користувач має заповнити поля форми авторизації: логін та пароль. У результаті заповнення полів та натиснення на кнопку «Sign in» користувач потрапить на головну сторінку системи або побачить помилку авторизації, якщо некоректно ввів логін чи пароль.

На головній сторінці вебзастосунку для бронювання переговорних кімнат офісу користувач системи має можливість переглядати завантаженість приміщень станом на день, тиждень і місяць. За допомогою календаря, що розташований у верхньому лівому куті сторінки, можна обирати день, тиждень або місяць, що буде відображатись у таблиці.

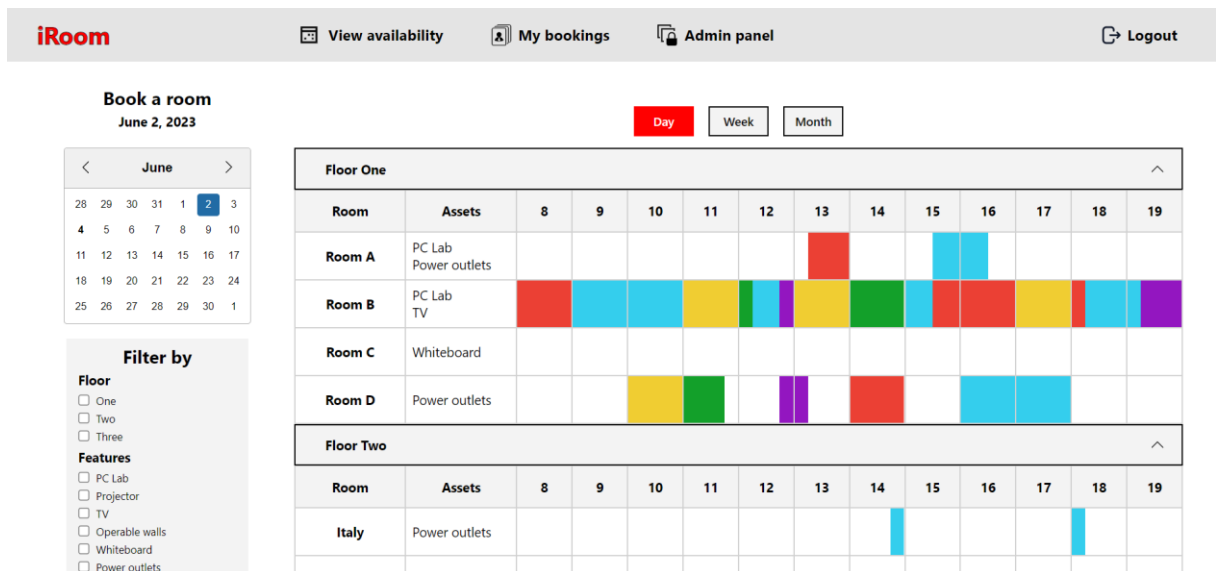


Рис. 4.3. Головна сторінка

Крім цього, на головній сторінці системи розміщено панель для встановлення значень фільтрів, за допомогою якої користувачі можуть обирати переговорні кімнати за певними критеріями, а саме: поверх, обладнання, місткість та доступність у певний період часу. Обравши деякі параметри фільтрації та натиснувши на кнопку «Submit», користувач матиме можливість бачити у таблиці завантаженості лише ті переговорні кімнати, що задовольняють обрані критерії. Якщо ж користувач натисне на кнопку «Reset» у полі фільтрації, система анулює усі параметри та відображатиме усі переговорні кімнати офісу.



Рис. 4.4. Результат фільтрації записів про переговорні кімнати

Таблиця завантаженості кімнат складається з декількох стовпчиків. У першому з них розміщені назви переговорних кімнат офісу. Другий стовпчик «Assets» відображає обладнання кожної з наявних кімнат. Кількість та наповнення наступних стовпчиків таблиці залежить від обраного вигляду: «Day», «Week» та «Month».

Якщо обраним відтворенням є «Day», то останні колонки таблиці містять інформацію про робочі години офісу. Кольори є певним ідентифікатором команди, що залучена на дане бронювання. Якщо клітинки таблиці білі, то це означає, що дана переговорна кімната не заброньована на певний період часу.

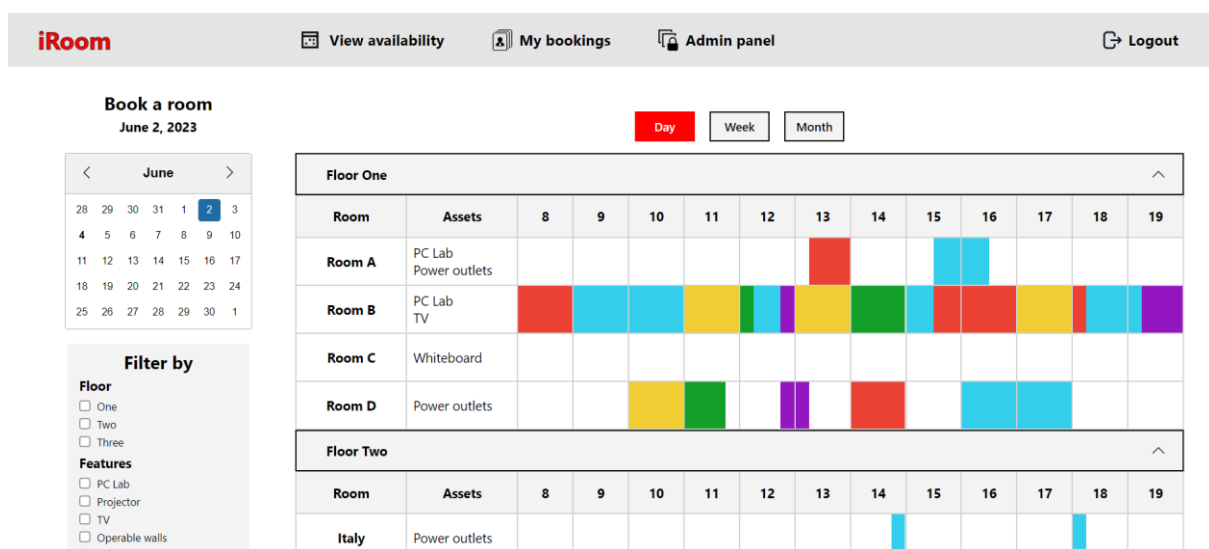


Рис. 4.5. Таблиця завантаженості станом на день

Якщо користувач обрав «Week» вигляд, то система відображатиме 7 колонок, що відповідатимуть за дні тижня. Оранжевий колір означає, що дана переговорна кімната заброньована частково на цей день. Червоним позначаються дні, у які кімната заброньована на всі робочі години офісу. Якщо клітинка біла, то приміщення повністю вільне у відповідний день.

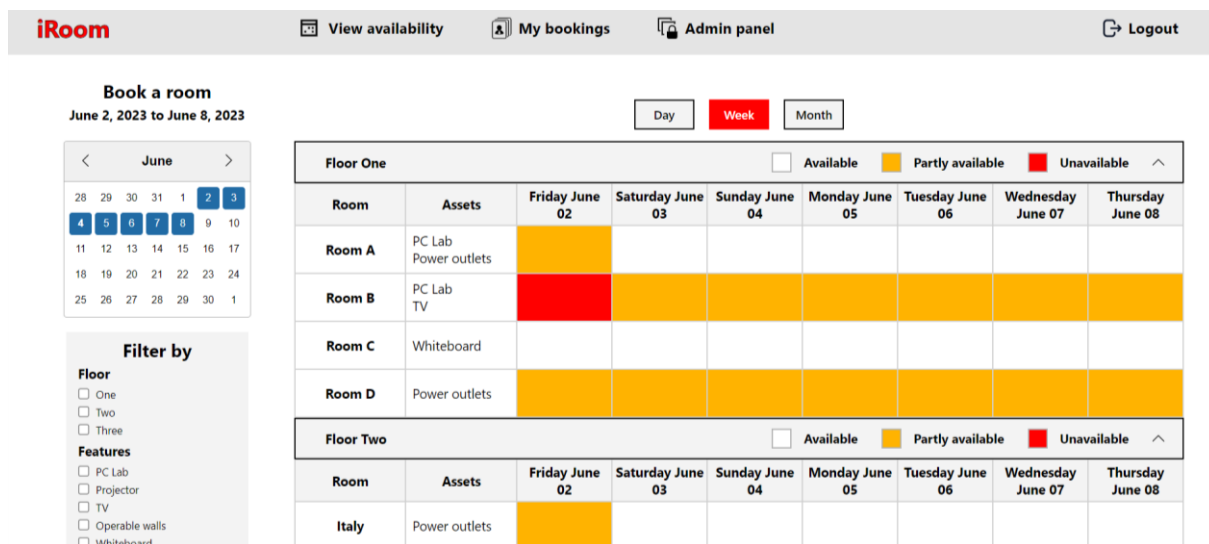


Рис. 4.6. Таблиця завантаженості станом на тиждень

Стовпчики таблиці завантаженості станом на «Month» відображатимуть тижні обраного місяця з поділом на дні.

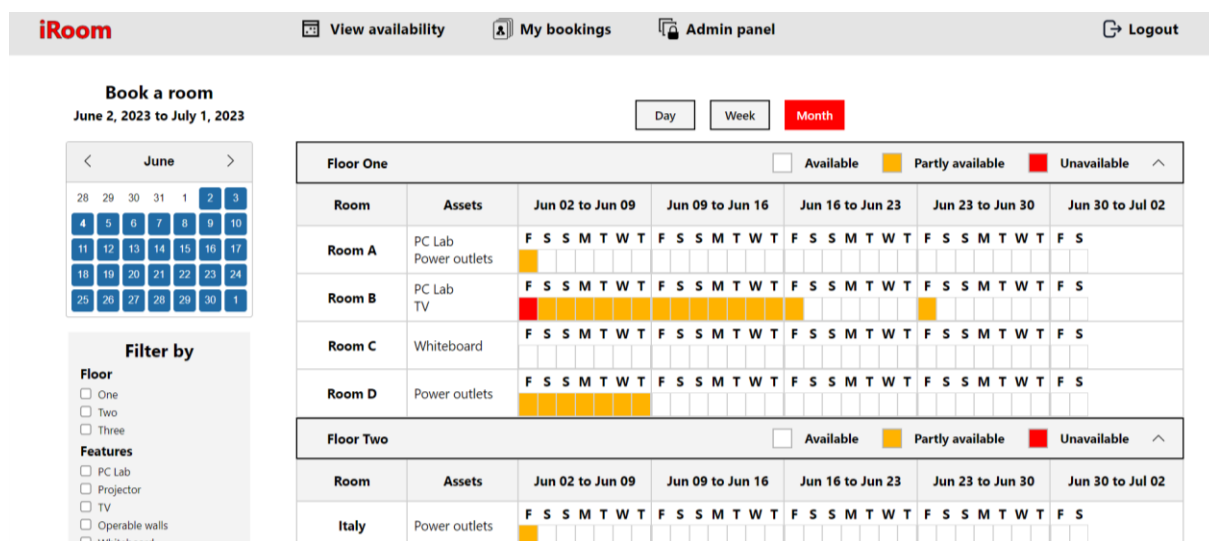


Рис. 4.7. Таблиця завантаженості станом на місяць

Коли користувач наводить курсор на білу клітинку таблиці завантаженості, система підсвічує найближчий 15-хвилинний період часу, коли дана кімната є вільною.

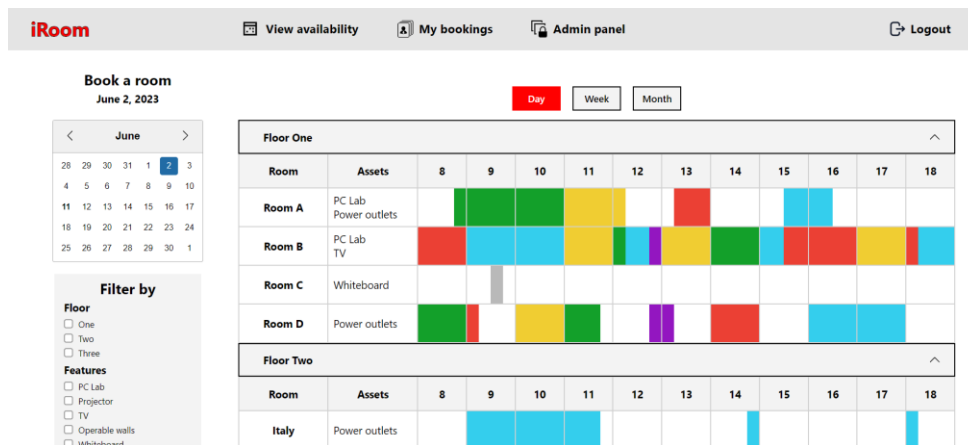


Рис. 4.8. Таблиця завантаженості з підсвіченням 15-хвилинним інтервалом

При натисненні на нього відвідувач сайту направляється на сторінку для створення бронювання, яка розміщує на собі інформацію про обрану кімнату, календар, таблицю завантаженості даної кімнати в обраний день та форму для створення зустрічі. Форма складається з декількох полів для заповнення: час початку та кінця бронювання, назва, команда, ціль, повторюваність та опис. Після того як користувач заповнив усі поля та натиснув на клавішу «Submit», система перевіряє поля на коректність заповнення, зберігає дані та направляє користувача на головну сторінку.

Floor One Room A		8	9	10	11	12	13	14	15	16	17	18	19
8													
9													
10													
11													
12													
13													
14													
15													
16													
17													
18													
19													

Make a booking	
Start Time	End Time
11:00	12:30
Title	
Sprint 80	
Business unit	
Octo	
Purpose	
Retrospective session	
Recurring	
Once a month	
Recurring end date	
08/11/2023	
Description	
Be ready to discuss our work of the previous sprint!	
Cancel	Submit

Рис. 4.9. Сторінка для створення бронювання

Коли користувач натискає на зафарбовану клітинку таблиці завантаженості кімнат, система перевіряє, чи є організатором відповідного бронювання даний відвідувач сайту. Якщо цей користувач є власником

зустрічі, система відкриває сторінку для редагування бронювання. Форма редагування подібна до форми створення бронювання, до усіх її полів дозволяється вносити зміни.

The screenshot shows the 'iRoom' application interface for editing a booking. At the top, there's a navigation bar with the 'iRoom' logo and links for 'View availability', 'My bookings', 'Admin panel', and 'Logout'. The main area is split into three parts. On the left, a calendar for 'June 2, 2023' shows the date selected. In the center, a table for '02 June, 2023' lists time slots from 8 to 18. Slots 9, 10, and 11 are highlighted in green, yellow, and red respectively. On the right, the 'Edit booking' form is displayed. It includes fields for 'Start Time' (11:00), 'End Time' (12:15), 'Title' (Sprint 90), 'Business unit' (Spin), 'Purpose' (Planning), 'Recurring' (No recurring), and a 'Description' field. 'RESET' and 'SUBMIT' buttons are at the bottom of the form.

Рис. 4.10. Сторінка для редагування власного бронювання

Якщо користувач хоче видалити власне бронювання, тоді йому необхідно натиснути на клавішу «Delete» з діалогового вікна.

Якщо організатором бронювання, на який натиснув користувач, є інша людина, тоді система відображає діалогове вікно з короткою інформацією про дану зустріч і клавішу для надсилання електронного листа власнику даного бронювання. При натисканні на клавішу «Contact» користувач направляється у поштовий агент, де вже створено шаблон повідомлення з заповненими полями адресата, адресанта і теми листа.

При натисненні на вкладку «My bookings» відвідувач сайту направляється на сторінку з інформацією про бронювання, що є створеними ним та тими, на які його запросили, у вигляді таблиці. Щоб переключатись між ними, користувачеві потрібно натискати на відповідну вкладку: «Created» або «Invited».

Created

Invited

Title Purpose	Start	End	Floor	Room	Actions
New tasks discussion Refinement session	June 14, 2023 08:30	June 14, 2023 09:30	Floor Two	Italy	
New tasks discussion Refinement session	June 21, 2023 08:30	June 21, 2023 09:30	Floor Two	Italy	
New tasks discussion Refinement session	June 28, 2023 08:30	June 28, 2023 09:30	Floor Two	Italy	
Stand up Daily meeting	June 12, 2023 09:30	June 12, 2023 10:00	Floor Two	Italy	
Stand up Daily meeting	June 13, 2023 09:30	June 13, 2023 10:00	Floor Two	Italy	
Stand up Daily meeting	June 14, 2023 09:30	June 14, 2023 10:00	Floor Two	Italy	
Backlog review Planning	June 14, 2023 16:00	June 14, 2023 16:30	Floor Two	Italy	
Backlog review Planning	June 21, 2023 16:00	June 21, 2023 16:30	Floor Two	Italy	

Рис. 4.11. Сторінка з власними бронюваннями користувача

Для користувачів системи, які мають права адміністратора, доступна додаткова вкладка, яка називається «Admin panel». При натисканні на неї, користувач потрапляє до панелі адміністратора, за допомогою якої може налаштовувати та управляти значеннями параметрів сутностей системи, серед яких: робочі години офісу, поверхи та переговорні кімнати, обладнання, цілі зустрічей, користувачі та команди.

Working hours

Floors + Rooms

Assets

Purposes

Users

Business units

Floor One

Go back

+ ROOM

Title	Capacity	Assets	Actions
Room A	20	PC Lab Power outlets	
Room B	20	PC Lab TV	
Room C	10	Whiteboard	
Room D	20	Power outlets	
Room E	20	Whiteboard	
Room F	10	TV Whiteboard	
Room G	20	Operable walls	
Room H	15	Power outlets	

Рис. 4.12. Панель адміністратора системи

4.3. Особливості проведеного тестування

Для проведення структурованої та детальної перевірки системи на відповідність поставленим перед нею вимогами, створено план тестування, що складається з тестових випадків. Для тестування вебзастосунку для бронювання переговорних кімнат офісу обрано димний метод, що полягає у проходженні усіх сценаріїв використання системи як її користувач.

Нижче наведено покрокову інструкцію проведеного тестування.

Таблиця 4.1

Тестові випадки

№	Назва	Передумова	Сценарій дій	Очікуваний результат
1	Можливість реєстрації у системі	Користувач не має облікового запису	1. Користувач натискає на клавішу «Sign up». 2. Користувач заповнює поля. 3. Користувач натискає на клавішу «Sign up».	Користувач успішно зареєстрований
2	Можливість авторизації у системі звичайним користувачем	Користувач зареєстрований у системі	1. Користувач натискає на клавішу «Sign in». 2. Користувач заповнює поля і натискає на клавішу «Sign in».	Користувач успішно авторизувався
3	Можливість доступу до панелі адміністратора	Користувач має логін та пароль адміністратора	1. Користувач натискає на клавішу «Sign in». 2. Користувач заповнює поля і натискає на клавішу «Sign in». 4. Користувач натискає на вкладку «Admin panel».	Користувач має можливість управляти сутностями і налаштовувати доступні дані у панелі адміністратора

4	Можливість перегляду завантаженості кімнат на день, тиждень і місяць	Користувач є авторизованим у системі	1. Користувач натискає на дату в календарі. 2. Користувач обирає «Week» вигляд таблиці. 3. Користувач виділяє дати іншого тижня на календарі. 4. Користувач обирає «Month» вигляд таблиці. 5. Користувач виділяє дати іншого місяця на календарі.	Система відображає таблицю завантаженості станом на обраний період
5	Можливість створення бронювання переговорної кімнати	Користувач є авторизованим у системі	1. Користувач обирає у календарі дату. 2. Користувач натискає на клітинку таблиці у рядку, який відповідає за певну кімнату. 3. Користувач обирає час початку бронювання. 4. Користувач заповнює усі необхідні поля і натискає на клавішу «Submit».	Створене бронювання відображається у таблиці завантаженості та у вкладці «My bookings»
6	Можливість фільтрації переговорних кімнат за заданими параметрами	Користувач є авторизованим у системі	1. Користувач обирає необхідні йому параметри з запропонованих. 2. Користувач натискає на клавішу «Filter».	Система відображає лише ті переговорні кімнати, які задовольняють параметри фільтрації

7	Можливість надсилання електронного листа організатору зустрічі	Користувач є авторизованим у системі	1. Користувач натискає на зафарбовану клітинку таблиці. 2. Користувач натискає на клавішу «Contact».	Користувач направляється до поштового агента з відкритим шаблоном повідомлення
8	Можливість перегляду інформації про обране бронювання	Користувач є авторизованим у системі	1. Користувач натискає на зафарбовану клітинку таблиці.	Система відображає детальну інформацію про бронювання
9	Можливість редагування власного бронювання	Користувач має власні бронювання	1. Користувач натискає на зафарбовану клітинку таблиці. 2. Користувач натискає на клавішу «Edit».	Система зберігає усі внесені зміни до бронювання
10	Можливість видалення власного бронювання	Користувач має власні бронювання	1. Користувач натискає на зафарбовану клітинку таблиці. 2. Користувач натискає на клавішу «Delete».	Система видаляє бронювання з таблиці завантаженості та вкладки «My bookings»
11	Можливість перегляду списку власних бронювань	Користувач має власні бронювання	1. Користувач натискає на вкладку «My bookings».	Система відображає таблицю з усіма бронюваннями користувача

4.4. Рекомендації щодо подальшого вдосконалення

У результаті аналізу створеного вебзастосунку для бронювання переговорних кімнат офісу визначено декілька місць для подальшого вдосконалення системи.

Насамперед важливою функцією є інтеграція календаря користувача застосунку «iRoom» з іншими його календарями, наприклад, Google Calendar або Microsoft Outlook. Це допомогло б користувачам зберігати усі свої заплановані бронювання в одному місці і уникати конфліктів онлайн-та офлайн-зустрічей.

Другим покращенням системи є розширення можливостей створення зустрічей не лише для команди, але й для окремих відвідувачів офісу. У такому разі користувачі вебзастосунку зможуть бронювати кімнату для зустрічей формату один-на-один або з окремою групою людей, що належать іншим командам.

Крім того, покращенням для вебзастосунку є надсилання нагадувань про заплановану нараду за 15 хвилин до її початку, щоб відвідувачі офісу мали час на те, щоб дістатись потрібної переговорної кімнати та підготуватись до зустрічі.

4.5. Висновки до розділу 4

У рамках даного розділу надано особливості реалізації клієнтської та серверної частин вебзастосунку, описано створений користувацький інтерфейс, складено таблицю з тестовими випадками, за допомогою яких виконувалось димне тестування системи, та надано рекомендації для подальшого розвитку вебзастосунку.

ВИСНОВКИ

Метою даного дипломного проєкту було допомогти компаніям, що мають офісні приміщення, автоматизувати процес відстеження бронювань на ту чи іншу переговорну кімнату. Результатом роботи став програмний продукт, реалізований у вигляді вебзастосунку, доступного до використання у мережі Інтернет.

У процесі виконання дипломного проєкту проаналізовано предметну галузь програмних забезпечень для бронювання переговорних кімнат, виділено ряд проблем, що зустрічаються серед відвідувачів офісів, досліджено існуючі програмні рішення, наведено переваги та недоліки кожного з них, зібрано вимоги до системи та обрано інструменти для її реалізації. Порівнявши існуючі аналоги, виявлено, що жоден із сервісів не вирішує усі проблеми користувачів та не задовольняє їхні потреби. Тому розроблення вебзастосунку для бронювання переговорних кімнат офісу є актуальним. У результаті аналізу інструментів для розроблення програмного продукту обрано мову програмування JavaScript, бібліотеку React.js та СКБД PostgreSQL. Вебзастосунок розроблено за принципами клієнт-серверної архітектури з використанням патерну проєктування Model-View-Controller. У дипломному проєкті реалізовано користувацький інтерфейс, алгоритм перевірки кімнати на зайнятість у певний період часу, спроектовано базу даних.

Розроблення виконано у повному обсязі, воно відповідає висунутим до нього вимогам, що описані у даному документі та технічному завданні, тестування продукту здійснено у відповідності до затвердженої програми та методики тестування.

Використання вебзастосунку для бронювання переговорних кімнат допоможе відвідувачам офісу обирати та бронювати на визначений період часу переговорну кімнату, яка задовольняє їхні критерії, а менеджерам офісу – управляти даними про конференц-зали та користувачів системи.

СПИСОК ВИКОРИСТАНИХ ЛІТЕРАТУРНИХ ДЖЕРЕЛ

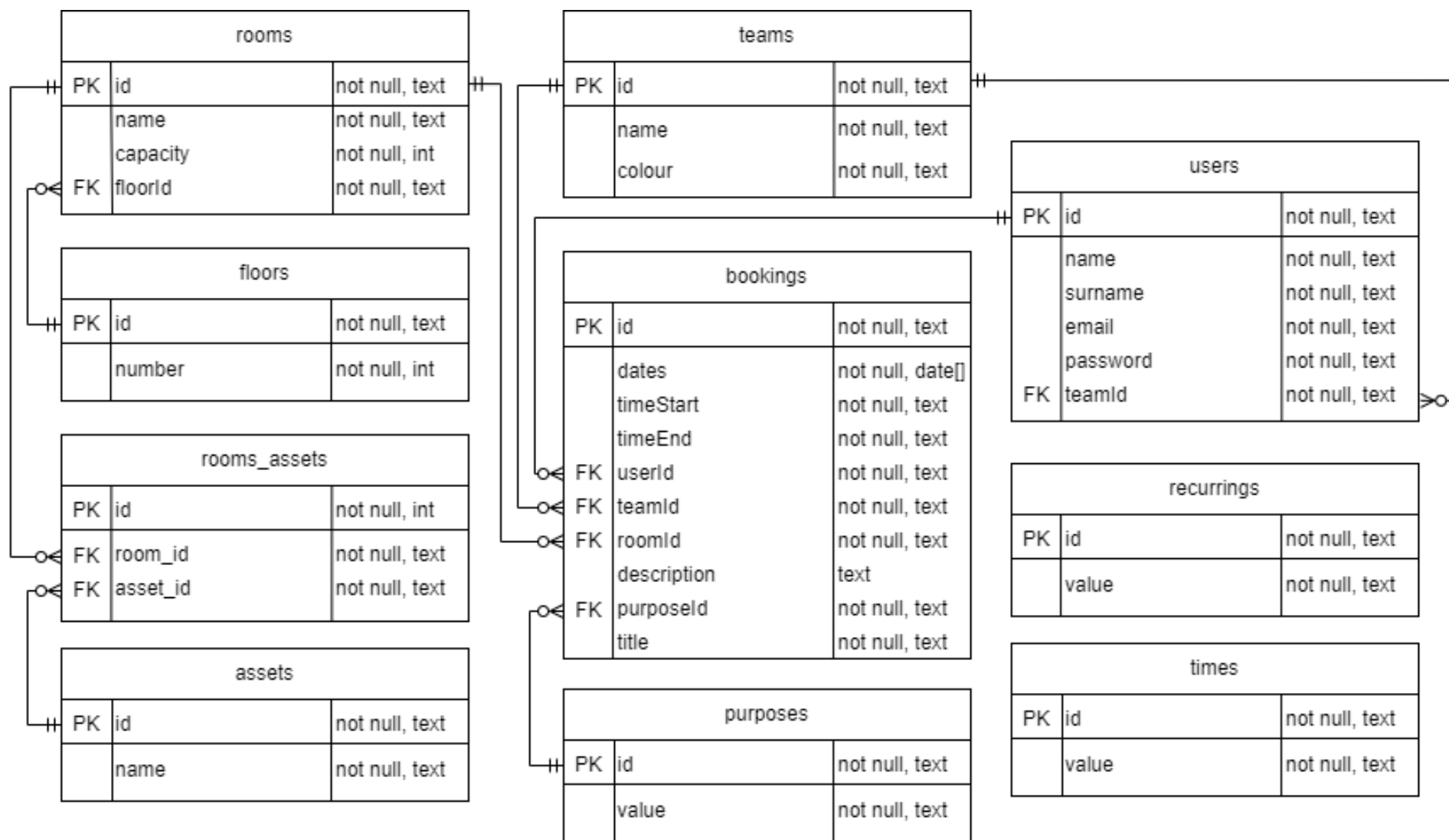
1. Система для бронювання переговорних кімнат Evoko Liso [Електронний ресурс]. — Режим доступу: <https://avexpert.com.ua/uk/product/sistema-dlya-bronirovaniya-peregovornyh-komnat-evoko-liso/> — (28.03.2023).
2. Microsoft Excel [Електронний ресурс]. — Режим доступу: https://uk.wikipedia.org/wiki/Microsoft_Excel — (30.03.2023).
3. Google Calendar [Електронний ресурс]. — Режим доступу: https://uk.wikipedia.org/wiki/Google_Calendar — (31.03.2023).
4. Річний пакет Google Workspace Business Starter [Електронний ресурс]. — Режим доступу: <https://cloudfresh.com/ua/product/g-suite-basic-annual/> — (31.03.2023).
5. What is Python? Executive Summary Starter [Електронний ресурс]. — Режим доступу: <https://www.python.org/doc/essays/blurb/> — (20.04.2023).
6. The Pros and Cons of Python Programming [Електронний ресурс]. — Режим доступу: <https://www.linode.com/docs/guides/pros-and-cons-of-python/> — (20.04.2023).
7. Основи програмування на Java. Мова програмування Java. [Електронний ресурс]. — Режим доступу: <https://promoter.net.ua/articles/osnovi-programuvannya-na-java.html> — (20.04.2023).
8. Java — одна з найпопулярніших мов програмування [Електронний ресурс]. — Режим доступу: <https://np.pl.ua/2021/03/perevahy-movy-programuvannia-java/> — (20.04.2023).
9. The Good and the Bad of Java Programming [Електронний ресурс]. — Режим доступу: <https://www.altexsoft.com/blog/engineering/pros-and-cons-of-java-programming/> — (20.04.2023).

10. JavaScript For Backend Development: An Introduction [Електронний ресурс]. — Режим доступу: <https://blog.hubspot.com/website/javascript-backend> — (23.04.2023).
11. How to Use JavaScript for Backend Development in 2023 [Електронний ресурс]. — Режим доступу: <https://www.turing.com/kb/javascript-for-backend-development#what-is-backend-development?> — (23.04.2023).
12. The difference between Virtual DOM and DOM [Електронний ресурс]. — Режим доступу: <https://reactkungfu.com/2015/10/the-difference-between-virtual-dom-and-dom/> — (23.04.2023).
13. Ahead-of-time (AOT) compilation [Електронний ресурс]. — Режим доступу: <https://angular.io/guide/aot-compiler> — (25.04.2023).
14. Система управління базами даних [Електронний ресурс]. — Режим доступу:
https://uk.wikipedia.org/wiki/Система_управління_базами_даних — (28.04.2023).
15. Documentation PostgreSQL [Електронний ресурс]. — Режим доступу: <https://www.postgresql.org/docs/> — (29.04.2023).
16. ACID [Електронний ресурс]. — Режим доступу: <https://uk.wikipedia.org/wiki/ACID> — (29.04.2023).
17. СУБД: які бувають, як вибрати [Електронний ресурс]. — Режим доступу: <https://highload.today/uk/subd-yaki-buvayut-yak-vibrati/> — (29.04.2023).
18. MongoDB: The Developer Data Platform [Електронний ресурс]. — Режим доступу: <https://www.mongodb.com/> — (29.04.2023).
19. MySQL [Електронний ресурс]. — Режим доступу: <https://www.mysql.com/> — (29.04.2023).
20. SMART [Електронний ресурс]. — Режим доступу: <https://uk.wikipedia.org/wiki/SMART> — (30.04.2023).
21. Client-Server Architecture. Advantages and Disadvantages of the Network Computing Model [Електронний ресурс]. — Режим доступу:

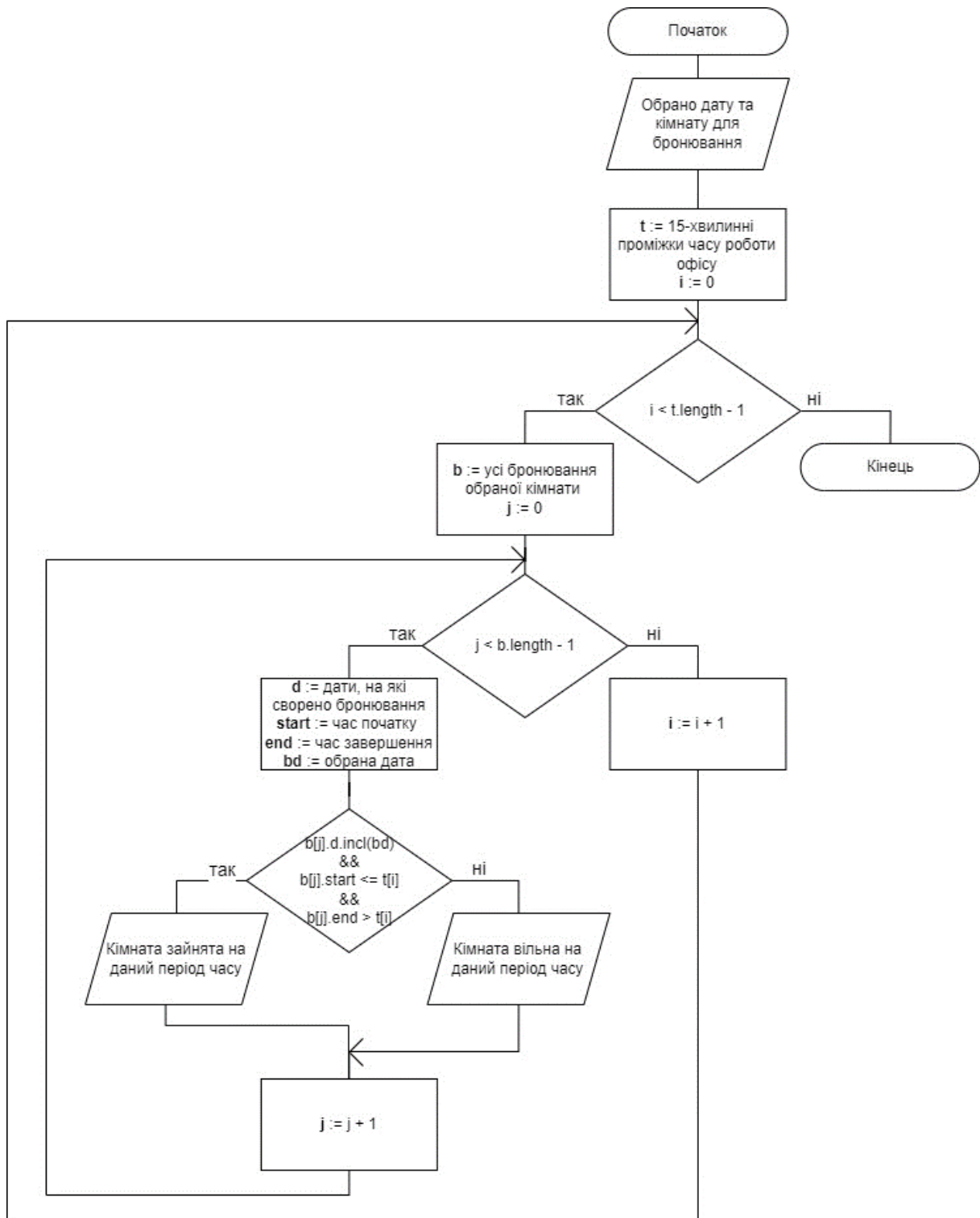
- <https://kitrum.com/blog/client-server-architecture-advantages-and-disadvantages/> — (23.04.2023).
22. Redux Toolkit [Электронный ресурс]. — Режим доступа: <https://redux-toolkit.js.org/> — (27.04.2023).
23. Moment.js [Электронный ресурс]. — Режим доступа: <https://momentjs.com/> — (27.04.2023).
24. File-loader [Электронный ресурс]. — Режим доступа: <https://v4.webpack.js.org/loaders/file-loader/> — (27.04.2023).
25. Nodemailer [Электронный ресурс]. — Режим доступа: <https://nodemailer.com/about/> — (27.04.2023).

ДОДАТКИ

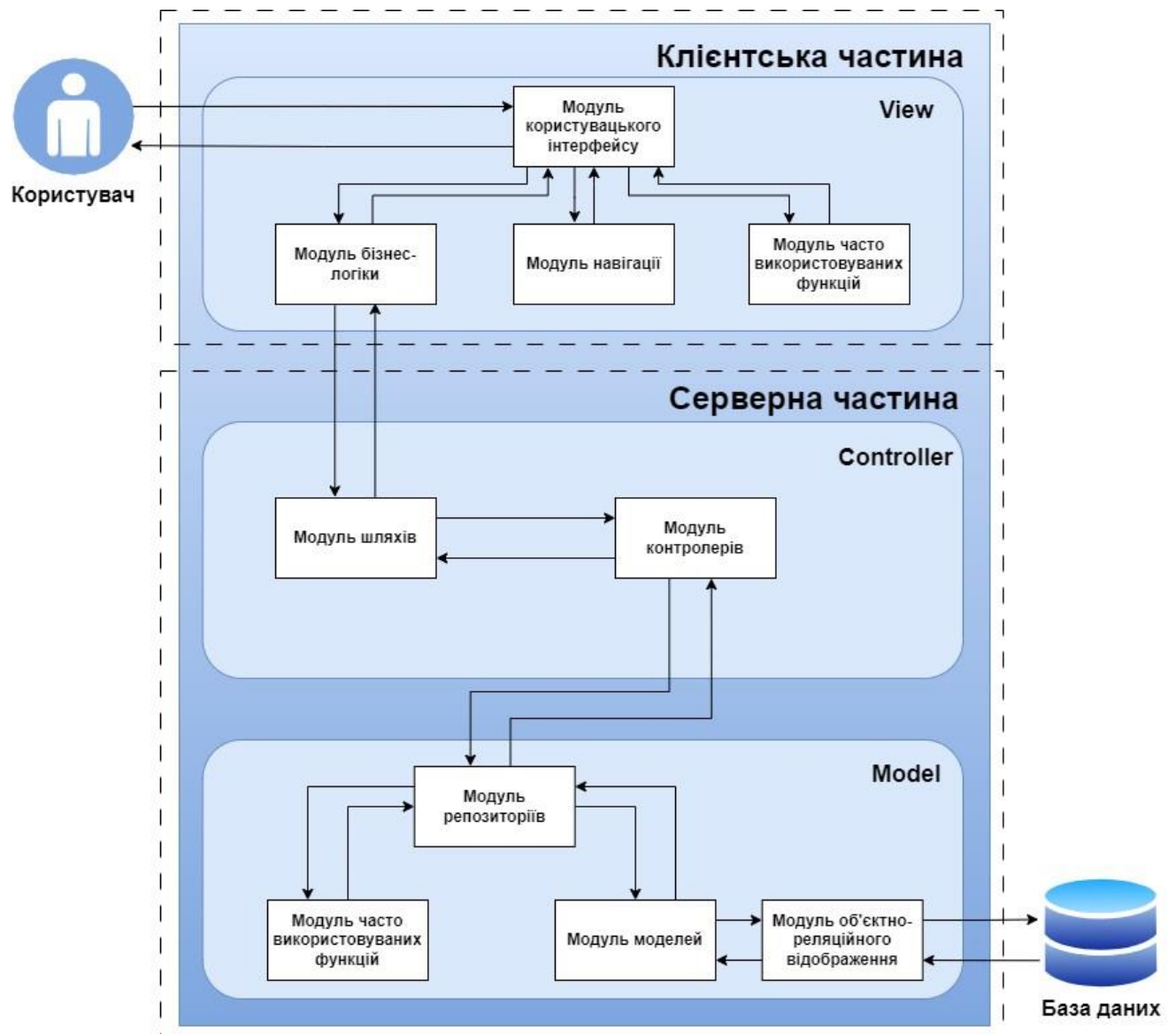
Додаток 1
Копії графічних матеріалів



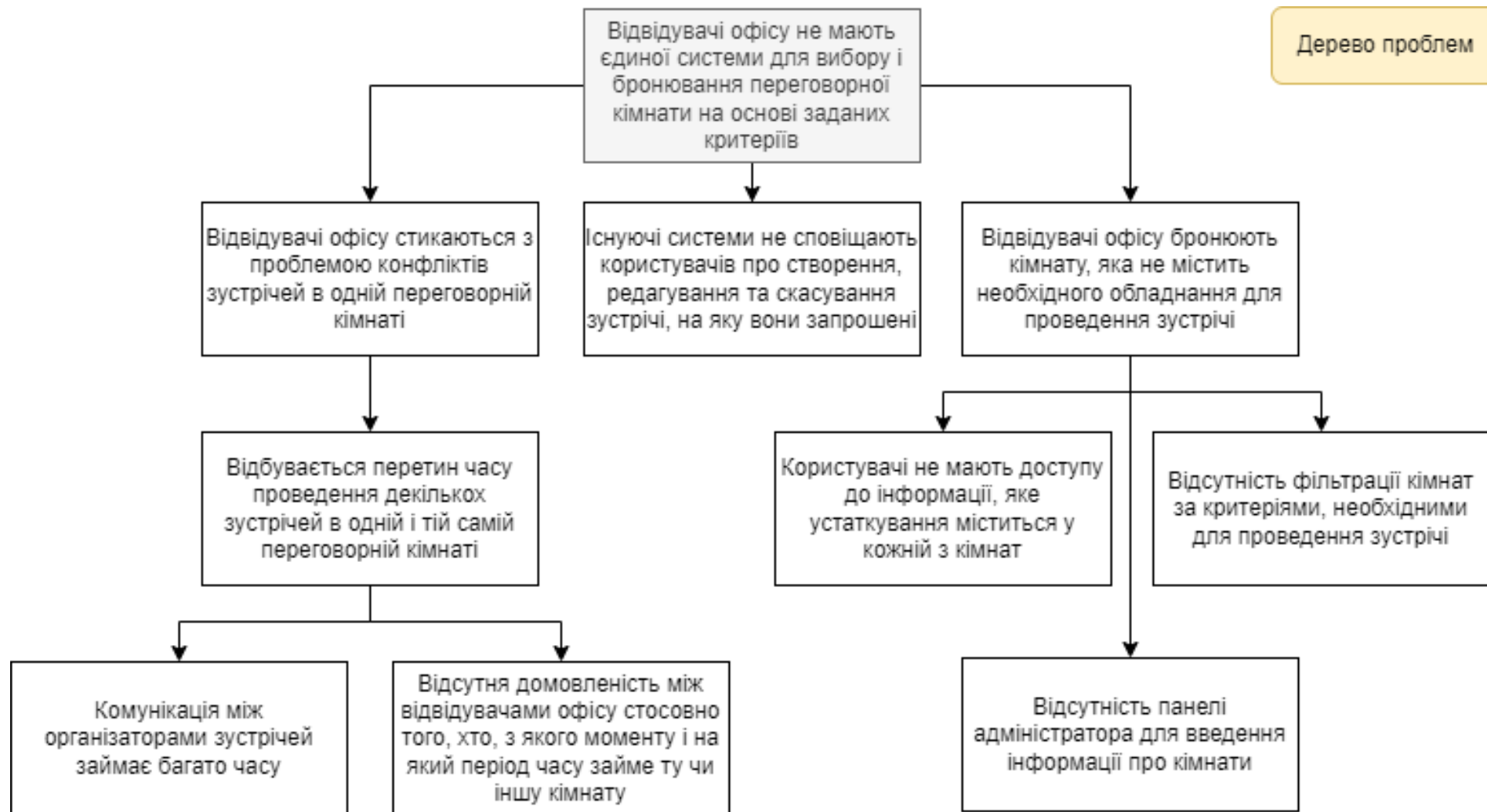
ДП.045440-06-99. Вебзастосунок для бронювання переговорних кімнат офісу. Структура бази даних. ERD-діаграма



ДП.045440-07-99. Вебзастосунок для бронювання переговорних кімнат офісу. Алгоритм перевірки кімнати на зайнятість. Блок-схема алгоритму



Дерево проблем



Дерево проблем потенційних користувачів системи
Музичук М.А., група КП-92

Додаток 2
Лістинг програми

```

export const checkTimeAndDateForBooking = (time, date, bookings) => {
  const formattedCurrentDate = date.format('YYYY-MM-DD')
  const formattedTime = moment(`${formattedCurrentDate}T${time}`)
  let _booking = null
  bookings.forEach(booking => {
    booking.dates.forEach(date => {
      const dateStart = moment(`${date}T${booking.timeStart}`)
      const dateEnd = moment(`${date}T${booking.timeEnd}`)
      if(dateStart <= formattedTime && formattedTime < dateEnd){
        _booking = booking
        return _booking
      }
    })
  })
  return _booking
}

export const checkDayForBusiness = (day, bookingsInCurrentRoom, times) => {
  let availableTimesNumber = 0
  times.forEach(time => {
    minutes.forEach(subTime => {
      const formattedTime = `${time.title}:${subTime}`
      const booking = checkTimeAndDateForBooking(formattedTime,
day.date, bookingsInCurrentRoom)
      if(!booking){
        availableTimesNumber++
      }
    })
  })
  return availableTimesNumber
}

export const getColorByDayAvailability = (availableTimes, times) => {
  const maxTimes = times.length * minutes.length
  if(availableTimes === maxTimes)
    return 'transparent'
  if(availableTimes === 0)
    return '#ff0000'
  return '#ffb300'
}

export const getAvailabilityOfRooms = (rooms, bookings, period, day, times)
=> {
  const maxTimesPerDay = times.length * minutes.length
  const partly = []
  const booked = []
  const available = []
  rooms.forEach(room => {
    if(period.id === 0){
      const bookingsInCurrentRoom = bookings.filter(booking =>
room.bookingsIds.includes(booking.id))
      const availableTimesNumber = checkDayForBusiness({ date: day },
bookingsInCurrentRoom, times)
      if(availableTimesNumber === maxTimesPerDay)
        available.push(room)
      else if(availableTimesNumber === 0)
        booked.push(room)
      else
        partly.push(room)
    }
  })
}

```

```

    }
    else if(period.id === 1){
      let availableTimesNumber = 0
      for(let i = 0; i < 7; i++){
        const currentDate = moment(day).add(i, 'days')
        const bookingsInCurrentRoom = bookings.filter(booking =>
room.bookingsIds.includes(booking.id))
        availableTimesNumber += checkDayForBusiness({ date:
currentDate }, bookingsInCurrentRoom, times)
      }
      if(availableTimesNumber === maxTimesPerDay * 7)
        available.push(room)
      else if(availableTimesNumber === 0)
        booked.push(room)
      else
        partly.push(room)
    }
    else{
      let availableTimesNumber = 0
      for(let i = 0; i < moment(day).daysInMonth(); i++){
        const currentDate = moment(day).add(i, 'days')
        const bookingsInCurrentRoom = bookings.filter(booking =>
room.bookingsIds.includes(booking.id))
        availableTimesNumber += checkDayForBusiness({ date:
currentDate }, bookingsInCurrentRoom, times)
      }
      if(availableTimesNumber === maxTimesPerDay *
moment(day).daysInMonth())
        available.push(room)
      else if(availableTimesNumber === 0)
        booked.push(room)
      else
        partly.push(room)
    }
  })
  const result = {
    partly,
    booked,
    available
  }
  return result
}

const processTime = (time) => {
  return time?.length === 5 ? time : 0+time
}

export const getAllAvailableStartTimes = (times, bookings, currentDate,
currentBooking) => {
  const prefixDate = currentDate.format('YYYY-MM-DDT')
  const hoursWithMinutes = []
  times.forEach(hour => {
    minutes.forEach(minute => {
      hoursWithMinutes.push(`${hour}:${minute}`)
    })
  })
  const availableStartTimes = []
  hoursWithMinutes.forEach(time => {
    const processedTime = processTime(time)

```

```

        let isTimeAvailable = true
        const processedDate = `${prefixDate}${processedTime}`
        bookings.filter(booking => booking.id !==
currentBooking?.id).forEach(booking => {
            const dateStart = moment(`${prefixDate}${booking.timeStart}`)
            const dateEnd = moment(`${prefixDate}${booking.timeEnd}`)
            if(dateStart <= moment(processedDate) && moment(processedDate)
< dateEnd)
                isTimeAvailable = false
            });
            if(isTimeAvailable)
                availableStartTimes.push(time)
        })
        return availableStartTimes
    }

export const getAllAvailableEndTimes = (times, bookings, currentDate,
selectedStartTime, currentBooking) => {
    const hoursWithMinutes = []
    times.forEach(hour => {
        minutes.forEach(minute => {
            hoursWithMinutes.push(`${hour}:${minute}`)
        })
    })
    const filteredBookings = bookings.filter(booking =>
booking.dates.includes(currentDate.format('YYYY-MM-DD')))
    const prefixDate = currentDate.format('YYYY-MM-DDT')
    const endTimes = []
    const lastHour = parseInt(hoursWithMinutes[hoursWithMinutes.length -
1]?.split(':')[0])
    hoursWithMinutes.push(`${lastHour + 1}:00`)
    hoursWithMinutes.forEach((time, id) => {
        const processedTime = processTime(time)
        const processedStartTime = processTime(selectedStartTime)
        const processedDate = `${prefixDate}${processedTime}`
        const processedStartDate = `${prefixDate}${processedStartTime}`
        if(moment(processedDate) > moment(processedStartDate)){
            const isTimeAvailable = filteredBookings.filter(booking => {
                const dateStart =
moment(`${prefixDate}${booking.timeStart}`)
                return dateStart > moment(processedStartDate) && booking.id
!== currentBooking?.id
            }).every(booking => {
                const dateStart =
moment(`${prefixDate}${booking.timeStart}`)
                return moment(processedDate) <= dateStart &&
moment(processedDate) > moment(processedStartDate)
            })
            if(isTimeAvailable)
                endTimes.push(time)
            else
                return endTimes
        }
    })
    return endTimes
}

const Room = ({ room, floor }) => {

```

```

const assets = useSelector(store => store.assetsSlice.assets)
const times = useSelector(store => store.timesSlice.times)
const bookings = useSelector(store => store.bookingsSlice.bookings)
const currentPeriod = useSelector(store => store.periodSlice.period)
const [isModalVisible, setIsModalVisible] = useState(false)
const [isWarningVisible, setIsWarningVisible] = useState(false)
const currentDate = useSelector(store => store.dateSlice.date)
const [monthTimeAvailability, setMonthTimeAvailability] = useState([])
const [weekTimeAvailability, setWeekTimeAvailability] = useState([])
const [hoveredMeeting, setHoveredMeeting] = useState('')

const weekDays = getNextWeekDays(currentDate)
const monthDays = getNextMonthWeeks(currentDate)

const isDayPeriod = currentPeriod.id === 0
const isWeekPeriod = currentPeriod.id === 1

const timesUnits = isDayPeriod ? times : isWeekPeriod ? weekDays :
monthDays

const getAssetsByRoom = (room) => {
  const assetsIds = room.assetsIds
  return assets.filter(asset => assetsIds.includes(asset.id))
}

const getBookingsByRoom = (room) => {
  const bookingsIds = room.bookingsIds
  return bookings.filter(booking =>
bookingsIds?.includes(booking.id))
}

const handleClosePopup = () => {
  setIsModalVisible(false)
}

const handleOpenPopup = () => {
  setIsModalVisible(true)
}

const assetsInCurrentRoom = getAssetsByRoom(room)
const bookingsInCurrentRoom = getBookingsByRoom(room)

const getTimesAvailability = () => {
  if(currentPeriod.id){
    const roomData = {
      roomId: room.id,
      days: (currentPeriod.id === 2 ? timesUnits.map(timeUnit =>
timeUnit.days.map(day => day.date.format('YYYY-MM-DD')) :
timesUnits.map(timeUnit => timeUnit.date.format('YYYY-MM-
DD'))).flat(Infinity)
    }
    return
  }
  fetch('http://localhost:3000/booking/getAvailableTimesInRoom', {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(roomData)
  })
}

```

```

        }).then(data => data.json())
    }
}

const getMonthTimesAvailability = () => {
    getTimesAvailability().then(data => {
        setMonthTimeAvailability(timesUnits.map(timeUnit => ({
            ...timeUnit,
            days: timeUnit.days.map(day => ({
                ...day,
                timeAvailability: data[moment(day.date).format('YYYY-MM-DD')]
            })))
    })))
}

const getWeekTimesAvailability = () => {
    getTimesAvailability().then(data => {
        setWeekTimeAvailability(timesUnits.map(timeUnit => ({
            ...timeUnit,
            timeAvailability: data[moment(timeUnit.date).format('YYYY-MM-DD')]
        })))
    })
}

useEffect(() => {
    if(currentPeriod.id === 1)
        getWeekTimesAvailability()
    else if(currentPeriod.id === 2)
        getMonthTimesAvailability()
}, [bookings])

useEffect(() => {
    if(currentPeriod.id === 1 && !weekTimeAvailability.length)
        getWeekTimesAvailability()
    else if(currentPeriod.id === 2 && !monthTimeAvailability.length)
        getMonthTimesAvailability()
}, [currentPeriod])

return (
    <div className={styles.roomWrapper}>
        <div className={styles.infoWrapper}>
            <div className={styles.infoCol}>{room.name}</div>
            <div className={styles.infoCol}>
                <div className={styles.assetsWrapper}>
                    {
                        assetsInCurrentRoom.map(asset => {
                            return (
                                <p
                                    className={styles.asset}>{asset.name}</p>
                                )
                            })
                    }
                </div>
            </div>
        </div>
    </div>
)

```



```

<div className={styles.times}>
  {
    (currentPeriod.id === 2 ?
      (monthTimeAvailability.length ?
monthTimeAvailability : timesUnits) :
      currentPeriod.id === 1 ?
      (weekTimeAvailability.length ?
weekTimeAvailability : timesUnits) : timesUnits).map(timesUnit => {
      return isDayPeriod ? (
        <DayTimeUnit
          time={timesUnit}
          setHoveredMeeting={setHoveredMeeting}
          hoveredMeeting={hoveredMeeting}

bookingsInCurrentRoom={bookingsInCurrentRoom}
          room={room}
          floor={floor}
          handleOpenPopup={handleOpenPopup}
        />
      ) : isWeekPeriod ? (
        <WeekTimeUnit time={timesUnit}
bookingsInCurrentRoom={bookingsInCurrentRoom} room={room} floor={floor}/>
      ) : <MonthTimeUnit time={timesUnit}
bookingsInCurrentRoom={bookingsInCurrentRoom} room={room} floor={floor} />
    ))
  }
  {isModalVisible && <MeetingInfoPopup
handleClosePopup={handleClosePopup}/>}
</div>
</div>
)
}

```

```

const DayTimeUnit = ({ time, bookingsInCurrentRoom, room, floor,
setHoveredMeeting, hoveredMeeting, handleOpenPopup }) => {
  const navigate = useNavigate()
  const dispatch = useDispatch()

  const teams = useSelector(store => store.teamsSlice.teams)
  const currentDate = useSelector(store => store.dateSlice.date)
  const currentUser = useSelector(store => store.usersSlice.currentUser)

  const getTeamByBooking = (booking) => {
    const teamId = booking.teamId
    return teams.find(team => team.id === teamId)
  }

  const navigateToBookingCreate = (time) => {
    navigate('/createBooking', { state: { bookingsInCurrentRoom, room,
floor, time } })
  }

  const onExistingBookingClick = (bookingNow) => {
    console.log(bookingNow, currentUser)
    dispatch(setBookingContent({
      ...bookingNow,
      userClicked: currentUser,

```

```

        bookingsInCurrentRoom,
        room,
        floor,
        time
    )))
    handleOpenPopup()
}

const onMeetingPartHover = (id) => {
    setHoveredMeeting(id)
}

return (
    <div className={styles.subTimesWrapper} >
        {
            minutes.map(minute => {
                const formattedTime = `${time.title}:${minute}`
                const bookingNow = checkTimeForBooking(formattedTime,
bookingsInCurrentRoom, currentDate)
                let bookedTeam = null
                if(bookingNow)
                    bookedTeam = getTeamByBooking(bookingNow)
                return (
                    <div
                        className={bookingNow ? styles.subtimeBooked :
styles.subtime}
                        style={{backgroundColor: bookingNow ?
bookingNow.id === hoveredMeeting ? 'rgb(185, 185, 185)' : bookedTeam?.color
: 'transparent'}}
                        onClick={() => {
                            if(!bookingNow)
                                navigateToBookingCreate(`${time.title}:${minute}`)
                            else{
                                onExistingBookingClick(bookingNow)
                            }
                        }}
                        onMouseOver={() =>
onMeetingPartHover(bookingNow?.id)}
                    >

                        </div>

                    )
                })
        }
    </div>
)
}

```

```

const TimeManager = ({ title }) => {
    const dispatch = useDispatch()
    const currentDate = useSelector(store => store.dateSlice.date)
    const currentPeriod = useSelector(store => store.periodSlice.period)

    const [startDate, setStartDate] = useState(currentDate.toDate());
    const [endDate, setEndDate] = useState(currentDate.toDate());

    const formattedDate = currentPeriod.id === 0 ? currentDate.format('MMMM

```

```

D, YYYY')
    : currentPeriod.id === 1 ? `${moment(currentDate).format('MMMM D,
YYYY')} to ${moment(currentDate).add(6, 'days').format('MMMM D, YYYY')}`
    : `${moment(currentDate).format('MMMM D, YYYY')} to
${moment(currentDate).add(moment(startDate).daysInMonth() - 1,
'days').format('MMMM D, YYYY')}`

    const onDateChange = (date) => {
      dispatch(editDate({
        date: moment(date)
      }))
      setStartDate(moment(date).toDate())
    }

    useEffect(() => {
      if(currentPeriod.id === 0)
        setEndDate(startDate)
      else if(currentPeriod.id === 1)
        setEndDate(moment(startDate).add(6, 'days').toDate())
      else if(currentPeriod.id === 2)

setEndDate(moment(startDate).add(moment(startDate).daysInMonth() - 1,
'days').toDate())
      }, [currentPeriod, startDate])

    return (
      <div className={styles.timeWrapper}>
        <div className={styles.titleWrapper}>
          <p className={styles.title}>{title}</p>
          <p className={styles.date}>{formattedDate}</p>
        </div>
        <div className={styles.datePickerWrapper}>
          <DatePicker
            selected={startDate}
            onChange={onDateChange}
            inline
            startDate={startDate}
            endDate={endDate}
            calendarClassName={styles.calendar}
            dayClassName={() => styles.calendarDay}
            renderCustomHeader={() => (
              date,
              decreaseMonth,
              increaseMonth,
              prevMonthButtonDisabled,
              nextMonthButtonDisabled,
            ) => (
              <div className={styles.headerWrapper}>
                <button onClick={decreaseMonth}
disabled={prevMonthButtonDisabled} className={styles.navButton}>
                  <img src={Arrow} alt="arrow"
className={styles.arrowLeft} />
                </button>

                <div className={styles.monthWrapper}>
                  {moment(date).format('MMMM')}
                </div>
              </div>
            )}
          />
        </div>
      </div>
    )
  )
}

```

```

                <button onClick={increaseMonth}
disabled={nextMonthButtonDisabled} className={styles.navButton}>
                    <img src={Arrow} alt="arrow"
className={styles.arrowRight} />
                </button>
            </div>
        )}
    </div>
</div>
)
}

```

```

const MyBookingsList = ({isCreatedShown}) => {

    const bookings = useSelector(store => store.bookingsSlice.bookings)
    const user = useSelector(store => store.usersSlice.currentUser)
    const teams = useSelector(store => store.teamsSlice.teams)
    const usersTeam = teams.find(team => team.id === user.teamId)
    const purposes = useSelector(store => store.purposesSlice.purposes)
    const floors = useSelector(store => store.floorsSlice.floors)
    const rooms = useSelector(store => store.roomsSlice.rooms)
    const userBookings = isCreatedShown ? bookings.filter(booking =>
booking.userId === user.id) : bookings.filter(booking => booking.teamId ===
usersTeam.id)
    const [isWarningVisible, setIsWarningVisible] = useState(false)
    const [currentBooking, setCurrentBooking] = useState({})
    const [dateToDelete, setDateToDelete] = useState({})
    const navigate = useNavigate()
    const dispatch = useDispatch()

    const getPurposeValueById = (id) => {
        return purposes.find(purpose => purpose.id === id).value
    }

    const getRoomNumberById = (id) => {
        return rooms.find(room => room.id === id)
    }

    const getFloorNumberById = (id) => {
        const room = rooms.find(room => room.id === id)
        return floors.find(floor => floor.id === room.floorId)
    }

    const getBookingsByIds = (bookingsIds) => {
        return bookingsIds.map(id => bookings.find(booking => booking.id
=== id))
    }

    const handleCloseWarning = () => {
        setIsWarningVisible(false)
    }

    const handleOpenWarning = (booking, bookingsInCurrentRoom, room, floor,
date) => {
        dispatch(setBookingContent({
            ...booking,

```

```

        userClicked: user,
        bookingsInCurrentRoom,
        room,
        floor,
    )))
    setIsWarningVisible(true)
    setCurrentBooking(booking)
    setDateToDelete(date)
}

const handleBookingDelete = () => {
    const bookingWithoutCurrentDate = {
        id: currentBooking.id,
        dates: currentBooking.dates.filter(date => date !==
dateToDelete),
        timeStart: currentBooking.timeStart,
        timeEnd: currentBooking.timeEnd,
        userId: currentBooking.userId,
        teamId: currentBooking.teamId,
        roomId: currentBooking.roomId,
        description: currentBooking.description,
        purposeId: currentBooking.purposeId,
        title: currentBooking.title,
    }
    dispatch(deleteBooking({
        updatedBooking: bookingWithoutCurrentDate
    })))
}

const navigateToBookingEdit = (bookingsInCurrentRoom, bookingNow, room,
floor) => {

    navigate('/editBooking', { state: { bookingsInCurrentRoom,
bookingNow, room, floor } })
}

return (
    <div className={styles.wrapper}>
        {!userBookings.length || userBookings.every(booking =>
booking.dates?.length === 0) ? (
            <p className={styles.emptyText}>{isCreatedShown ? 'You
don't have any created bookings' : 'You don't have any invited
bookings'}</p>
        ) : (
            <div className={styles.colsTitle}>
                <div className={styles.colWrapper}>
                    <p className={styles.colTitle}>Title | Purpose</p>
                </div>
                <div className={styles.colWrapper}>
                    <p className={styles.colTitle}>Start</p>
                </div>
                <div className={styles.colWrapper}>
                    <p className={styles.colTitle}>End</p>
                </div>
                <div className={styles.colWrapperMini}>
                    <p className={styles.colTitle}>Floor</p>
                </div>
                <div className={styles.colWrapperMini}>
                    <p className={styles.colTitle}>Room</p>
                </div>
            </div>
        )
    </div>
)

```

```

        </div>
        {isCreatedShown && <div className={styles.colWrapperMini}>
        </div>}
    </div>
    )}

    {
        userBookings.map(booking => {
            const purposeValue =
getPurposeValueById(booking.purposeId)
            const floor = getFloorNumberById(booking.roomId)
            const room = getRoomNumberById(booking.roomId)
            const bookingsInCurrentRoom =
getBookingsByIds(room.bookingsIds)
            return booking.dates.map(date => {
                const formattedDate = moment(date).format('MMMM DD,
YYYYY')

                if(moment(date) > moment())
                return (
                    <div className={` ${styles.colsTitle}
${styles.values}`>

                        <div className={styles.colWrapper}>
                            <p
className={styles.colTitle}>{booking.title} | {purposeValue}</p>
                        </div>
                        <div className={styles.colWrapper}>
                            <p
className={styles.colTitle}>{formattedDate} {booking.timeStart}</p>
                        </div>
                        <div className={styles.colWrapper}>
                            <p
className={styles.colTitle}>{formattedDate} {booking.timeEnd}</p>
                        </div>
                        <div className={styles.colWrapperMini}>
                            <p className={styles.colTitle}>Floor
{floor.name}</p>
                        </div>
                        <div className={styles.colWrapperMini}>
                            <p
className={styles.colTitle}>{room.name}</p>
                        </div>
                        {isCreatedShown &&<div
className={styles.colWrapperMini}>
                            <img src={Edit} alt="edit"
className={styles.icon}
onClick={()=>navigateToBookingEdit(bookingsInCurrentRoom, booking, room,
floor)}/>
                            <img src={Delete} alt="delete"
className={styles.icon} onClick={()=>{handleOpenWarning(booking,
bookingsInCurrentRoom, room, floor, date)}}/>
                        </div>}
                    </div>
                )
            }
            else
                return null
        })
    }
    {isWarningVisible && <WarningPopup

```

```

        handleClosePopup={handleCloseWarning}
        title="Are you sure you want to delete"
        onSubmit={handleBookingDelete}
      />
    </div>
  )
}

```

```

const FilterSection = () => {

  const dispatch = useDispatch()
  const floors = useSelector(store => store.floorsSlice.floors)
  const [backupFloors, setBackUpFloors] = useState(floors)
  const assets = useSelector(store => store.assetsSlice.assets)
  const rooms = useSelector(store => store.roomsSlice.rooms)
  const currentPeriod = useSelector(store => store.periodSlice.period)
  const bookings = useSelector(store => store.bookingsSlice.bookings)
  const currentDate = useSelector(store => store.dateSlice.date)
  const times = useSelector(store => store.timesSlice.times)
  const [backupRooms, setBackUpRooms] = useState(rooms)
  const capacityNumbers = [...new Set(backupRooms.map(room =>
room.capacity))].sort((a, b) => {
    if(a > b)
      return 1
    else if(a < b)
      return -1
    return 0
  })
  const capacities = capacityNumbers.map(capacity => ({
    title: `${capacity} seats`,
    type: 'checkbox'
  }))

  useEffect(() => {
    if(backupRooms.length === 0)
      setBackUpRooms(rooms)
  }, [rooms])

  const filterData = [
    {
      title: 'Floor',
      items: backupFloors.map(asset => ({
        ...asset,
        title: asset.name,
        type: 'checkbox'
      })))
    },
    {
      title: 'Features',
      items: assets.map(asset => ({
        ...asset,
        title: asset.name,
        type: 'checkbox'
      })))
    },
    {
      title: 'Capacity',

```

```

        items: capacities
    },
    {
        title: 'Availability',
        items: [
            {
                title: 'Fully available',
                type: 'checkbox'
            },
            {
                title: 'Partly available',
                type: 'checkbox'
            },
            {
                title: 'Fully booked',
                type: 'checkbox'
            }
        ]
    },
]

const filterByFloor = (values) => {
    const processedValues = values.map(value => value.name)
    if(processedValues.length)
        dispatch(setFloors(backupFloors.filter(floor =>
processedValues.includes(floor.name))))
}

const filterByFeature = (values) => {
    const filteredAssetsIds = assets
        .filter(asset => values.map(value =>
value.name).includes(asset.name))
        .map(val => val.id)
    if(filteredAssetsIds.length){
        const filteredRooms = backupRooms.filter(room => {
            return room.assetsIds.some( id =>
filteredAssetsIds.includes(id))
        })
        dispatch(setRooms(filteredRooms))
    }
}

const filterByCapacity = (values) => {
    const processedCapacities = values.map(val =>
parseInt(val.name.split(' ')[0]))
    if(processedCapacities.length){
        const filteredRooms = backupRooms.filter(room => {
            return processedCapacities.includes(room.capacity)
        })
        dispatch(setRooms(filteredRooms))
    }
}

const filterByAvailability = (values) => {
    const availability = getAvailabilityOfRooms(backupRooms, bookings,
currentPeriod, currentDate, times)
    if(values.length){
        const filteredRooms = []
        values.forEach(value => {

```



```

        if(value.name === 'Fully available')
            filteredRooms.push(...availability.available)
        else if(value.name === 'Partly available')
            filteredRooms.push(...availability.partly)
        else
            filteredRooms.push(...availability.booked)
    })
    dispatch(setRooms(filteredRooms))
}

const handleResetFilter = (e) => {

Array.from(document.getElementsByClassName(styles.filterCheckbox)).forEach(
element => element.checked = false)
    e.preventDefault()
    dispatch(setFloors([...backupFloors]))
    dispatch(setRooms([...backupRooms]))
}

const filterEntities = (filterData) => {
    filterData.forEach(data => {
        switch(data.name){
            case 'Floor':
                filterByFloor(data.values)
                break;
            case 'Features':
                filterByFeature(data.values)
                break;
            case 'Capacity':
                filterByCapacity(data.values)
                break;
            case 'Availability':
                filterByAvailability(data.values)
                break;
            default:
                break;
        }
    })
}

const handleOnSubmit = (e) => {
    e.preventDefault()
    const _filterData = []
    let iterator = 0
    filterData.forEach((filterItem, id) => {
        const _filterItem = {
            name: filterItem.title,
            values: []
        }
        filterItem.items.forEach((subItem, subId) => {
            if(e.target[iterator].checked)
                _filterItem.values.push({
                    name: subItem.title
                })
            iterator++
        })
        _filterData.push(_filterItem)
    })
}

```

```

        filterEntities(_filterData)
    }

    return (
      <div className={styles.filterWrapper}>
        <form className={styles.filter} onSubmit={handleOnSubmit}>
          <p className={styles.filterTitle}>Filter by</p>
          <div className={styles.filterItemsWrapper}>
            {
              filterData.map(filterItem => {
                return (
                  <div className={styles.filterItemWrapper}>
                    <p
                      className={styles.filterItemTitle}>{filterItem.title}</p>
                    <div
                      className={styles.filterSubItemsWrapper}>
                        {
                          filterItem.items.map(subItem =>
                            {
                              return (
                                <div
                                  className={styles.filterSubItemWrapper}>
                                    {
                                      subItem.type
                                      === 'checkbox' ?
                                      <input
                                        type="checkbox" className={styles.filterCheckbox}/> :
                                      null
                                    }
                                  <p
                                    className={styles.filterSubItemTitle}>{subItem.title}</p>
                                </div>
                              )
                            }
                          )
                        }
                      </div>
                    </div>
                  )
                }
              )
            }
          </div>
          <div className={styles.buttonsWrapper}>
            <div className={styles.buttonWrapper}>
              <CommonButton text="Reset"
                onClick={handleResetFilter}/>
            </div>
            <div className={styles.buttonWrapper}>
              <PrimaryButton />
            </div>
          </div>
        </form>
      </div>
    )
  }

```

Додаток 3
Копія презентації

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО”



ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ КОМП'ЮТЕРНИХ СИСТЕМ

ВЕБЗАСТОСУНОК ДЛЯ БРОНЮВАННЯ ПЕРЕГОВОРНИХ КІМНАТ ОФІСУ

Виконала: Музичук Марина Анатоліївна

Керівник: доц. кафедри ПЗКС, к.т.н., доц. Заболотня Тетяна Миколаївна

Київ – 2023

1/20

ПОСТАНОВКА ЗАДАЧІ



Мета проєкту: забезпечення ефективного використання переговорних кімнат, уникнення конфліктів зустрічей при бронюванні приміщення, шляхом розроблення відповідного програмного забезпечення.

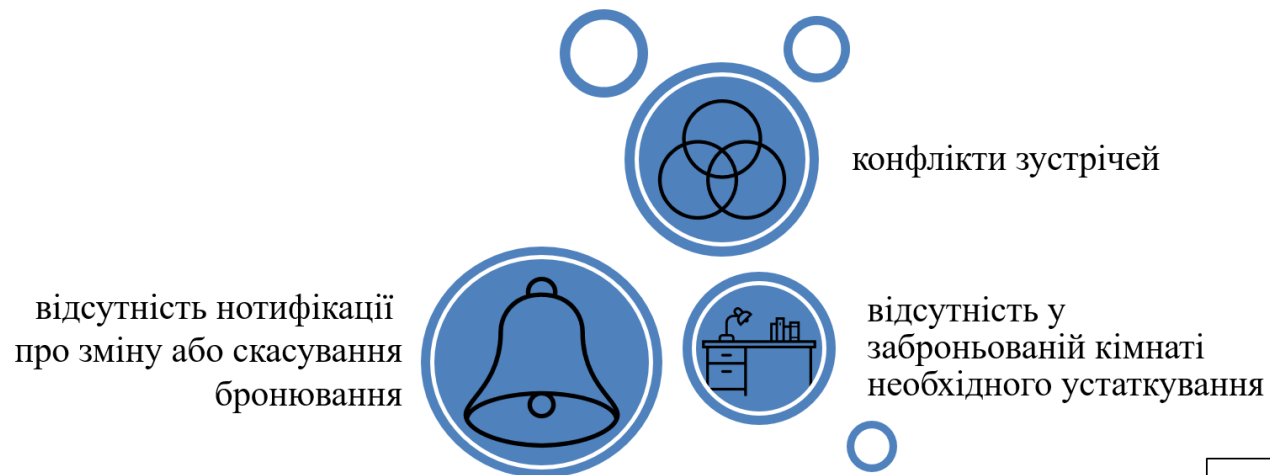
Завдання:

1. Проаналізувати ринок ПЗ для бронювання конференц-залів
2. Порівняти конкурентів, виявити їхні сильні та слабкі сторони
3. Визначити функціональні та нефункціональні вимоги до ПЗ
4. Підготувати дизайн застосунку
5. Обґрунтувати вибір стеку технологій для розроблення ПЗ
6. Програмно реалізувати вебзастосунок для бронювання переговорних кімнат офісу
7. Протестувати систему на відповідність вимогам
8. Навести шляхи подальшого покращення застосунку

АКТУАЛЬНІСТЬ



Станом на **2023 рік** кількість людей, що повернулись до офісного формату роботи, у порівнянні зі статистикою на **середину 2021 року** зросла на **80%**.



ІСНУЮЧІ РІШЕННЯ



Evoko Liso



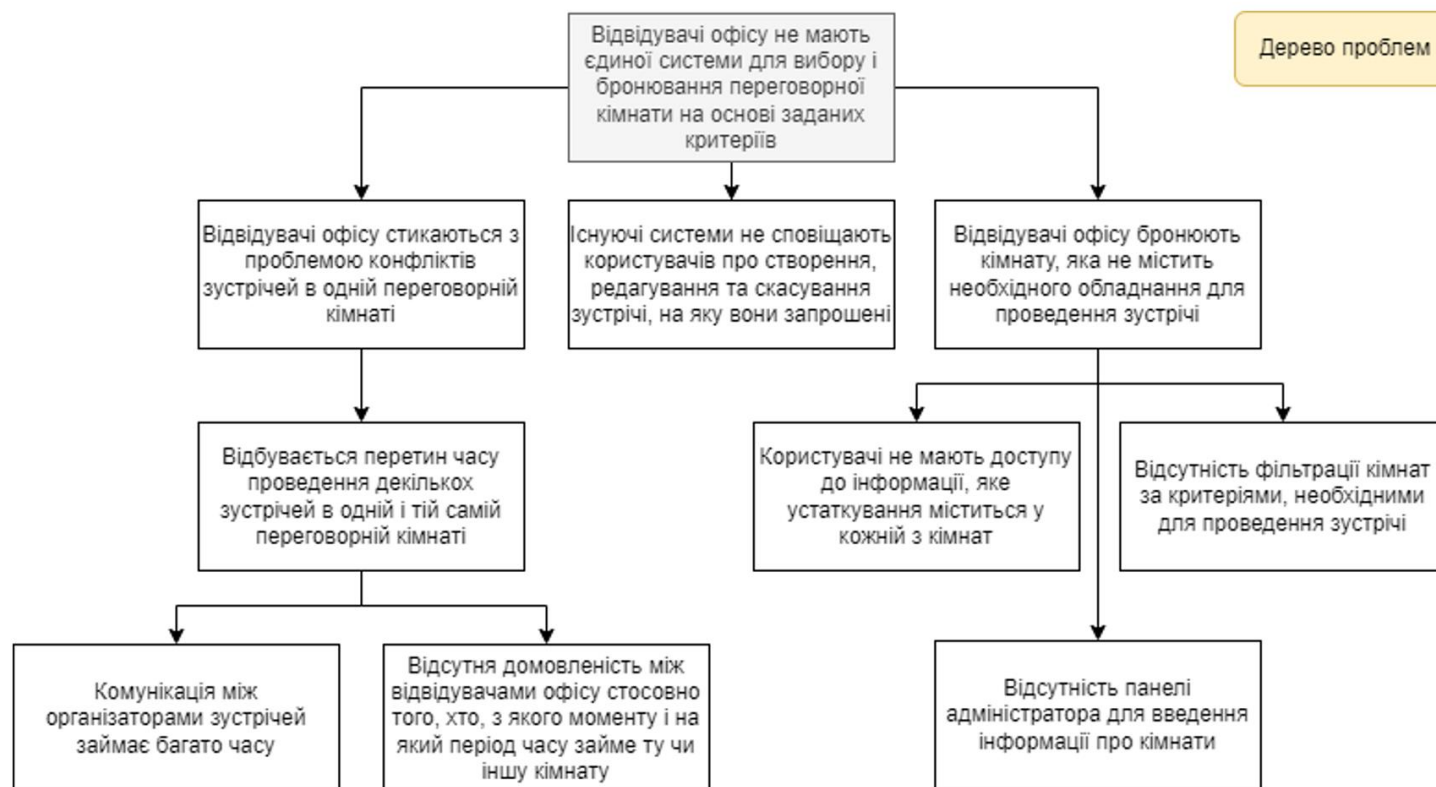
Google Calendar



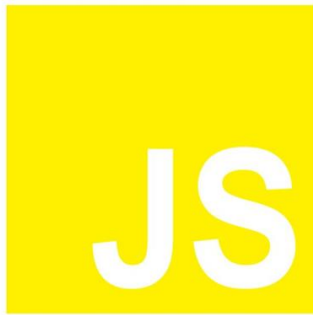
Microsoft Excel



ДЕРЕВО ПРОБЛЕМ



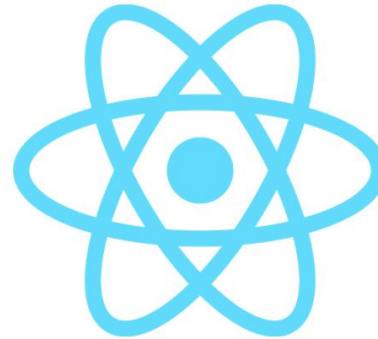
ВИБІР ТЕХНОЛОГІЙ РОЗРОБЛЕННЯ



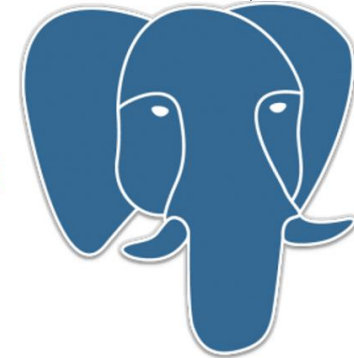
JavaScript



Express.js



React.js



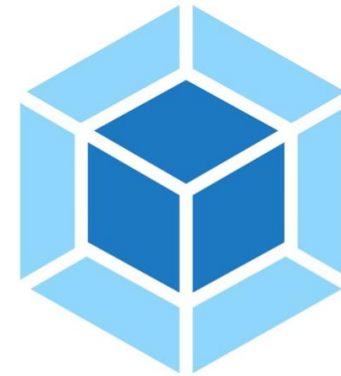
PostgreSQL



Moment.js

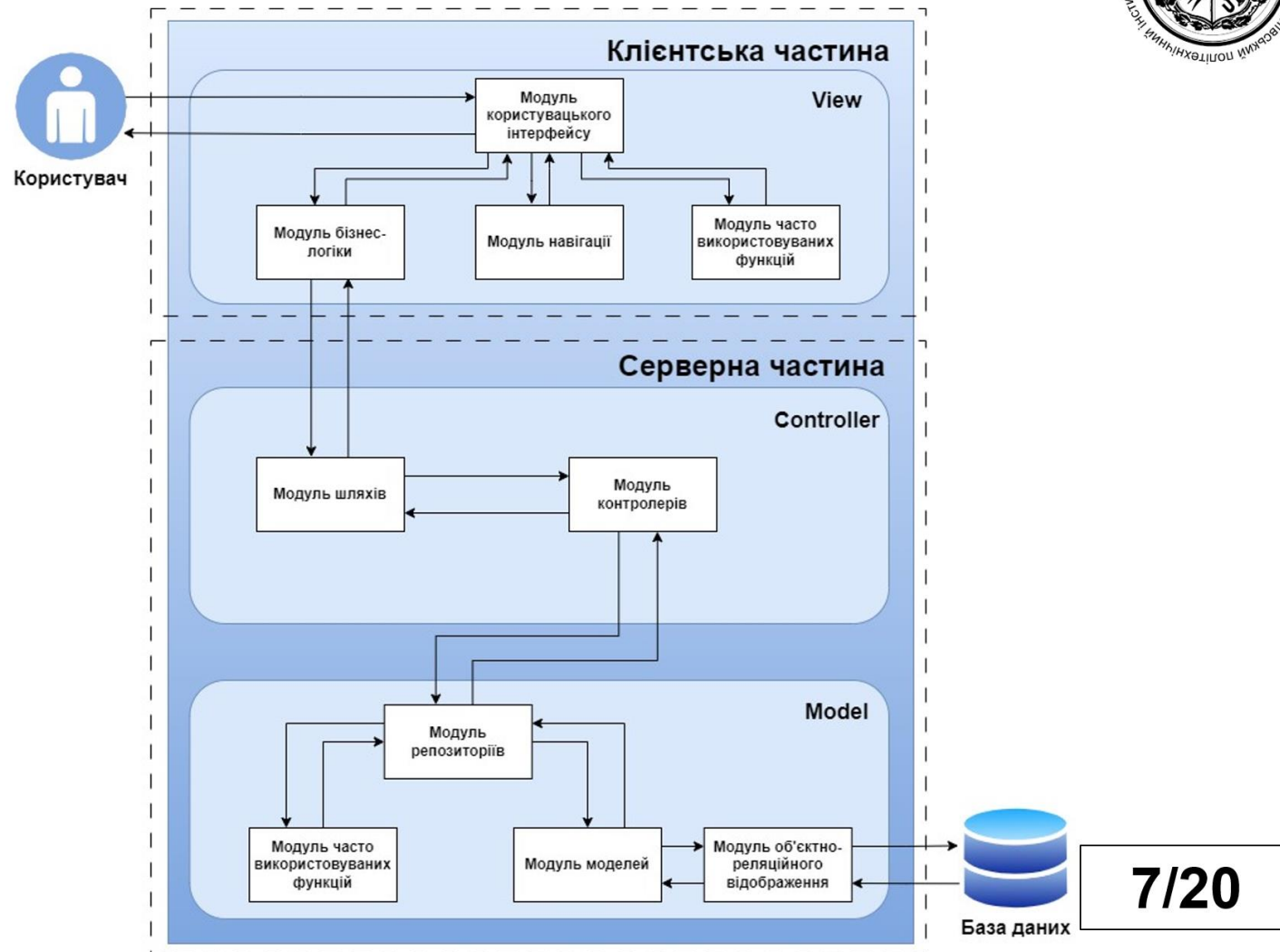


Nodemailer

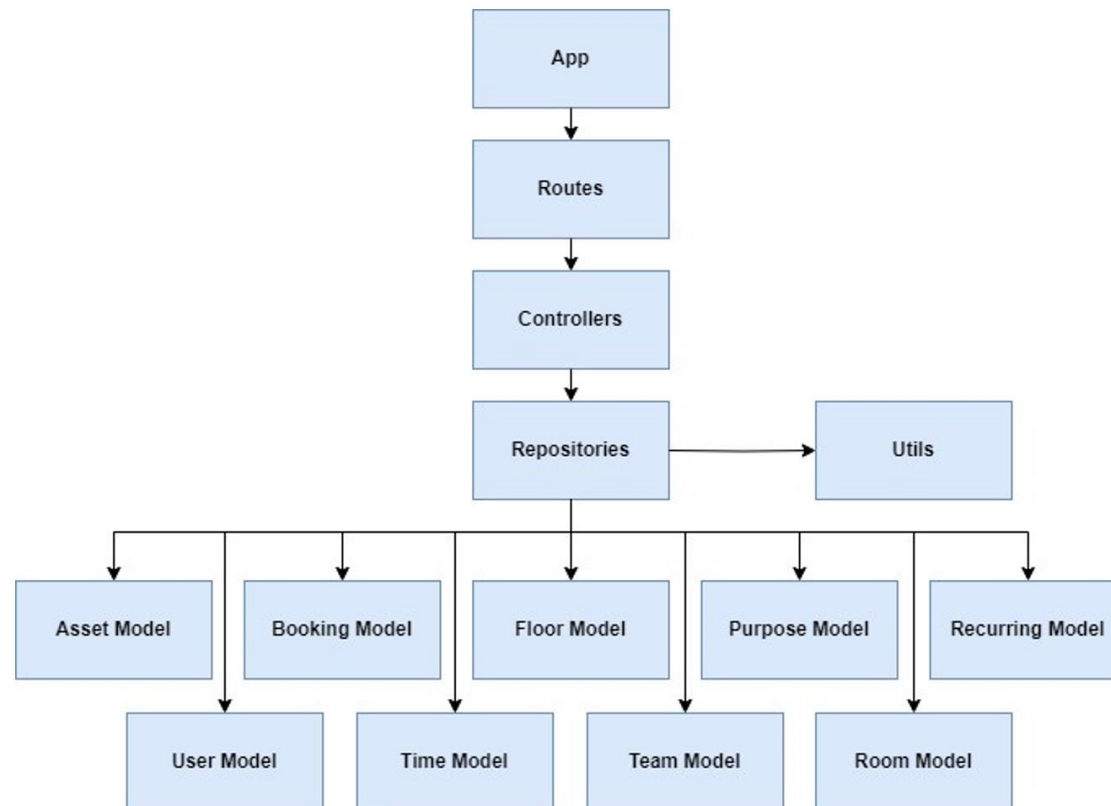


File-loader

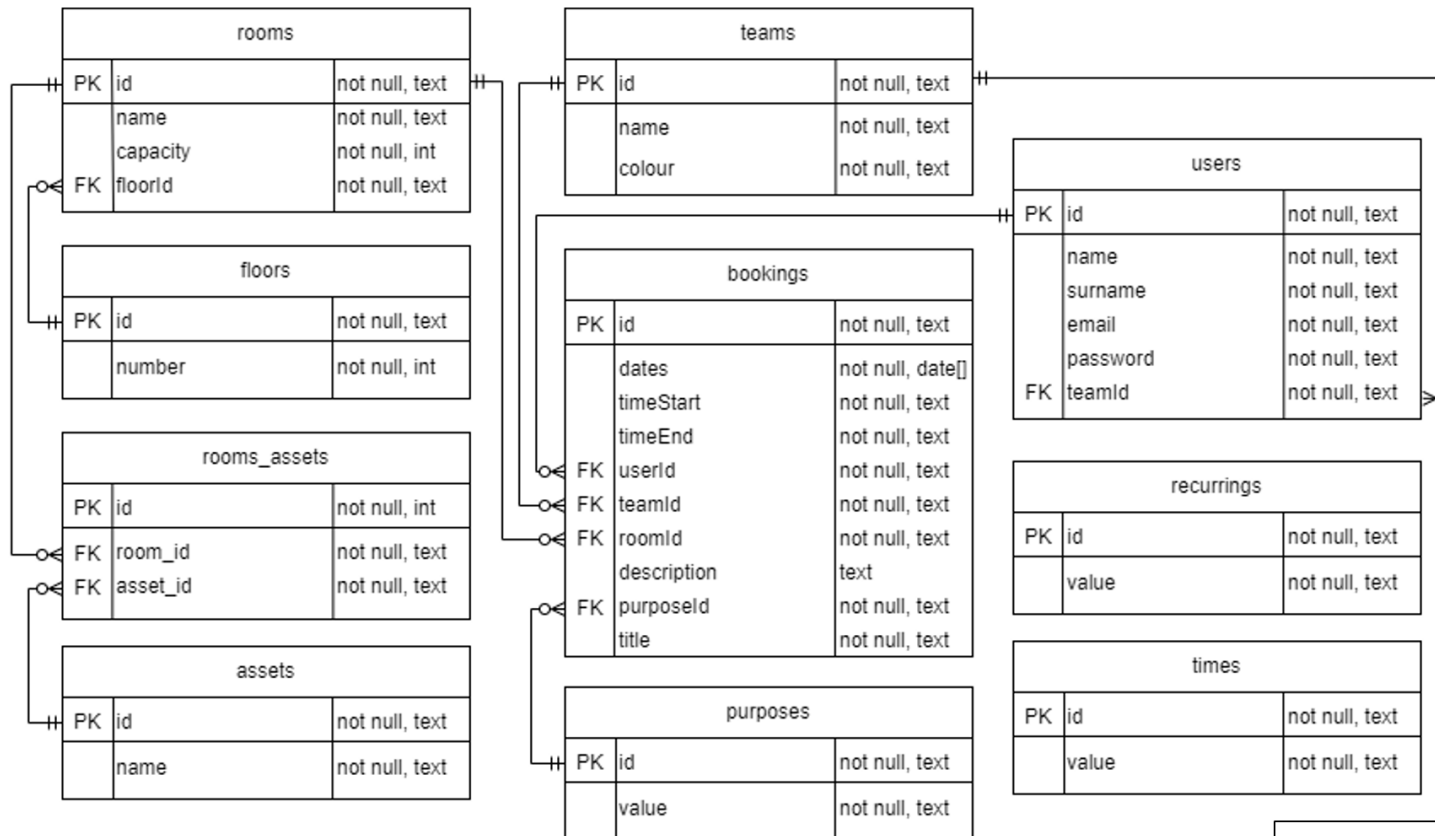
АРХІТЕКТУРА СИСТЕМИ



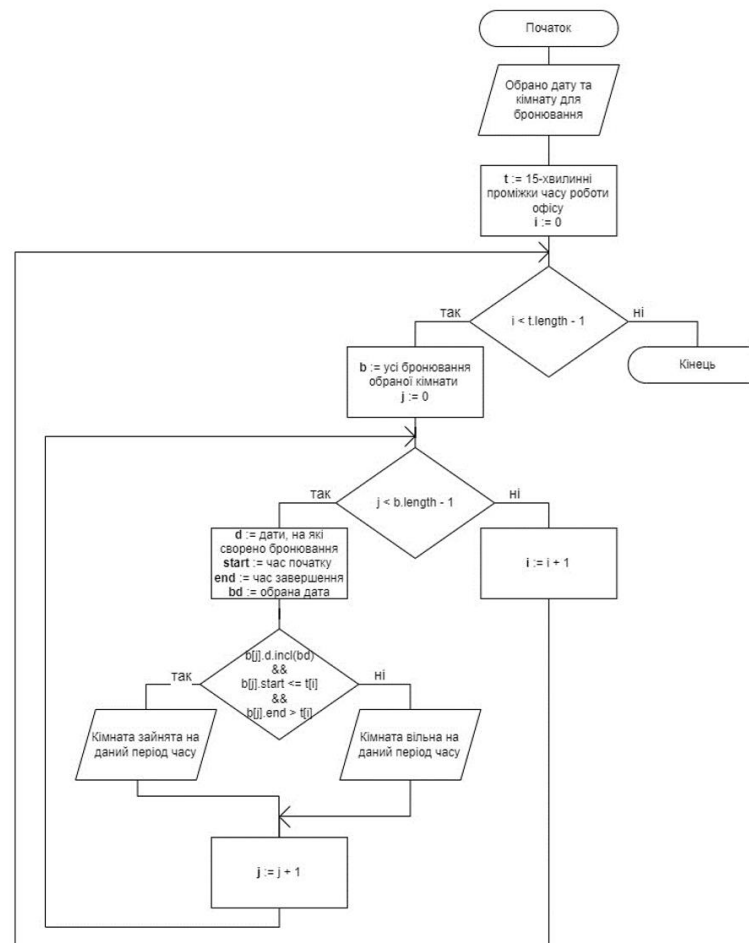
ДІАГРАМА МОДУЛЬНОЇ ЗАЛЕЖНОСТІ СЕРВЕРНОЇ ЧАСТИНИ ВЕБЗАСТОСУНКУ



СТРУКТУРА БД



ДІАГРАМА ДІЯЛЬНОСТІ. АЛГОРИТМ ПЕРЕВІРКИ КІМНАТИ НА ЗАЙНЯТІСТЬ



СТОРІНКА ПЕРЕГЛЯДУ ЗАВАНТАЖЕНОСТІ КІМНАТ



iRoom

View availability

My bookings

Admin panel

Logout

Book a room

June 2, 2023

Day

Week

Month

June													
28	29	30	31	1	2	3							
4	5	6	7	8	9	10							
11	12	13	14	15	16	17							
18	19	20	21	22	23	24							
25	26	27	28	29	30	1							

Filter by

Floor

☐ One

☐ Two

☐ Three

Features

☐ PC Lab

☐ Projector

☐ TV

☐ Operable walls

☐ Whiteboard

☐ Power outlets

Floor One													
Room	Assets	8	9	10	11	12	13	14	15	16	17	18	19
Room A	PC Lab Power outlets												
Room B	PC Lab TV												

Floor Two													
Room	Assets	8	9	10	11	12	13	14	15	16	17	18	19
Italy	Power outlets												
Spain	Operable walls												
France	Whiteboard												

11/20

СТОРІНКА ДЛЯ СТВОРЕННЯ БРОНЮВАННЯ



iRoom

View availability

My bookings

Admin panel

Logout

Floor One | Room A

June 2, 2023

June						
28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	1

02 June, 2023	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	
19	

Make a booking

Start Time End Time

Title

Business unit

Purpose

Recurring

Recurring end date

Description

12/20

ПАНЕЛЬ АДМІНІСТРАТОРА



iRoom

View availability

My bookings

Admin panel

Logout

- Working hours
- Floors + Rooms
- Assets
- Purposes
- Users
- Business units

Floor One

Go back

+ ROOM

Title	Capacity	Assets	Actions
Room A	20	PC Lab Power outlets	
Room B	20	PC Lab TV	
Room C	10	Whiteboard	
Room D	20	Power outlets	

ОБ'ЄКТИ ОЦІНЮВАННЯ ЯКОСТІ РОЗРОБЛЕНОГО ПЗ



ЗАСОБИ ТА ПОРЯДОК ОЦІНЮВАННЯ ЯКОСТІ РОЗРОБЛЕНОГО ПЗ



РЕЗУЛЬТАТИ ТЕСТУВАННЯ РОЗРОБЛЕНОГО ПЗ ТА АНАЛОГІВ



Критерій	Evoko Liso	Google Calendar	Microsoft Excel	iRoom
Упередження появи конфліктів зустрічей	+	-	-	+
Доступ до панелі адміністратора користувачам з відповідними правами	+	-	-	+
Можливість створення, редагування, скасування власних бронювань	+	+	+	+
Можливість створення одноразових та регулярних бронювань	+	+	+/-	+
Можливість надсилання електронного листа організатору іншої зустрічі	-	-	-	+
Можливість отримування електронних листів про запрошення на зустріч, її редагування та скасування	+/-	+	-	+
Можливість перегляду списку власних бронювань	+/-	-	-	+

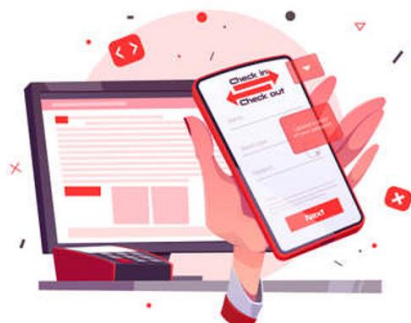
ШЛЯХИ ПОДАЛЬШОГО РОЗВИТКУ ПРОЄКТУ



інтеграція з іншими
електронними
календарями



розширення можливостей
створення бронювань



створення мобільної версії
застосунку



надсилання нагадувань
про проведення зустрічі
за 15 хв. до її початку

17/20

ВИСНОВКИ



Досліджено ряд проблем, які постають перед відвідувачами та менеджерами офісу



Проаналізовано існуючі програмні рішення, виділено переваги та недоліки кожного з них



Запропоновано архітектуру ПЗ, яка допоможе вирішенню виявлених проблем



Розроблено алгоритм перевірки переговорних кімнат на зайнятість



Розроблено ПЗ, яке дозволяє налаштовувати значення параметрів сутностей офісу та відслідковувати бронювання



Протестовано та порівняно ПЗ з аналогами за основними критеріями та проаналізовано, що розробка повністю відповідає вимогам



Знайдено шляхи подальшого розвитку проєкту

УНІКАЛЬНІСТЬ



Имя пользователя:
Заболотня Тетяна Миколаївна

ID проверки:
1015586387

Дата проверки:
13.06.2023 15:11:35 EEST

Тип проверки:
Doc vs Internet + Library

Дата отчета:
13.06.2023 15:26:37 EEST

ID пользователя:
83364

Название файла: Антиплагиат Музичук

Количество страниц: 49 Количество слов: 10424 Количество символов: 78625 Размер файла: 135.07 KB ID файла: 1015233178

0.61% Совпадения

Наибольшее совпадение: 0.3% с источником из Библиотеки (ID файла: 1008350372)

0.15% Источники из Интернета 4 Страница 51

0.54% Источники из Библиотеки 33 Страница 51

0.9% Цитат

Цитаты 9 Страница 52

Исключение списка библиографических ссылок выключено

0% Исключений

Нет исключенных источников

19/20



Дякую за увагу!

Факультет прикладної математики
Кафедра програмного забезпечення комп'ютерних систем

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Євгенія СУЛЕМА

“ ____ ” _____ 2022 р.

ВЕБЗАСТОСУНОК ДЛЯ БРОНЮВАННЯ ПЕРЕГОВОРНИХ
КІМНАТ ОФІСУ

Програма та методика тестування

ДП.045440-04-51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЗАБОЛОТНЯ

Нормоконтроль:

_____ Микола ОНАЙ

Виконавець:

_____ Марина МУЗИЧУК

ЗМІСТ

1. Об'єкт випробувань	3
2. Мета тестування	3
3. Методи тестування.....	3
4. Засоби та порядок тестування.....	4

1. ОБ'ЄКТ ВИПРОБУВАНЬ

Вебзастосунок для бронювання переговорних кімнат офісу, який являє собою вебсайт, створений з використанням мови програмування JavaScript.

2. МЕТА ТЕСТУВАННЯ

У процесі тестування має бути перевірено наступне:

- 1) функціональна працездатність елементів сторінок вебзастосунку;
- 2) можливість створення, редагування та скасування власних бронювань;
- 3) можливість перегляду списку власних бронювань;
- 4) отримування електронних листів про запрошення на зустріч, її редагування та скасування;
- 5) доступ до панелі адміністратора для користувачів з відповідними правами;
- 6) забезпечення належного рівня безпеки даних;
- 7) зручність роботи з вебзастосунком;
- 8) відповідність дизайну вимогам Технічного завдання.

3. МЕТОДИ ТЕСТУВАННЯ

Тестування виконується методом Gray Box Testing. Перевіряється як код, так і безпосередньо програмний продукт на відповідність функціональним вимогам. Тестування відбувається на рівні «системного тестування».

Використовуються наступні методи:

- 1) функціональне тестування, зокрема на рівні інтеграційного тестування (тестування взаємодії різних модулів програми) та тестування інтерфейсу (тестування користувацької взаємодії);

- 2) тестування продуктивності програмного забезпечення, зокрема Stability testing (тестування стабільності) та Load testing (навантажувальне тестування);
- 3) тестування інтерфейсу.

4. ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується засобами інструментарію SpecFlow.

Працездатність вебзастосунку перевіряється шляхом:

- 1) динамічного ручного тестування – введенням граничних та недопустимих значень в поля, які можна редагувати;
- 2) динамічного ручного тестування на відповідність функціональним вимогам;
- 3) тестування вебзастосунку в різних веббраузерах;
- 4) тестування при максимальному навантаженні;
- 5) тестування стабільності роботи при різних умовах;
- 6) тестування зручності використання;
- 7) тестування інтерфейсу.

Факультет прикладної математики
Кафедра програмного забезпечення комп'ютерних систем

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Євгенія СУЛЕМА

“ ____ ” _____ 2023 р.

ВЕБЗАСТОСУНОК ДЛЯ БРОНЮВАННЯ ПЕРЕГОВОРНИХ
КІМНАТ ОФІСУ

Керівництво користувача

ДП.045440-05-34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Тетяна ЗАБОЛОТНЯ

Нормоконтроль:

_____ Микола ОНАЙ

Виконавець:

_____ Марина МУЗИЧУК

ЗМІСТ

1. Опис структури вебзастосунку.....	3
2. Процедура авторизації користувача.....	3
3. Сторінка з таблицею завантаженості переговорних кімнат	5
4. Сторінка для створення бронювання	8
5. Сторінка для редагування бронювання	9
6. Сторінка для перегляду власних бронювань	11
7. Сторінка для налаштування робочих годин офісу	13
8. Сторінка для управління поверхами та переговорними кімнатами	14
9. Сторінка для управління обладнанням кімнат	16
10. Сторінка для управління цілями зустрічей	16
11. Сторінка для управління сутностями користувачів системи	17
12. Сторінка для управління сутностями команд	18

1. Опис структури вебзастосунку

Вебзастосунок для бронювання переговорних кімнат офісу реалізовано у вигляді односторінкового застосунку. Він містить у собі наступні складові:

- 1) сторінку реєстрації та авторизації;
- 2) головну сторінку з таблицею завантаженості переговорних кімнат;
- 3) сторінку для створення бронювання;
- 4) сторінку для редагування бронювання;
- 5) сторінку для перегляду власних бронювань;
- 6) сторінку для налаштування робочих годин офісу;
- 7) сторінку для управління сутностями поверхів та переговорних кімнат офісу;
- 8) сторінку для управління обладнанням кімнат;
- 9) сторінку для управління цілями зустрічей;
- 10) сторінку для управління користувачами системи;
- 11) сторінку для управління командами.

Останні 6 сторінок, згаданих у переліку існуючих, відносяться до адміністративної панелі вебзастосунку, доступної лише адміністратору системи.

2. Процедура авторизації користувача

При відвідуванні вебзастосунку для бронювання переговорних кімнат офісу користувач потрапляє на сторінку для авторизації. В залежності від того, чи даний користувач уже має обліковий запис у системі чи ні, він вирішує, яку процедуру йому потрібно виконати: реєстрацію чи авторизацію.

Якщо відвідувач вебзастосунку має на меті зареєструватись у системі, йому потрібно натиснути на кнопку «Sign up», у результаті чого система відобразить форму для реєстрації з трьома полями для заповнення: логін, пароль та підтвердження паролю. Заповнивши усі поля та натиснувши на кнопку «Sign up», користувач направляється на «View availability» сторінку системи за умови, що усі правила валідації виконано успішно. Зареєструватись у вебзастосунку можуть лише ті користувачі, чий записи заздалегідь завантажив у систему адміністратор. У разі некоректного заповнення полів форми реєстрації відвідувач системи побачить помилку і не буде допущеним до наступної сторінки, поки не задовольнить вимоги.

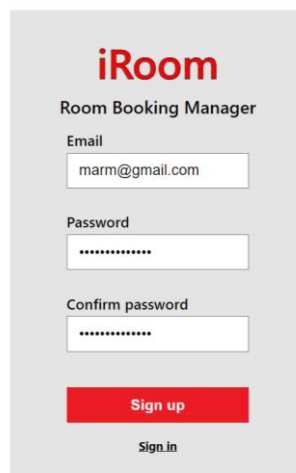
The image shows a registration form for 'iRoom Room Booking Manager'. At the top, the logo 'iRoom' is in red, followed by 'Room Booking Manager' in black. Below this are three input fields: 'Email' with the text 'marm@gmail.com', 'Password' with masked characters '*****', and 'Confirm password' also with masked characters '*****'. At the bottom, there is a prominent red button labeled 'Sign up' and a smaller, less visible link labeled 'Sign in' below it.

Рис. 1. Форма для реєстрації користувача

Якщо користувач при відвідуванні вебзастосунку уже має свій обліковий запис у системі, то проходити реєстрацію йому не потрібно. У такому разі користувач має заповнити поля форми авторизації: логін та пароль. У результаті заповнення полів та натиснення на кнопку «Sign in» користувач потрапить на головну сторінку системи або побачить помилку авторизації, якщо некоректно ввів логін чи пароль.

Рис. 2. Форма авторизації користувача

3. Сторінка з таблицею завантаженості переговорних кімнат

Після успішної процедури авторизації користувач направляється на головну сторінку системи. Потрапити на неї відвідувач вебзастосунку може з будь-якого місця програми, натиснувши на вкладку «View availability». Основний елемент сторінки – таблиця завантаженості переговорних кімнат, яка може мати 3 вигляди: станом на день, тиждень та місяць. Змінювати відображення можна, переключаючи відповідні клавiши над таблицею. Щоб обрати інший день, тиждень чи місяць, користувач може використовувати календар у лівому верхньому куті сторінки.

Таблиця завантаженості кімнат складається з декількох стовпчиків. У першому розміщені назви переговорних кімнат. Другий стовпчик «Assets» відображає обладнання кімнат. Кількість та наповнення наступних стовпців таблиці залежить від обраного вигляду: «Day», «Week» або «Month». Якщо обрано відтворення «Day», то останні колонки таблиці відповідають за робочі години офісу. Кольори є певним ідентифікатором команди, що запрошена на бронювання. Якщо клітинки таблиці білі, це означає, що дана переговорна кімната не заброньована на відповідний період часу.

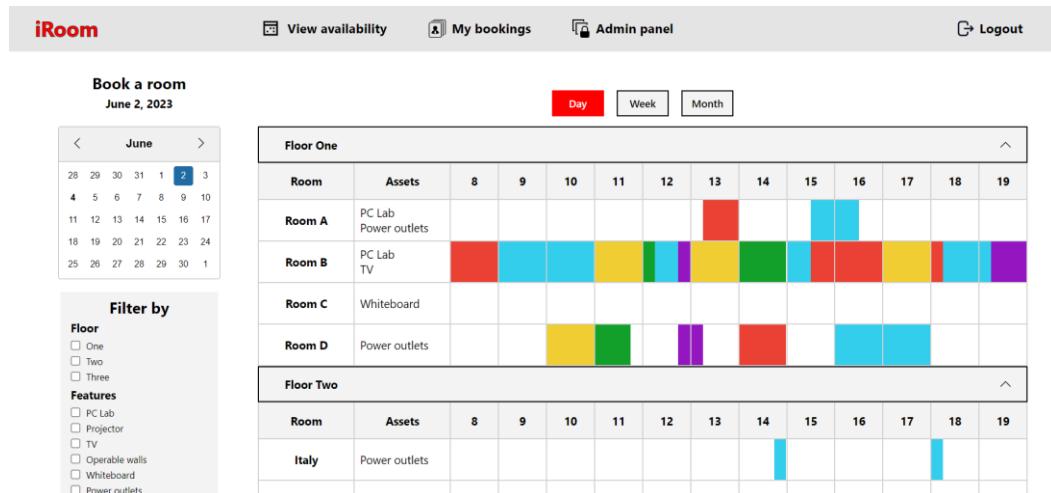


Рис. 3. Таблиця завантаженості кімнат станом на день

Якщо користувач обрав «Week» вигляд, то система відображатиме 7 колонок, що відповідатимуть за дні тижня. Оранжевий колір означає, що дана переговорна кімната заброньована частково на цей день. Червоним позначаються дні, у які кімната заброньована на всі робочі години офісу. Якщо клітинка біла, то приміщення повністю вільне у відповідний день.

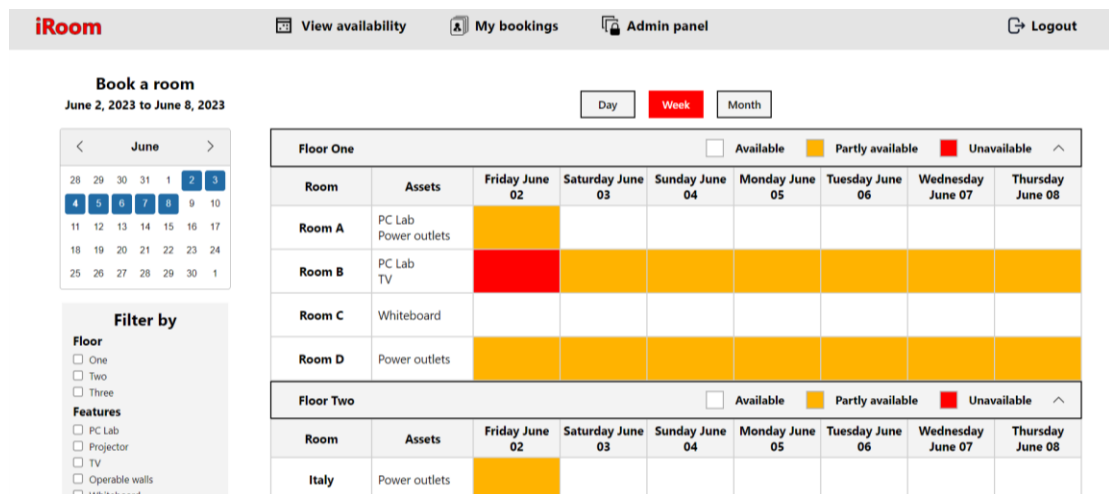


Рис. 4. Таблиця завантаженості кімнат станом на тиждень

Стовпчики таблиці завантаженості станом на «Month» відображатимуть тижні обраного місяця з поділом на дні.

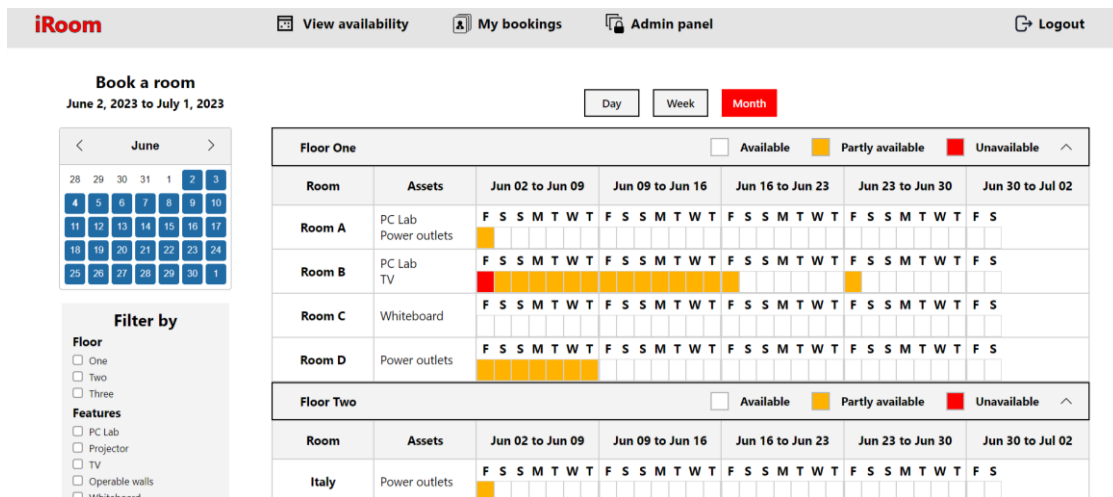


Рис. 5. Таблиця завантаженості кімнат станом на місяць

Крім цього, на головній сторінці системи розміщено фільтрацію, за допомогою якої користувачі можуть обирати переговорні кімнати за певними критеріями, а саме: поверх, обладнання, місткість та доступність. Обравши деякі параметри фільтрації та натиснувши на кнопку «Submit», користувач матиме можливість бачити у таблиці завантаженості лише ті переговорні кімнати, що задовольняють обрані критерії. Якщо ж користувач натисне на кнопку «Reset» у полі фільтрації, система анулює усі параметри та відображатиме усі переговорні кімнати офісу.



Рис. 6. Результат фільтрації записів про переговорні кімнати

У випадку, якщо за заданими параметрами не знайдено жодної кімнати, система відображає відповідне повідомлення на сторінці.

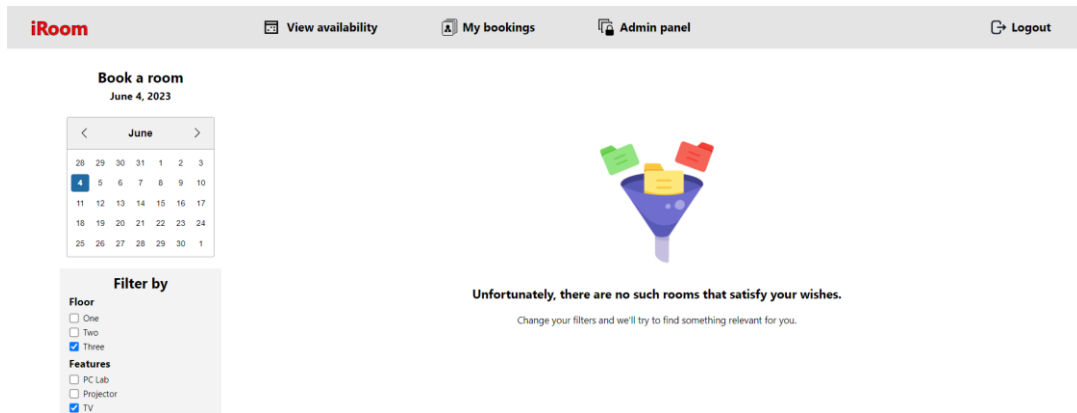


Рис. 7. Повідомлення про відсутність кімнат, що задовольняють параметри

4. Сторінка для створення бронювання

Після того, як користувач натискає на білу клітинку таблиці завантаженості, система направляє його на сторінку для створення бронювання з обраним часом його початку. Оскільки мінімальна тривалість букінгу – 15 хвилин, час його завершення автоматично заповнюється значенням, що на 15 хвилин більше за час його початку. При цьому користувач має можливість змінити ці поля, обравши доступні вільні проміжки часу.

Сторінка для створення бронювання розміщує на собі інформацію про обрану кімнату, календар, таблицю завантаженості даної кімнати в обраний день та форму для створення зустрічі. Форма складається з декількох полів для заповнення, серед них час початку та кінця бронювання, назва, команда, ціль, повторюваність та опис. Після того як користувач заповнив усі поля та натиснув на клавішу «Submit», система перевіряє поля на коректність заповнення, зберігає дані та направляє користувача на «View availability» сторінку.

iRoom

View availability

My bookings

Admin panel

Logout

Floor One | Room A

June 2, 2023

< June >

28 29 30 31 1 2 3

4 5 6 7 8 9 10

11 12 13 14 15 16 17

18 19 20 21 22 23 24

25 26 27 28 29 30 1

02 June, 2023

8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	
19	

Make a booking

Start Time

11:00

End Time

12:30

Title

Sprint 80

Business unit

Octo

Purpose

Retrospective session

Recurring

Once a month

Recurring end date

08/11/2023

Description

Be ready to discuss our work of the previous sprint!

RESET

SUBMIT

Рис. 8. Сторінка для створення бронювання

5. Сторінка для редагування бронювання

Коли користувач натискає на зафарбовану клітинку таблиці завантаженості, програма перевіряє, хто є організатором даного бронювання. Якщо це інша людина, то система відображає користувачеві діалогове вікно з інформацією про обрану зустріч та клавішу «Contact».

iRoom

View availability

My bookings

Admin panel

Logout

Book a room

June 2, 2023

< June >

28 29 30 31 1 2 3

4 5 6 7 8 9 10

11 12 13 14 15 16 17

18 19 20 21 22 23 24

25 26 27 28 29 30 1

Filter by

Floor

☐ One
 ☐ Two
 ☐ Three

Features

☐ PC Lab
 ☐ Projector
 ☐ TV
 ☐ Operable walls
 ☐ Whiteboard
 ☐ Power outlets

Capacity

☐ 10 seats
 ☐ 15 seats

Day Week Month

Floor One

Room	Assets	8	9	10	11	12	13	14	15	16	17	18	19
Room A	PC Lab, Power s												
Room B	PC Lab, TV												
Room C	Whiteboard												
Room D	Power s												

Booking details

Booking title: New tasks discussion

Room: Room D

Floor: One

Duration: from 08:00 till 09:00

Team: Octo

Organizer: Yuri Pylypovych

CONTACT

Floor Two

Room	Assets	8	9	10	11	12	13	14	15	16	17	18	19
Italy	Power outlets												
Spain	Operable walls												
France	Whiteboard												

Рис. 9. Діалогове вікно з інформацією про стороннє бронювання

При натисканні на клавішу «Contact» користувач направляється у поштовий агент, де вже створено шаблон повідомлення з заповненими полями адресата, адресанта і теми листа.

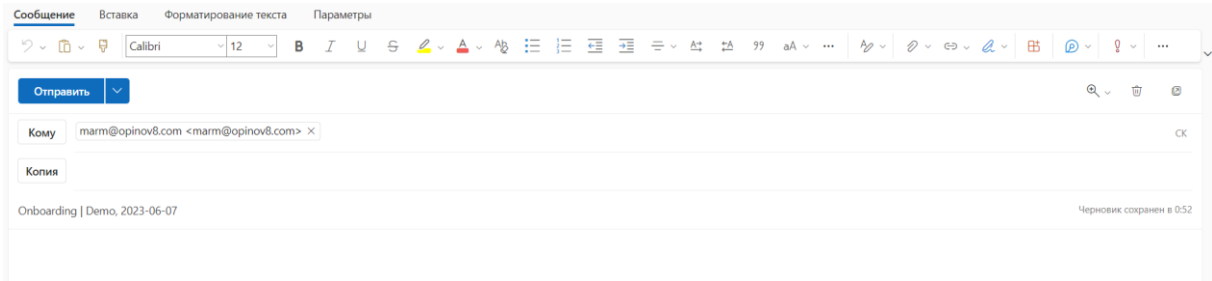


Рис. 10. Поштовий агент з заповненим шаблоном електронного листа

Якщо організатором бронювання, на яке натиснув користувач, є він, система відображає діалогове вікно з детальною інформацією про зустріч та дві клавіші «Edit» та «Delete».

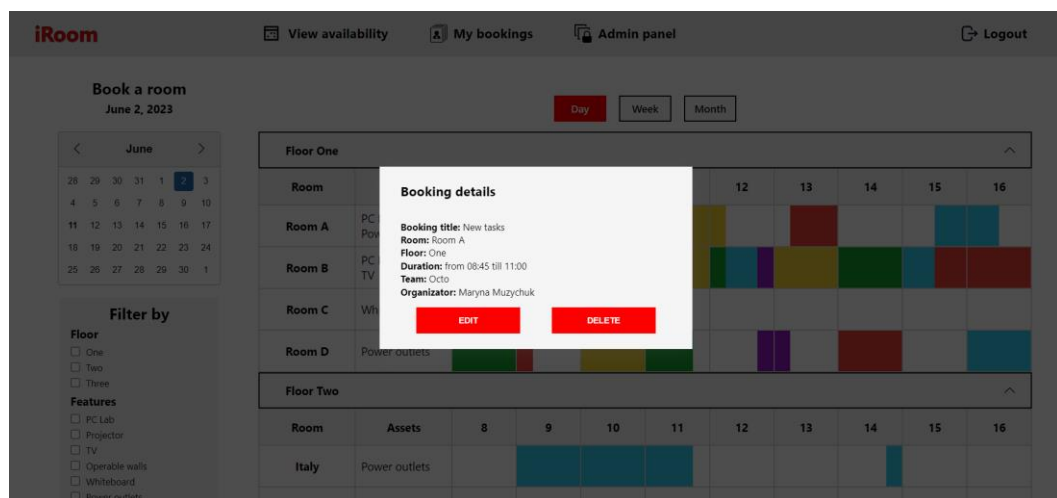


Рис. 11. Діалогове вікно з інформацією про власне бронювання

При натисканні на клавішу «Delete» користувач побачить додаткове діалогове вікно з підтвердженням, чи він дійсно хоче скасувати дане бронювання.

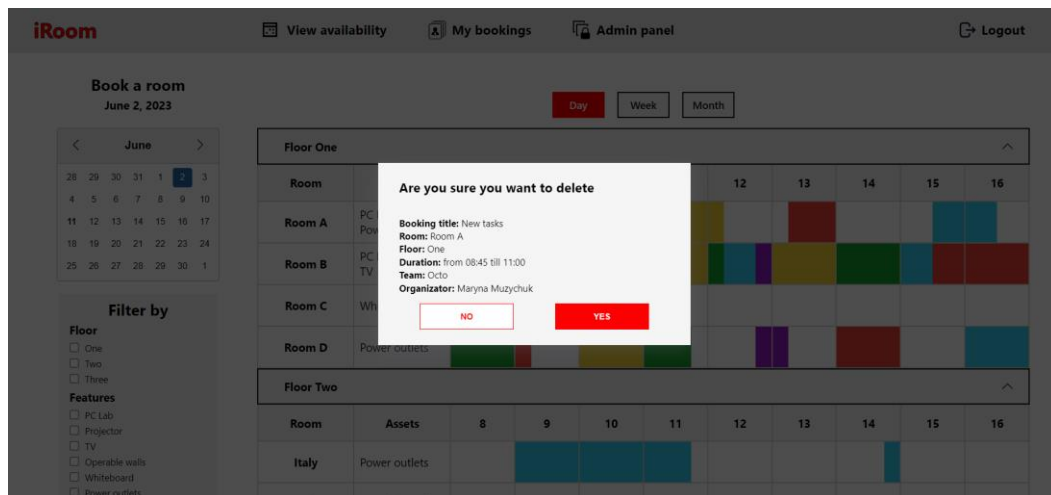


Рис. 12. Діалогове вікно з підтвердженням скасування бронювання

Якщо користувач натиснув на клавішу «Edit» у діалоговому вікні з інформацією про власне бронювання, система направляє його на сторінку для редагування бронювання. Форма редагування подібна до форми створення бронювання, до усіх її полів дозволяється вносити зміни.

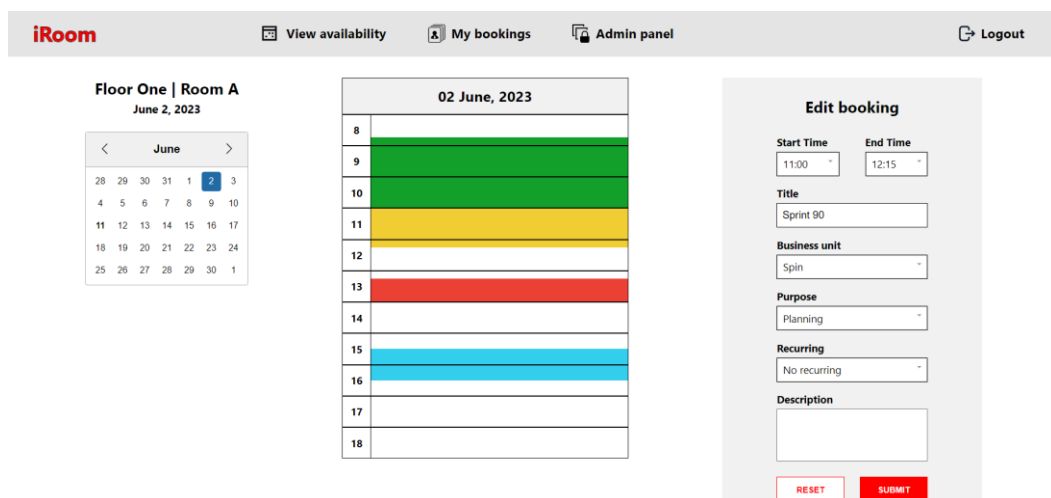


Рис. 13. Сторінка для редагування власного бронювання

6. Сторінка для перегляду власних бронювань

При натисненні на вкладку «My bookings» відвідувач сайту направляється на сторінку з інформацією про бронювання, що є створеними

ним та тими, на які його запросили, у вигляді таблиці. Щоб переключатись між ними, користувачеві потрібно натискати на відповідну вкладку: «Created» або «Invited».

iRoom View availability My bookings Admin panel Logout					
Created Invited					
Title Purpose	Start	End	Floor	Room	Actions
New tasks discussion Refinement session	June 14, 2023 08:30	June 14, 2023 09:30	Floor Two	Italy	
New tasks discussion Refinement session	June 21, 2023 08:30	June 21, 2023 09:30	Floor Two	Italy	
New tasks discussion Refinement session	June 28, 2023 08:30	June 28, 2023 09:30	Floor Two	Italy	
Stand up Daily meeting	June 12, 2023 09:30	June 12, 2023 10:00	Floor Two	Italy	

Рис. 14. Сторінка з бронюваннями, створеними користувачем

Функції редагування та видалення доступні лише для бронювань, у яких даний користувач є організатором, тобто тих, що розміщені у вкладці «Created». В останній колонці таблиці «Actions» розміщуються клавіші, при натисненні на які користувач матиме змогу внести зміни до бронювання або видалити його. У таблиці з вкладки «Invited» таких клавіш немає, що унеможлиблює користувача редагувати та скасовувати чужі зустрічі.

iRoom View availability My bookings Admin panel Logout					
Created Invited					
Title Purpose	Start	End	Floor	Room	
Sprint 80 Planning	June 12, 2023 09:30	June 12, 2023 10:30	Floor One	Room A	
Sprint 80 Planning	June 13, 2023 09:30	June 13, 2023 10:30	Floor One	Room A	
Sprint 80 Planning	June 14, 2023 09:30	June 14, 2023 10:30	Floor One	Room A	
Backlog review Planning	June 14, 2023 16:00	June 14, 2023 16:30	Floor Two	Italy	

Рис. 15. Сторінка з бронюваннями, на які запрошено користувача

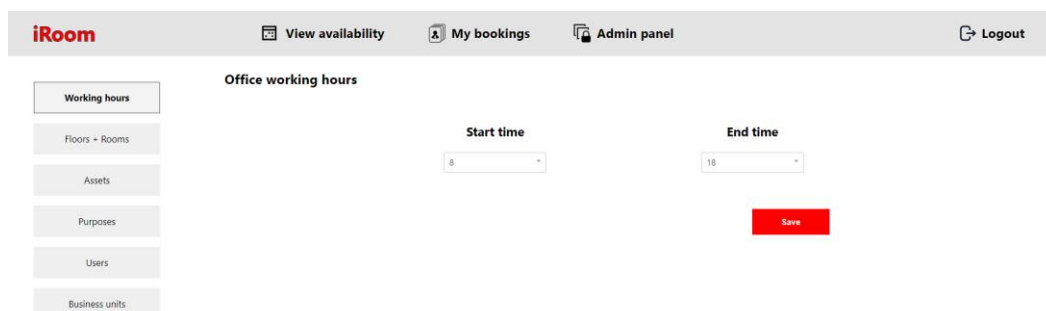
7. Сторінка для налаштування робочих годин офісу

Для користувачів, що мають адміністраторські права доступу, у системі у верхній частині сторінці відображається вкладка «Admin panel», за допомогою якої можна управляти різними сутностями системи.

Панель адміністратора складається з декількох розділів, а саме:

- 1) налаштування робочих годин офісу;
- 2) управління сутностями поверху та кімнати;
- 3) управління сутностями обладнань переговорних кімнат;
- 4) управління сутностями цілей бронювань;
- 5) управління сутностями користувачів системи;
- 6) управління сутностями команд.

За замовчуванням, при натисненні на вкладку «Admin panel» система направляє користувача на сторінку для налаштування робочих годин офісу. На ній розміщено два випадаючих списки для вибору початкового та кінцевого часу, а також клавішу для підтвердження.



The screenshot shows the 'iRoom' Admin panel. The top navigation bar includes 'View availability', 'My bookings', 'Admin panel' (active), and 'Logout'. The main content area is titled 'Office working hours'. On the left is a sidebar menu with options: 'Working hours' (selected), 'Floors - Rooms', 'Assets', 'Purposes', 'Users', and 'Business units'. The main area contains two dropdown menus for 'Start time' (set to 8) and 'End time' (set to 18). A red 'Save' button is located at the bottom right of the form.

Рис. 16. Сторінка для налаштування робочих годин офісу

Після того, як користувач оновить часи роботи офісу та натисне на клавішу «Save», система направить його на головну сторінку з оновленою таблицею, у якій відображатимуться змінені години. Крім цього, при створенні та редагуванні бронювань, користувач матиме змогу обирати час початку та кінця зустрічі, що знаходиться у рамках робочих годин офісу.

8. Сторінка для управління поверхами та переговорними кімнатами

При натисненні на розділ «Floors + Rooms» у вкладці «Admin panel» користувач потрапить на сторінку для управління сутностями поверхів офісу. На ній розміщено таблицю з уже створеними поверхами, клавішами для редагування та видалення відповідних сутностей, а також клавішу для додавання нового поверху до системи.

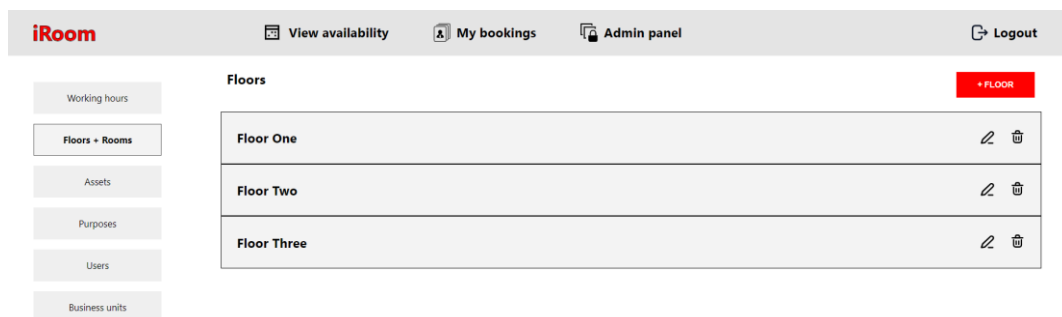


Рис. 17. Сторінка для управління сутностями поверхів офісу

Після того як користувач натисне на існуючий у таблиці запис поверху, система направить його на сторінку з кімнатами, що знаходяться на даному поверху.

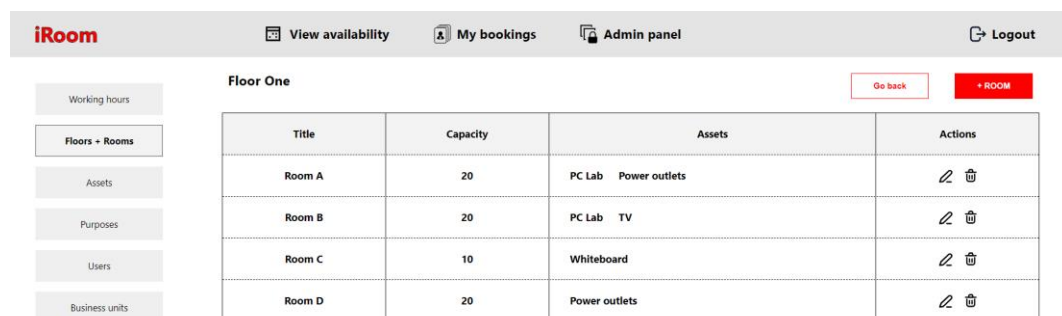


Рис. 18. Сторінка для управління сутностями переговорних кімнат

Користувач має можливість переглядати, редагувати, видаляти існуючі записи кімнат всередині даного поверху, а також створювати нову, натиснувши на відповідну клавiшу.

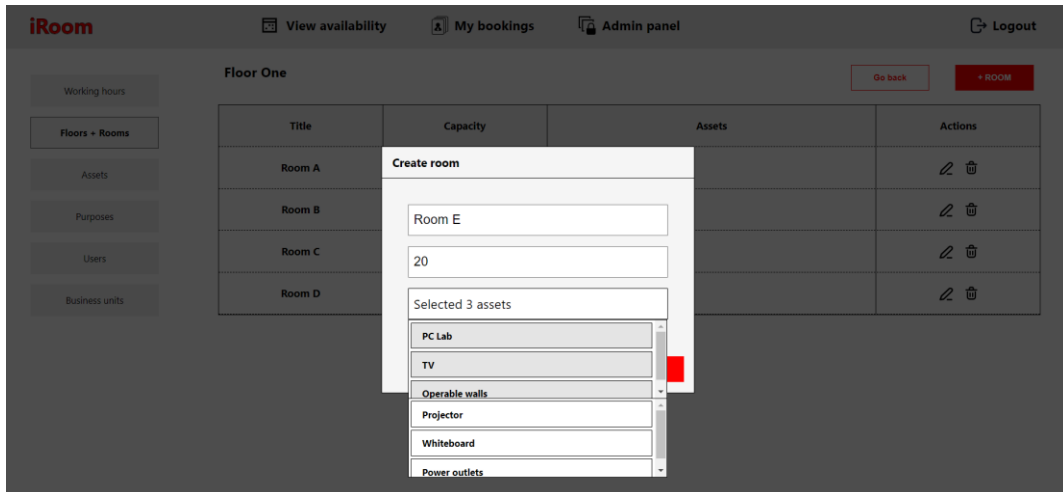


Рис. 19. Модальне вікно для додавання сутності переговорної кімнати

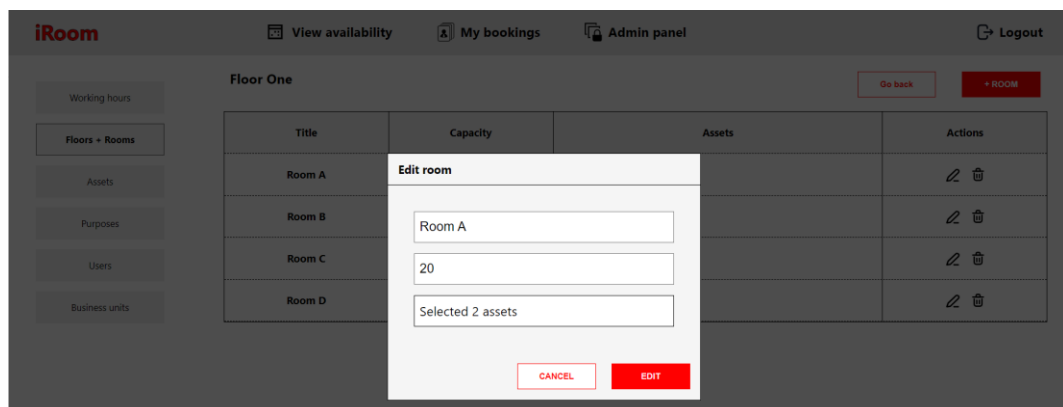


Рис. 20. Модальне вікно для редагування сутності переговорної кімнати

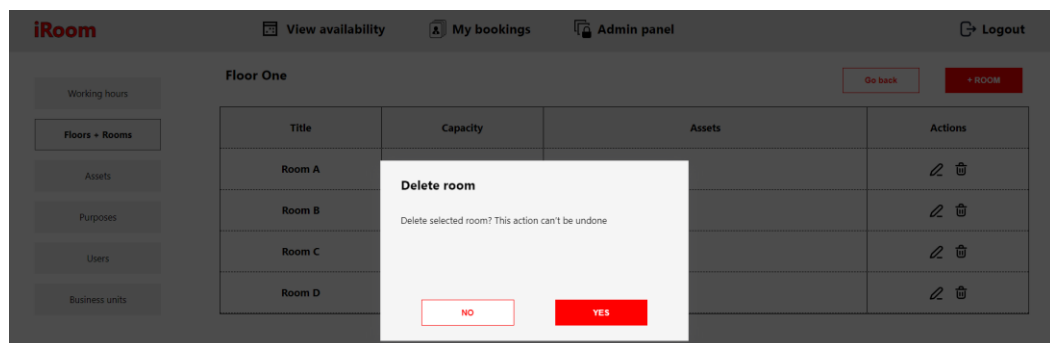


Рис. 21. Модальне вікно для підтвердження видалення сутності кімнати

9. Сторінка для управління обладнанням кімнат

Якщо користувач натисне на розділ «Assets», він потрапить на сторінку зі списком раніше створених сутностей обладнання переговорних кімнат. За допомогою клавiш для редагування, видалення та створення користувач має можливість вносити зміни, видаляти існуючі у системі обладнання, а також додавати нові.

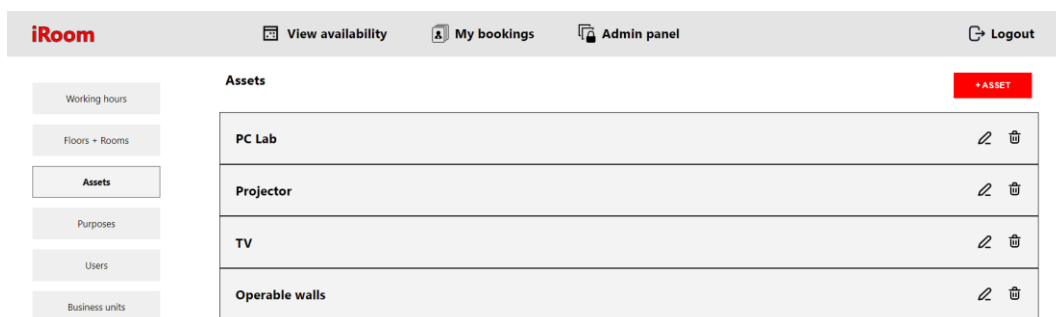


Рис. 22. Сторінка для управління сутностями обладнання кімнат

10. Сторінка для управління цілями зустрічей

При навігації до розділу «Purposes», користувач перейде на сторінку з таблицею існуючих цілей зустрічей. Аналогічно до функціональних можливостей, що можуть бути виконані на попередній сторінці, користувач має змогу створювати, редагувати та видаляти сутності цілей зустрічей, користуючись відповідними клавiшами.

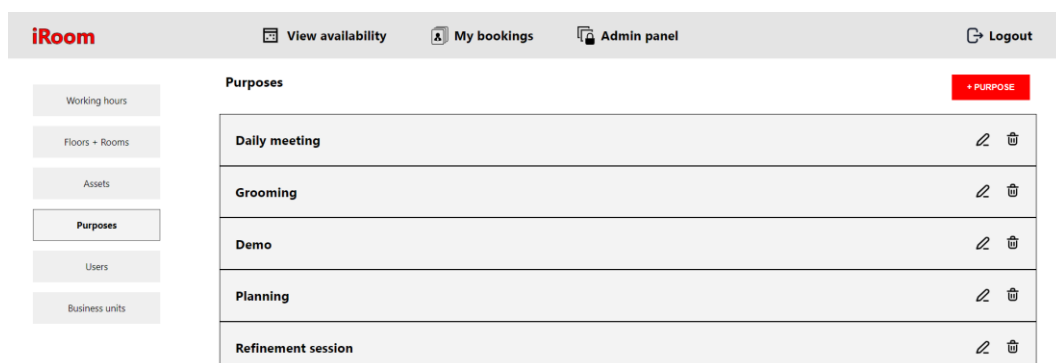
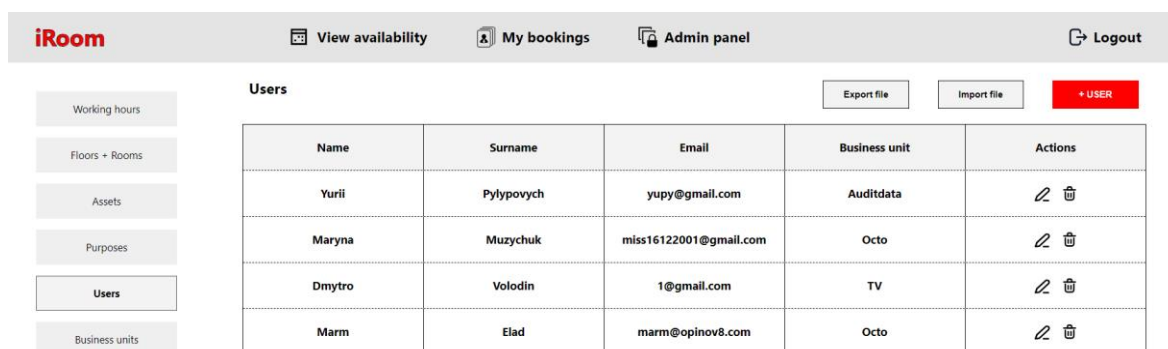


Рис. 23. Сторінка для управління сутностями цілей зустрічей

11. Сторінка для управління сутностями користувачів системи

Дана сторінка надає більше можливостей, що можуть бути виконані користувачем. Крім стандартних функцій створення, редагування та видалення сутностей, користувач має можливість ще виконати експорт та імпорт файлу CSV формату, у якому міститимуться записи про усіх користувачів.

Для того, щоб завантажити файл з користувачами із системи до своєї файлової системи, користувачеві потрібно натиснути на клавішу «Export file». При натисненні на «Import file» система відкриє файлову систему персонального комп'ютеру користувача, де він зможе обрати CSV-файл та завантажити його. У результаті усі записи з файлу будуть відображені у таблиці користувачів.



Name	Surname	Email	Business unit	Actions
Yurii	Pylypovych	yupy@gmail.com	Auditdata	 
Maryna	Muzychuk	miss16122001@gmail.com	Octo	 
Dmytro	Volodin	1@gmail.com	TV	 
Marm	Elad	marm@opinov8.com	Octo	 

Рис. 24. Сторінка для управління сутностями користувачів системи

Важливо зазначити, що зареєструватись та авторизуватись у системі матимуть змогу лише ті користувачі, чії електронні адреси (логіни) наведені у таблиці користувачів.

12. Сторінка для управління сутностями команд

При натисненні на розділ «Business units» користувач потрапить на сторінку з існуючими командами. На ній він може переглянути раніше створені команди, внести зміни до їхніх записів, видалити їх, а також додати нове об'єднання користувачів.

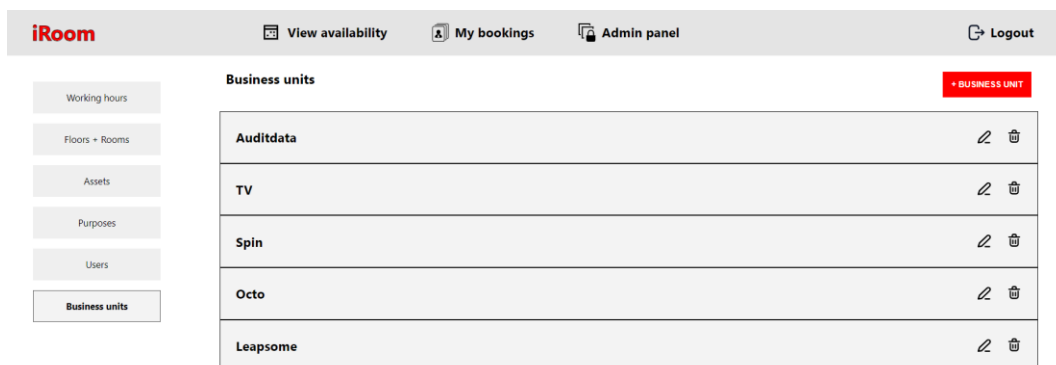


Рис. 25. Сторінка для управління сутностями команд

Для цього користувач повинен натиснути на кнопку «+ BUSINESS UNIT», у результаті чого система відобразить модальне вікно, де потрібно вказати назву команди та обрати їй колір з палітри, яким будуть позначатись бронювання для даної команди у таблиці завантаженості переговорних кімнат офісу.

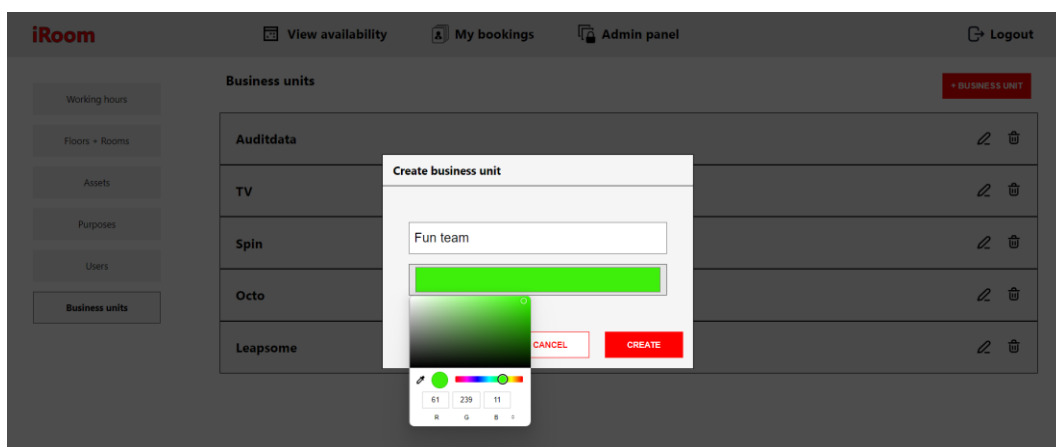


Рис. 26. Модальне вікно для створення нової команди